Sustainability and Business Practices

An Introduction to Project Management (MGMT-3105)

Martin Volkening

2026-10-13

Table of contents

Welcome **1**
Why a course that pairs sustainability with project management? . . . 1
Course map . . . 2
Course learning outcomes . . . 2
The running project . . . 2
How the book is organized . . . 3
Assessment (for the live course) . . . 3
A note on academic integrity . . . 3

1 Sustainability **5**
1.1 Learning Objectives . . . 5
1.2 Introduction: every gram, multiplied by a hundred thousand . . . 6
1.3 What sustainability means . . . 6
1.3.1 The three pillars and the triple bottom line . . . 7
1.4 The four points of the sustainability platform . . . 8
1.4.1 Sustainable design principles . . . 8
1.5 Recycling codes . . . 9
1.5.1 Try it yourself: which resin codes get recycled? . . . 10
1.6 Recyclable vs. biodegradable: a real tension . . . 10
1.7 Material reduction . . . 11
1.7.1 Worked Example 1: thinning the RC-car body . . . 12
1.7.2 Try it yourself: material reduction by hand . . . 13
1.7.3 Try it yourself: the material-choice decision . . . 13
1.8 Designing for recyclability . . . 15
1.9 The consumer's perspective . . . 16
1.9.1 Whole-life cost: cheaper to buy vs. cheaper to own . . . 16
1.10 Sustainability in project management . . . 18
1.10.1 The circular economy . . . 19
1.10.2 Try it yourself: a sustainability KPI snapshot . . . 20
1.11 Key Takeaways . . . 20
1.12 Case Study: Meridian R/C — Week 1 . . . 21
1.13 Practice Problems . . . 22
1.14 Further Resources . . . 25

2 Introduction to Project Management and Communication 27
2.1 Learning Objectives . . . 27
2.2 Introduction: only one project in four . . . 28
2.3 Project versus process . . . 28
2.4 The triple constraint . . . 29
2.4.1 Try it yourself: put a number on each side of the triangle 30
2.5 The five process phases . . . 31
2.5.1 The project manager's decision hierarchy . . . 32
2.6 Project communications . . . 32
2.6.1 The communication plan . . . 32
2.6.2 The communication model . . . 33
2.6.3 Communication channels grow faster than the team . . . 33
2.6.4 Worked Example 1: channels on the RC-vehicle project . 33
2.6.5 Try it yourself: channel growth and the cost of one more stakeholder . . . 34
2.6.6 Try it yourself: a merger by hand . . . 35
2.7 Listening: the mode we use most and understand least . . . 35
2.7.1 The listening filters . . . 36
2.7.2 Try it yourself: the tune-out budget . . . 36
2.8 Choosing a communication method . . . 37
2.8.1 Try it yourself: pick the method . . . 37
2.9 Performance reporting . . . 38
2.9.1 Try it yourself: which report answers the question? . . . 38
2.10 Key Takeaways . . . 39
2.11 Case Study: Meridian R/C — Week 2 . . . 39
2.12 Practice Problems . . . 40
2.13 Further Resources . . . 42

3 Project Scope, Charter, and Organizational Structures 43
3.1 Learning Objectives . . . 43
3.2 Introduction: nothing official without a charter . . . 44
3.3 Project selection: is it worth doing? . . . 44
3.3.1 Payback period . . . 44
3.3.2 Worked Example 1: payback on the RC-vehicle tooling investment . . . 44
3.3.3 Net present value . . . 45
3.3.4 Nominal vs. discounted payback for Example 1 . . . 46
3.3.5 Try it yourself: NPV of the project . . . 47
3.3.6 Try it yourself: a discounting table by hand . . . 48
3.4 The project charter . . . 49
3.4.1 Five inputs to a charter . . . 49
3.4.2 What a charter contains . . . 49
3.4.3 Why bother . . . 49
3.5 Stakeholders . . . 50
3.5.1 Try it yourself: score your own stakeholders onto the grid 50
3.5.2 Try it yourself: place each stakeholder on the grid . . . 52

3.6 The scope statement . . . 52
3.6.1 The scope management processes . . . 53
3.7 Scope creep . . . 53
3.7.1 Try it yourself: the cost of an "innocent" change . . . 53
3.8 Organizational culture . . . 54
3.9 Organizational structures . . . 54
3.9.1 Try it yourself: which structure is it? . . . 55
3.10 Key Takeaways . . . 56
3.11 Case Study: Meridian R/C — Week 3 . . . 57
3.12 Practice Problems . . . 58
3.13 Further Resources . . . 59

4 Leadership and the Project Manager 61
4.1 Learning Objectives . . . 61
4.2 Introduction: the project manager as mini-CEO . . . 61
4.3 Leadership versus management . . . 62
4.3.1 Bennis's four competencies of the new project leader . . . 62
4.4 The five leadership styles . . . 63
4.4.1 Try it yourself: pick the style . . . 64
4.4.2 Try it yourself: which quadrant is this team member in? . . . 65
4.4.3 Try it yourself: the leadership self-assessment . . . 66
4.5 Building the team . . . 67
4.5.1 Try it yourself: which stage is the team in? . . . 67
4.6 Motivating the team . . . 68
4.6.1 Worked Example 1: Herzberg — is this a hygiene problem or a motivator problem? . . . 68
4.6.2 Try it yourself: hygiene or motivator? . . . 69
4.6.3 Try it yourself: expectancy theory . . . 69
4.6.4 Try it yourself: expectancy by hand . . . 70
4.7 Emotional intelligence . . . 70
4.8 Key Takeaways . . . 71
4.9 Case Study: Meridian R/C — Week 4 . . . 71
4.10 Practice Problems . . . 72
4.11 Further Resources . . . 73

5 Work Breakdown Structures (WBS) 75
5.1 Learning Objectives . . . 75
5.2 Introduction: from a scope statement to something you can manage 75
5.3 The hierarchy . . . 76
5.4 Design principles . . . 76
5.4.1 The 100 % rule . . . 76
5.4.2 Worked Example 1: does this WBS roll up to 100 %? . . . 77
5.4.3 Mutually exclusive . . . 78
5.4.4 Plan outcomes, not actions . . . 78
5.4.5 Try it yourself: outcome or action? . . . 78
5.4.6 Try it yourself: does it roll up to 100 %? . . . 79

5.5 Level of detail . . . 79
5.5.1 Try it yourself: decomposition check . . . 80
5.5.2 Try it yourself: does it fit inside one reporting period? . . . 80
5.5.3 Try it yourself: is the package the right size? . . . 81
5.6 Why the WBS matters . . . 82
5.7 The WBS dictionary . . . 82
5.8 Work authorization and contracts . . . 83
5.8.1 Try it yourself: cost roll-up from the WBS . . . 83
5.8.2 Try it yourself: roll the hours up the tree . . . 84
5.9 Key Takeaways . . . 85
5.10 Case Study: Meridian R/C — Week 5 . . . 85
5.11 Practice Problems . . . 86
5.12 Further Resources . . . 87

6 Network Diagrams and Activity-on-Node (AON) 89
6.1 Learning Objectives . . . 89
6.2 Introduction: when can we launch? . . . 89
6.3 From WBS to precedence table . . . 90
6.3.1 Try it yourself: is this network valid? . . . 90
6.4 AON node anatomy . . . 91
6.5 Worked Example: the RC-vehicle sub-project network . . . 91
6.5.1 Forward pass (ES, EF) . . . 92
6.5.2 Backward pass (LS, LF) . . . 92
6.5.3 Float and the critical path . . . 92
6.5.4 Try it yourself: fill in the AON node boxes . . . 93
6.6 Three-point (PERT) duration estimates . . . 95
6.6.1 Try it yourself: PERT durations . . . 96
6.7 Shortening the critical path . . . 97
6.7.1 Try it yourself: recompute the critical path after a change 97
6.7.2 Try it yourself: which activities actually threaten the finish date? . . . 98
6.8 From network to Gantt chart . . . 100
6.9 The downside of network diagrams . . . 101
6.10 Key Takeaways . . . 101
6.11 Case Study: Meridian R/C — Week 6 . . . 102
6.12 Practice Problems . . . 104
6.13 Further Resources . . . 105

7 Risk Management 107
7.1 Learning Objectives . . . 107
7.2 Introduction: the known unknowns . . . 107
7.3 The four-stage process . . . 108
7.4 Risk identification and classification . . . 108
7.5 Qualitative analysis: the probability–impact matrix . . . 109
7.5.1 Try it yourself: colour the risks . . . 109
7.6 Quantitative analysis . . . 110

7.6.1 Worked Example 1: EMV of a single risk . . . 110
7.6.2 Try it yourself: decision tree — buy the copper now or later? . . . 111
7.6.3 Try it yourself: a risk register by hand . . . 112
7.7 Response strategies . . . 112
7.7.1 Try it yourself: choose the response . . . 113
7.8 Contingency reserves . . . 114
7.8.1 Try it yourself: does the mitigation pay for itself? . . . 114
7.9 FMEA as a risk tool . . . 115
7.9.1 The FMEA form . . . 115
7.9.2 Rating Severity, Occurrence, and Detection . . . 116
7.9.3 Calculating the Risk Priority Number . . . 117
7.9.4 Try it yourself: rank the failure modes . . . 118
7.10 Key Takeaways . . . 119
7.11 Case Study: Meridian R/C — Week 7 . . . 120
7.12 Practice Problems . . . 121
7.13 Further Resources . . . 123

8 Project Scheduling 125
8.1 Learning Objectives . . . 125
8.2 Introduction: from a network to a calendar . . . 125
8.3 Gantt charts . . . 126
8.4 Precedence relationships . . . 126
8.4.1 Try it yourself: which relationship is it? . . . 127
8.5 Probabilistic scheduling with PERT . . . 127
8.5.1 Worked Example 1: project duration and its spread . . . 128
8.5.2 Try it yourself: probability of hitting a target date . . . 128
8.5.3 Try it yourself: how likely is the target date? . . . 129
8.5.4 Try it yourself: simulate the schedule . . . 130
8.6 Crashing the schedule . . . 131
8.6.1 Try it yourself: least-cost crashing . . . 132
8.6.2 Try it yourself: crash slopes and the cheapest cut . . . 133
8.7 Fast-tracking versus crashing . . . 134
8.8 Key Takeaways . . . 134
8.9 Case Study: Meridian R/C — Week 8 . . . 135
8.10 Practice Problems . . . 136
8.11 Further Resources . . . 137

9 Cost Estimation and Budgeting 139
9.1 Learning Objectives . . . 139
9.2 Introduction: the number that decides whether the project happens 140
9.3 Where project cost comes from . . . 141
9.4 The eight cost types . . . 141
9.4.1 Direct labour cost . . . 141
9.4.2 Worked Example 1: fully-burdened cost of a work package 142
9.4.3 Try it yourself: cost and variance by hand . . . 142

9.4.4 Try it yourself: label each cost three ways 144
9.4.5 The dichotomies on the running project 145
9.5 Estimating methods and their accuracy 145
9.5.1 Try it yourself: parametric estimate and the accuracy band 146
9.5.2 Try it yourself: which estimating method? 147
9.6 What makes estimates good or bad 148
9.6.1 The cost of padding 148
9.6.2 Guidelines for estimating (cost and time) 149
9.7 Budgeting: top-down and bottom-up 149
9.8 Activity-based costing (ABC) 150
9.8.1 A preliminary ABC budget 150
9.8.2 Try it yourself: cost an activity, then a variance 150
9.9 The time-phased budget and the S-curve 152
9.9.1 Try it yourself: build the S-curve 152
9.10 Contingency reserves 153
9.10.1 Try it yourself: build the RC-vehicle project budget . . . 154
9.11 Key Takeaways 155
9.12 Case Study: Meridian R/C — Week 9 156
9.13 Practice Problems 157
9.14 Further Resources 159

10 Negotiation and Conflict Management 161
10.1 Learning Objectives 161
10.2 Introduction: the project manager negotiates every day 162
10.3 The negotiation mindset 162
10.4 Getting ready 163
10.4.1 The three questions — and who controls the room 163
10.5 BATNA, reservation point, and ZOPA 163
10.5.1 Worked Example 1: is there a ZOPA? 163
10.5.2 Try it yourself: reservation points from a BATNA 164
10.5.3 Try it yourself: reservation points and the ZOPA by hand 166
10.6 Language and tells 166
10.7 Conflict management 167
10.7.1 Try it yourself: what type of conflict is it? 167
10.7.2 Three views of conflict 167
10.7.3 Sources, ranked by intensity 168
10.7.4 Conflict-resolution methods (PMBOK) 168
10.7.5 Try it yourself: choose the resolution method 169
10.7.6 Try it yourself: score the supplier choice 169
10.7.7 Try it yourself: weighted supplier scoring by hand 170
10.8 Key Takeaways 171
10.9 Case Study: Meridian R/C — Week 10 171
10.10Practice Problems 172
10.11Further Resources 173

11 Resource Management 175

11.1 Learning Objectives . . . 175
11.2 Introduction: the optimism of the first schedule . . . 175
11.3 Resource loading and the histogram . . . 176
11.3.1 Try it yourself: build the histogram . . . 177
11.4 Resource leveling . . . 177
11.4.1 Worked Example 1: which task do you move? . . . 178
11.4.2 Try it yourself: which task do you move first? . . . 178
11.5 Time-constrained vs. resource-constrained . . . 179
11.6 Resource smoothing . . . 179
11.6.1 Try it yourself: smooth a resource within its float . . . 179
11.7 Splitting a task . . . 180
11.8 The multi-project resource problem . . . 180
11.8.1 Priority rules for allocating shared resources . . . 180
11.8.2 Try it yourself: which priority rule? . . . 181
11.8.3 Try it yourself: “40 hours is not one week’s work” . . . 181
11.8.4 Try it yourself: effective hours by hand . . . 183
11.9 Sustainability and resources . . . 183
11.10Key Takeaways . . . 183
11.11Case Study: Meridian R/C — Week 11 . . . 184
11.12Practice Problems . . . 185
11.13Further Resources . . . 186

12 Project Control Systems 187
12.1 Learning Objectives . . . 187
12.2 Introduction: from a plan to a control system . . . 187
12.3 The three curves . . . 188
12.4 The six core metrics . . . 189
12.4.1 Worked Example 1: the RC-vehicle project at week 8 . . . 190
12.4.2 Try it yourself: earned value by work package . . . 191
12.5 Forecasting . . . 191
12.5.1 Try it yourself: EAC scenarios . . . 192
12.5.2 Try it yourself: which forecast do you report? . . . 193
12.6 Reading the S-curve honestly . . . 194
12.6.1 Try it yourself: read the status . . . 194
12.7 Integrated change control . . . 194
12.8 Status reporting . . . 195
12.9 Key Takeaways . . . 195
12.10Case Study: Meridian R/C — Week 12 . . . 195
12.11Practice Problems . . . 196
12.12Further Resources . . . 197

13 Project Closeout and Termination 199
13.1 Learning Objectives . . . 199
13.2 Introduction: planning for extinction . . . 199
13.3 Types of termination . . . 200
13.3.1 Try it yourself: which way did the project end? . . . 201

13.4 The seven elements of closeout 201
13.4.1 What prevents effective closeout 201
13.4.2 Try it yourself: the seven elements in order 202
13.5 Project failure 202
13.5.1 Worked Example 1: continue or cancel? 202
13.5.2 Try it yourself: what counts in the decision? 203
13.5.3 Try it yourself: the cancel decision under uncertainty . . . 204
13.5.4 Try it yourself: continue or cancel, with an uncertain benefit 205
13.6 Lessons from spectacular failures 205
13.6.1 Try it yourself: the final project reconciliation 206
13.7 Closing out the RC-vehicle project 207
13.8 Key Takeaways 208
13.9 Case Study: Meridian R/C — Week 13 208
13.10Practice Problems 209
13.11Further Resources 210

Welcome

This book is the lecture-note companion for **MGMT-3105 – Sustainability and Business Practices**, a second-year course that introduces the philosophy of *sustainability* and the discipline of *project management* to students in electromechanical and mechanical engineering technology programs.

You are training for careers in the **automotive**, **food-processing**, and **defense** industries. In every one of those sectors you will, sooner than you expect, be handed a slice of a project: a line retrofit, a new product introduction, a validation campaign, a plant move. This course is about the structure that keeps such work on scope, on schedule, and on budget — and about doing it in a way that does not quietly borrow from the future to pay for the present.

Why a course that pairs sustainability with project management?

At first glance they look like two subjects stapled together. They are not. Every project consumes resources — materials, energy, money, human effort — and every project leaves something behind: a product, a process, a waste stream, a set of habits in the organization. **Project management is the set of tools that decides how those resources are spent, and sustainability is the lens that asks whether that spend was worth it beyond the next quarter.** A project manager who can build a work breakdown structure but cannot reason about lifecycle impact is only doing half the job.

What Is Project Management? (Association for Project Management): https://www.youtube.com/watch?v=Jk-JwtScIlw

What Is Sustainable Development? (FN-sambandet): https://www.youtube.com/watch?v=7V8oFI4GYMY

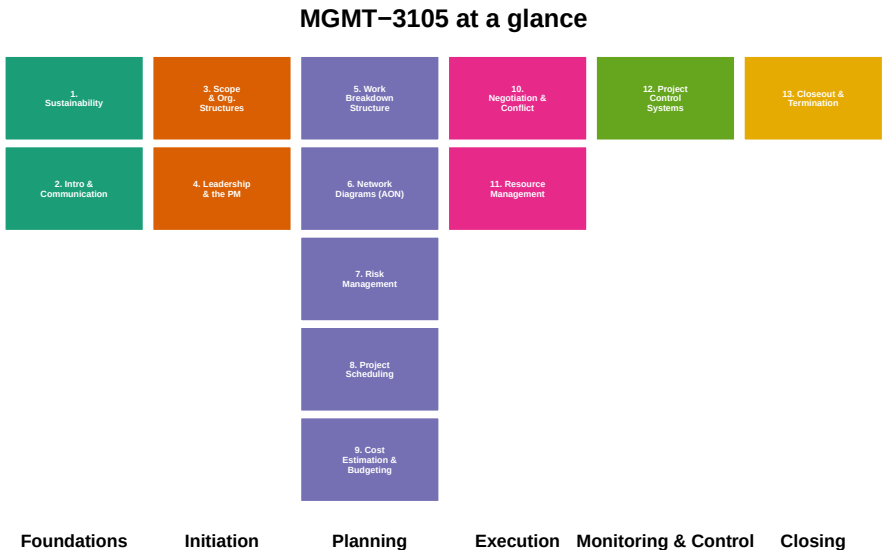


Figure 1: How the thirteen chapters of this book track the MGMT-3105 lecture sequence, grouped by project phase.

Course map

Course learning outcomes

On successful completion of MGMT-3105 you will be able to reliably:

1. Communicate the philosophy of project selection, including the Net Present Value method.
2. Communicate the theory of budgeting and scheduling.
3. Connect project-management concepts to real-world examples.
4. Identify the consequences to society of poor project planning.
5. Communicate the theory of progress management.
6. Discuss the problems of managing people and levelling allocated resources.
7. Communicate the organizational implications of closing a project.
8. Communicate negotiation theory.
9. Communicate effectively in a global environment.
10. Identify the use of sustainability in designs and in project management.

The running project

Throughout the book we return to one worked scenario, taken from the MGMT-3105 term project: your company designs and manages the launch of a **1/18-scale radio-controlled vehicle replica** for the toy market, then wins a contract to manufacture 100 000+ units for the following holiday season. You

choose the vehicle, the body and frame materials (steel, aluminium, or carbon fibre), and one of two Canadian motor/control-system suppliers. Every chapter uses a piece of that project — its scope, its WBS, its schedule, its budget, its risks, its closeout — so the tools connect to a single concrete thing rather than a dozen disconnected examples.

How the book is organized

Each chapter opens with measurable **learning objectives** and a real-world **hook**, develops the theory with plain-language explanations and LaTeX equations, and illustrates it with R-generated figures. Worked examples are shown step by step; wherever an example is a self-contained calculation, it is offered as a **live, editable R cell** you can run and modify in your browser. Collapsible `Practice` and `Discussion` blocks let you test yourself before revealing the answer, and each chapter closes with **curated YouTube videos** for a second explanation in a different voice.

Assessment (for the live course)

Table 1: MGMT-3105 grade breakdown

Component	Timing	Weight
Midterm test	Week 7	25 %
Final test	Week 14	25 %
Weekly quizzes	Weekly	12 %
Weekly case studies	Weekly	12 %
Assignment Phase A	Week 7	10 %
Assignment Phase B	Week 11	10 %
Assignment Phase C	Week 14	6 %

Quizzes and case studies require attendance. Tests are cumulative, closed-book with a handwritten aid sheet, and kit calculators are permitted. To pass, you need an overall grade of 50 % **and** completion of the mandatory work.

A note on academic integrity

Write your reports and answers in your own words. Whenever you take an idea or a passage from another author — including a classmate, a previous year's work, or an AI tool — acknowledge it with quotation marks where appropriate and a proper citation. If you are unsure how to credit shared work, ask **before** you submit. By submitting, you put your name to the integrity of the work.

Chapter 1

Sustainability

1.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Define** sustainability and sustainable design, and explain the three pillars (social, economic, ecological) and the "triple bottom line."
2. **Identify** the four points of the sustainability platform — recycling, bio-based/biodegradable polymers, material reduction, and design for recyclability — and give an engineering example of each.
3. **Interpret** the seven plastic recycling codes and distinguish post-consumer from post-industrial recyclate.
4. **Analyze** the trade-off between recyclability and biodegradability, using the corn-based PLA container case as the worked example.
5. **Calculate** the material, energy, and carbon consequences of a design decision (material substitution, wall-thickness reduction) for a production run.
6. **Evaluate** a product on whole-life cost, and explain why "cheaper to buy" and "cheaper to own" often disagree.
7. **Apply** a sustainability checklist to a project: supplier selection, contract clauses, impact assessment, circular-economy principles, and sustainability KPIs.

Before we begin: think of the last thing you threw away today

Where did its materials come from? How long did you use it before it became waste? Where is it now — landfill, recycling stream, incinerator? Could a different design decision, made years ago by an engineer you will never meet, have changed that answer? Keep the object in mind; we will come back to it.

1.2 Introduction: every gram, multiplied by a hundred thousand

Here is the situation that runs through this whole book. Your company designs and manages the launch of a **1/18-scale radio-controlled vehicle replica** for the toy market. If the prototype is accepted, you win a contract to build **100 000+ units** for the next holiday season. Early on, you must choose the body material — **steel, aluminium, or carbon fibre** — and the frame material — aluminium or carbon fibre.

That single choice is not just a strength-and-weight problem. Multiply it by 100 000 units and it becomes a decision about:

- how many tonnes of raw material leave the ground,
- how many megajoules of energy are spent turning that material into parts,
- how much scrap and packaging waste the line produces every shift,
- whether the finished toy can be recycled when a child outgrows it in three years,
- and whether your customer — a large retailer with its own public sustainability targets — will even put it on the shelf.

This is why **sustainability** belongs at the front of a project-management course, not tacked on at the end. The tools you meet in later chapters — scope, WBS, schedule, budget, risk — all decide how resources get spent. Sustainability is the question of whether that spend was worth it once you look past the launch date.

> **Note**
>
> **Industry snapshot.** General Motors' CAMI Assembly plant in Ingersoll, Ontario — an hour from where this course is taught — runs **landfill-free**: every waste stream from daily operations is reused, recycled, or converted to energy. That is not a marketing slogan bolted onto an existing plant; it is a set of thousands of small design and process decisions of exactly the kind this chapter is about.

The Story of Stuff (Annie Leonard) - the materials economy in 20 minutes: https://www.youtube.com/watch?v=9GorqroigqM

1.3 What sustainability means

The most quoted definition comes from the 1987 Brundtland report and is used almost word-for-word by the U.S. Environmental Protection Agency:

> Sustainability is *"meeting the needs of the present without compromising the ability of future generations to meet their own needs."*

The idea is older than the word. Thomas Jefferson expressed the same principle in financial terms: **do not leave a debt for future generations to pay.** Swap "debt" for "depleted resources, degraded ecosystems, and accumulated waste" and you have the modern version.

Sustainable design applies that principle to the things engineers actually produce. Wikipedia's working definition is useful: *the philosophy of designing physical objects, the built environment, and services to comply with the principles of social, economic, and ecological sustainability.* The intent is to *minimize negative environmental impact through skilful, sensitive design* while holding a **dynamic balance between economy and society**.

1.3.1 The three pillars and the triple bottom line

Sustainability is usually drawn as three overlapping domains. A decision is sustainable only where all three overlap: it has to be **socially** acceptable, **economically** viable, and **ecologically** sound at the same time.

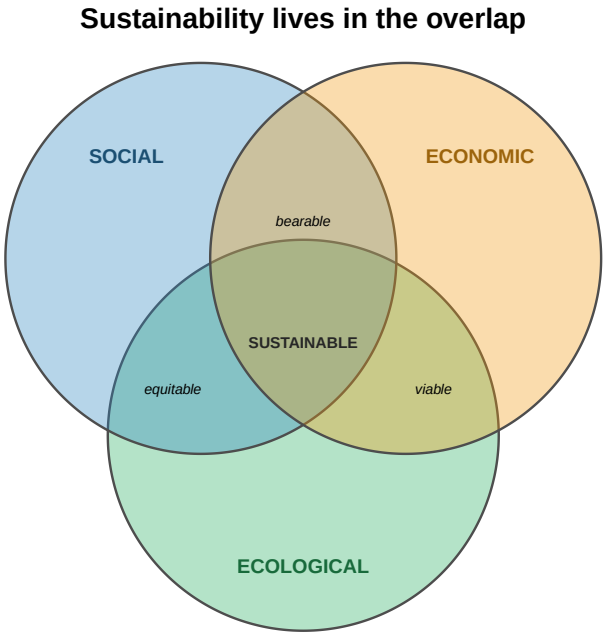


Figure 1.1: The three pillars of sustainability. Only decisions in the central overlap are genuinely sustainable; the pairwise overlaps (bearable, equitable, viable) each miss one dimension.

Discussion: which pillar does each of these decisions serve — and which does it ignore?

- A supplier offers motors 20 % cheaper, made in a plant with a poor safety record. *(economic yes; social no)*
- You switch the body to recycled aluminium, adding $0.40 per unit. *(ecological yes; economic maybe)*
- You keep an inefficient old assembly line running because retraining staff on a new one is disruptive. *(social short-term yes; ecological and economic long-term no)*

The point of the diagram is that a "win" on one pillar is not a win. You have to check all three.

1.4 The four points of the sustainability platform

For a product engineer, sustainability comes down to four levers:

1. **Recycling** — can the material be recovered and used again?
2. **Bio-based or biodegradable polymers** — can the material come from a renewable source, or return safely to the environment?
3. **Material reduction** — can the product do its job with less material in the first place?
4. **Designing for recyclability** — is the product built so that recovery is actually practical, not just theoretically possible?

The rest of the chapter takes these one at a time, then widens out to what sustainability means when you are managing a whole project rather than designing one part.

1.4.1 Sustainable design principles

The design literature expands the four levers into a set of working principles:

Principle	What it means in practice
Low-impact materials	Non-toxic, sustainably produced or recycled materials that need less energy to process.
Energy efficiency	Choose manufacturing processes — and design products — that require less energy over their life.
Emotionally durable design	Build things people want to keep. A product kept for ten years instead of three cuts consumption and waste by more than any recycling scheme.

Principle	What it means in practice
Design for reuse and recycling	Design for a commercial "afterlife" — but without compromising performance or service life to get there. It is a balancing act.

Notice the tension already built in: "make it last" and "make it recyclable" pull in opposite directions, and "use less material" can fight "make it durable." Good sustainable design is the art of resolving those tensions, not pretending they do not exist.

1.5 Recycling codes

Most rigid plastics carry one of seven **resin identification codes** — the number inside the chasing-arrows triangle. The code tells a sorting facility what polymer it is looking at. The three most commonly recycled are **PET (#1)**, **HDPE (#2)**, and **PP (#5)**.

Table 1.1: The seven resin identification codes.

Code	Abbreviation	Polymer	Typical products
1	PETE / PET	Polyethylene terephthalate	Drink bottles, food jars
2	HDPE	High-density polyethylene	Milk jugs, detergent bottles, piping
3	PVC / V	Polyvinyl chloride	Pipe, window frames, cable insulation
4	LDPE	Low-density polyethylene	Films, bags, squeeze bottles
5	PP	Polypropylene	Bumpers, battery cases, living hinges, t
6	PS	Polystyrene	Foam packaging, disposable cups, CD c
7	OTHER	Acrylic, polycarbonate, PLA, nylon, multi-layer	Eyewear, headlamp lenses, RC-car bodi

Two grades of recyclate matter to a manufacturer:

- **Post-industrial** (a.k.a. regrind): clean scrap from your own or a supplier's process — runners, sprues, start-up shots, off-spec parts. Easy to reuse because you know exactly what it is.
- **Post-consumer**: material that has been through a consumer's hands. It must be collected, sorted, washed, ground, and often re-pelletized, and it can carry metal, labels, food residue, and other polymers. Much harder to work with, but it is the stream that actually diverts waste from landfill.

Think about it: your RC-car body is moulded from a glass-filled ABS/PC blend. Which recycling code does it carry, and what does that tell you?

It carries **#7 (OTHER)** — any multi-polymer blend or filled material that is not one of codes 1–6 lands here. In practice, #7 is rarely recycled through curbside programs: the blend cannot be melted back into a single clean polymer. If recyclability matters for this product, that is a signal to reconsider the material *before* tooling is cut, not after.

1.5.1 Try it yourself: which resin codes get recycled?

How well does each product's plastic recycle?

Use the code table above. Pick one class for every product.

Categories: Recycled easily / Sometimes / Rarely

1. Soft-drink bottle (PET, #1) _____
2. Laundry-detergent jug (HDPE, #2) _____
3. Yoghurt tub (PP, #5) _____
4. Honey squeeze bottle (LDPE, #4) _____
5. Grocery-bag film (LDPE, #4) _____
6. Window frame (PVC, #3) _____
7. Foam takeaway clamshell (PS, #6) _____
8. Polycarbonate eyewear lens (#7) _____

Answers: 1 Recycled easily; 2 Recycled easily; 3 Recycled easily; 4 Sometimes; 5 Sometimes; 6 Rarely; 7 Rarely; 8 Rarely.

1.6 Recyclable vs. biodegradable: a real tension

Years ago the plastics industry pursued two waste strategies at once — **recycling** and **biodegradability** — and they turned out to be partly opposed. If you make a part from recyclate and want it to last (think plastic lumber, or a pallet), the last thing you want is biodegradable material mixed in, causing the part to fail early. Contaminate a recycling stream with enough compostable polymer and the whole batch can become unusable.

The industry chose recycling. But with fossil-fuel prices and supply security back in the spotlight, **biopolymers** have returned. Some of the earliest plastics were bio-based (cellulosics, natural rubber); many new ones are built from **corn or potato starch**. **Polylactic acid (PLA)** is made from corn starch and behaves a lot like PET, but it is harder to process and degrades faster at elevated temperature.

And "biodegradable" has an asterisk. Degradation is done by **microbes**, which need **oxygen and moisture**. A modern landfill is compacted, dry, and low-oxygen — almost nothing degrades well there. You can dig up decades-old newspaper and still read it.

> **!** Important
>
> **Case study — EcoPlast Packaging (MGMT-3105, Week 1).** EcoPlast launched corn-based PLA food containers. Then: customers found them brittle in the cold and warped in heat; recyclers reported PLA contaminating conventional-plastic batches; consumers wanted eco-friendly *and* the same durability as regular plastic; and environmental groups argued the containers mostly ended up in landfill, where they did not meaningfully biodegrade — calling it "green marketing."

Discussion (Week 1 case): was it unreasonably risky for EcoPlast to trade durability for sustainability, and what would you change?

Model answer. A product that fails in real-world use creates *more* waste, erodes consumer trust, and undermines the point of sustainability — so yes, the trade-off as executed was unreasonably risky. Consumers share *some* responsibility for unrealistic expectations, but the company had a duty to manage those expectations and to test properly before launch. Rushing to market compromised quality.

What to change: (1) real R&D and performance testing across the actual temperature and handling range before launch; (2) an accurate **caution** label ("not suitable for hot liquids") — a danger label would be excessive; (3) consumer education on correct disposal (industrial compost, not curbside or landfill); (4) stop marketing the product as fully "sustainable" until there is evidence of an environmental benefit in real disposal conditions — otherwise the claim is exposed to a misleading-marketing challenge.

The Life Cycle of a T-Shirt (TED-Ed) - why 'where does it go?' is the hard question: https://www.youtube.com/watch?v=BiSYoeqb_VY

1.7 Material reduction

Reducing the material that goes into a product — at the **design stage**, by thinning walls, removing bosses, right-sizing ribs, or substituting a lighter material — pays off several times over:

- the product **costs less** (less material bought per unit),
- there is **less to recycle or dispose of** at end of life,
- **more material is left** for other uses,
- and **less energy** is spent producing *and transporting* every unit.

The catch: you cannot thin your way past the product's functional requirements. Material reduction is enabled by better material properties, better processes, and better analysis — not by wishful thinking.

1.7.1 Worked Example 1: thinning the RC-car body

Your baseline RC-car body is injection-moulded ABS with a mass of **48 g**. Simulation says you can drop the nominal wall from 2.5 mm to 2.1 mm and still pass the drop test, cutting body mass to **41 g**. The contract run is **120 000 units**. ABS costs **$3.10/kg**, its production embodied energy is about **90 MJ/kg**, and its cradle-to-gate carbon intensity is about **3.4 kg CO2e/kg**.

Live cell — change the mass reduction, the run size, or the material's cost and energy factors.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: material-reduction-example-01
mass_before_g <- 48
mass_after_g  <- 41
units         <- 120000
abs_cost_per_kg   <- 3.10
abs_energy_MJ_kg  <- 90
abs_co2e_kg_kg    <- 3.4

saved_kg <- (mass_before_g - mass_after_g) / 1000 * units

cost_saved   <- saved_kg * abs_cost_per_kg
energy_saved <- saved_kg * abs_energy_MJ_kg
co2_saved    <- saved_kg * abs_co2e_kg_kg

fmt <- function(x) format(round(x), big.mark = " ", scientific = FALSE, trim = TRUE)

# Chart: CO2e avoided grows with the production run
runs <- seq(0, 500000, length.out = 100)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(runs / 1000, (mass_before_g - mass_after_g) / 1000 * runs * abs_co2e_kg_kg / 1000
     type = "l", lwd = 4, col = "#16a085", xlab = "Production run (thousand units)", yl
     main = "Carbon avoided vs. production run")
abline(v = units / 1000, lty = 2, col = "#bdc3c7")
points(units / 1000, co2_saved / 1000, pch = 19, cex = 2.2, col = "#e74c3c")

cat(sprintf("%s kg of material saved over the run\n", fmt(saved_kg)))
cat(sprintf("Mass removed per unit  %s g\n", mass_before_g - mass_after_g))
cat(sprintf("Material cost saved  $%s\n", fmt(cost_saved)))
cat(sprintf("Production energy saved  %s MJ\n", fmt(energy_saved)))
cat(sprintf("Equivalent electricity  ~%s kWh\n", fmt(energy_saved / 3.6)))
cat(sprintf("Cradle-to-gate CO2e avoided  %s kg (~%.1f t)\n", fmt(co2_saved), co2_saved
```

A 7 g decision, taken once on a screen, moves almost a tonne of material and several tonnes of CO2e once it is multiplied by the production run — before counting the lighter shipping weight.

1.7.2 Try it yourself: material reduction by hand

Do the same calculation for a part of your own. Work out the material, cost, energy and carbon consequences of thinning the part over the whole run. Each team picks its own **data set**.

Material reduction over a production run

Mass saved per unit x units, then multiply by each factor.

- Part: Washer-fluid bottle (HDPE)
- Mass before / after: 62 g / 54 g
- Production run: 300 000
- Material cost: $1.9 / kg
- Embodied energy: 80 MJ / kg
- Carbon intensity: 1.9 kg CO2e / kg
- Material saved over the run: _____ kg *(to 0 decimal places)*
- Material cost saved: $_____ *(to 0 decimal places)*
- Production energy saved: _____ MJ *(to 0 decimal places)*
- Cradle-to-gate CO2e avoided: _____ kg *(to 0 decimal places)*

Show the worked solution

Saved mass = (mass before - mass after) / 1000 x units, in kg. Each of cost, energy and carbon is that mass times its per-kg factor. For data set 1: 8 g x 300 000 = 2 400 kg, which at $1.90/kg is $4 560, at 80 MJ/kg is 192 000 MJ and at 1.9 kg CO2e/kg is 4 560 kg CO2e.

1.7.3 Try it yourself: the material-choice decision

Now compare the three candidate body materials head to head. The live cell below holds rough figures for **steel**, **aluminium**, and **carbon-fibre composite** bodies of the mass each material would need for equal stiffness. Change the numbers, add a recycled-content option, or change the run size and see what happens to cost, energy, and carbon.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: material-choice-compare-01
# Rough per-unit body mass for equal stiffness, and cradle-to-gate factors.
material <- c("Steel", "Aluminium", "Carbon-fibre")
mass_g   <- c(140,      70,          38)          # grams per body
cost_kg  <- c(1.20,     3.00,        28.00)       # $ per kg
energy_kg<- c(25,       155,         300)         # MJ per kg (primary)
co2_kg   <- c(2.0,      11.0,        24.0)        # kg CO2e per kg

units <- 120000

kg_total     <- mass_g / 1000 * units
cost_total   <- kg_total * cost_kg
energy_total <- kg_total * energy_kg
co2_total    <- kg_total * co2_kg

result <- data.frame(
  Material     = material,
  Body_mass_g  = mass_g,
  Material_t   = round(kg_total / 1000, 1),
  Cost_usd     = round(cost_total),
  Energy_MWh   = round(energy_total / 3600, 1),
  CO2e_t       = round(co2_total / 1000, 1)
)

money <- function(x) paste0("$", format(round(x), big.mark = " ", scientific = FALSE, t

cat("Lowest cradle-to-gate CO2e:", material[which.min(co2_total)], "\n")
cat("Lightest body  ", material[which.min(mass_g)], "\n", sep = "")
cat("Lowest material cost  ", material[which.min(cost_total)], "\n", sep = "")
for (i in seq_along(material))
  cat(sprintf("%s  %s | %s MWh | %s t CO2e\n", material[i], money(cost_total[i]),
              format(result$Energy_MWh[i], big.mark = " "), format(result$CO2e_t[i], bi

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(result$CO2e_t, names.arg = material, col = c("#7f8c8d", "#2980b9", "#8e44ad"),
        ylab = "Cradle-to-gate CO2e (tonnes)", main = "Carbon footprint of one producti
```

What the numbers usually show — and why the "greenest" answer is not obvious

Carbon fibre gives the lightest body, which cuts shipping energy and, in a real vehicle, fuel over the whole service life. But its *production* energy and carbon per kilogram are the highest of the three, and it is the hardest to recycle (code #7, thermoset matrix). Steel is cheap and highly recycled but heavy. Aluminium sits in between and is excellent if you can get **recycled** stock — secondary aluminium uses roughly 5 % of the energy of primary.

For a *toy* that is shipped once and never driven under its own power for thousands of kilometres, the use-phase fuel saving from carbon fibre never arrives — so the decision tilts back toward recycled aluminium or steel. The lesson: **the right material depends on where the impact actually lands in the life cycle.**

1.8 Designing for recyclability

Recyclability is designed in, not hoped for. The checklist a plastics engineer can actually influence:

- Make the assembly from a **single material**, or from **compatible** materials that can be recycled together.
- Make it **easy to disassemble**.
- Prefer **snap-fits and press-fits** over screws and metal fasteners.
- **Reduce the part count.**
- **Avoid labels** (or use same-polymer labels / in-mould labelling).
- Keep **thermosets** out of thermoplastic assemblies — use thermoplastic elastomers instead of rubber seals where you can.

You may not have the final word on a design, but you can always **offer the greener alternative**. And the mindset has shifted: products used to be designed for *end of life* — last for period X, then done. Now good products are designed for **next life**: what can this material become *after* its first use?

Try it: redesign the RC-car battery door for recyclability

Current design: ABS door, two stainless M2 screws, a printed polyester label, a moulded-in TPE gasket (thermoset-like behaviour), a steel spring.

One redesign

- Replace the two screws with a **living hinge + snap latch** moulded into the door (same ABS) — removes two dissimilar-metal fasteners and a driver operation on the line.
- Replace the adhesive polyester label with **in-mould labelling** in an ABS-compatible film, or mould the text into the part.
- Replace the steel spring with a **moulded ABS cantilever spring** if the cycle life allows.
- Keep the TPE gasket but make it a **removable press-in** part in a recycling-compatible grade, or switch to a moulded-in flexible ABS lip if the seal spec allows.

Result: the door goes from five materials and two fasteners to essentially one polymer, snaps apart by hand, and carries no adhesive.

1.9 The consumer's perspective

Sustainable design has to survive contact with real buyers. Market research is consistent on two points:

1. Environmentally conscious consumers **will pay marginally more** and **accept marginally lower fit and finish** for a product that is genuinely more environmentally friendly.
2. They have **zero tolerance** for a drop in **performance/functionality** or in **service life**. They bought the product to do a job. If a "sustainable" design wears out sooner, it just gets replaced sooner — consuming more resources and defeating the purpose.

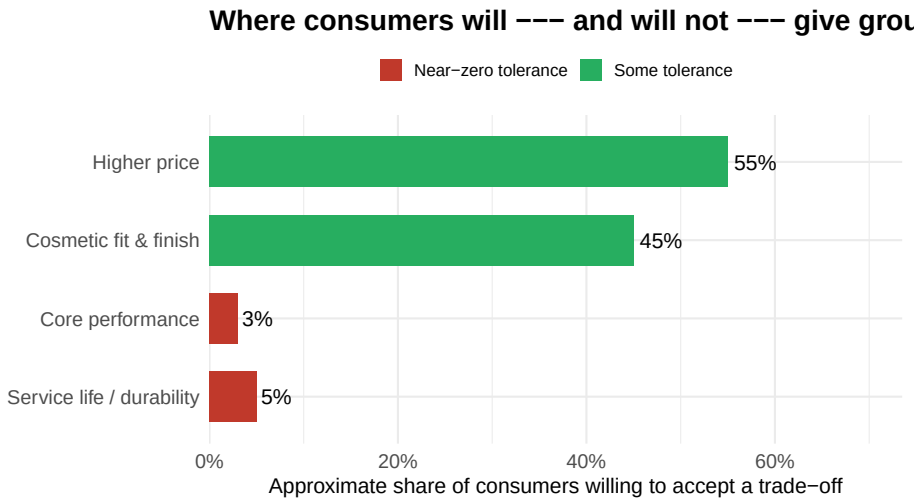


Figure 1.2: Consumer tolerance for trade-offs made in the name of sustainability: some room on price and cosmetics, essentially none on how well the product works or how long it lasts.

1.9.1 Whole-life cost: cheaper to buy vs. cheaper to own

This is where **planned obsolescence** enters. A business may have a commercial reason to limit a product's life — it protects repeat sales. Sustainable design pushes the other way: make it last. The buyer's own wallet is often on the side of durability, even when the sticker price is higher.

The live cell compares a **cheap** version of a product (low price, short life, replaced repeatedly) with a **durable** version (higher price, lasts the whole period). Change the prices, lifetimes, and horizon and watch which one wins on **total cost of ownership** — and on **units sent to landfill**.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: whole-life-cost-01
horizon_years <- 10

# Cheap option
cheap_price     <- 18
cheap_life_yrs  <- 2

# Durable option
durable_price     <- 42
durable_life_yrs  <- 10

cheap_purchases   <- ceiling(horizon_years / cheap_life_yrs)
durable_purchases <- ceiling(horizon_years / durable_life_yrs)

cheap_tco   <- cheap_purchases   * cheap_price
durable_tco <- durable_purchases * durable_price

better <- if (durable_tco < cheap_tco) "Durable" else if (durable_tco > cheap_tco) "Cheap" else "

tco <- c(Cheap = cheap_tco, Durable = durable_tco)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(tco, col = c("#e67e22", "#16a085"), ylim = c(0, max(tco) * 1.2),
              ylab = "Total cost of ownership ($)",
              main = paste("Cost over", horizon_years, "years"))
text(bp, tco, labels = paste0("$", tco), pos = 3, cex = 1.3)

cat(sprintf("Lower total cost: %s (saves $%s)\n", better, abs(cheap_tco - durable_tco)))
cat(sprintf("Cheap option  %d purchase(s) x $%s = $%s\n", cheap_purchases, cheap_price, cheap_tco
cat(sprintf("Durable option  %d purchase(s) x $%s = $%s\n", durable_purchases, durable_price, dur
cat(sprintf("Units to landfill, cheap  %d\n", cheap_purchases))
cat(sprintf("Units to landfill, durable  %d\n", durable_purchases))
```

Where does the cheap option actually win?

Shorten the horizon (you only need the product for a year), or make the durable version's premium extreme relative to its extra life, and "cheap" wins on cost. But the **waste count** almost always favours durable: five landfilled units versus one. Whole-life thinking means putting *both* numbers — dollars and units — in front of the decision-maker, not just the sticker price.

1.10 Sustainability in project management

Everything above was about designing a *part*. When you are running a *project*, sustainability becomes a set of management actions spread across the whole life cycle. The MGMT-3105 checklist:

Procurement and contracts

- Choose **eco-friendly and ethical suppliers**; audit them for compliance.
- Put **sustainability clauses** in contracts.
- Perform an **Environmental & Social Impact Assessment (ESIA)**.

Design and materials

- Minimize raw-material use; prioritize **recyclable, renewable, and local** resources.
- Design for **energy and water efficiency**.
- Apply **circular-economy** principles: reuse, refurbish, recycle.
- Plan the **product/service lifecycle** — what happens to it after the project ends.

Operations

- Reduce **carbon emissions** from transport and operations.
- Use **digital tools** to cut paper and travel.
- Run proper **waste management**: segregation, recycling, safe disposal.

People and governance

- Safeguard **health, safety, and well-being** of the team and the community.
- Support **social responsibility**: local hiring, fair wages, training.
- Track **sustainability KPIs**: carbon footprint, waste-reduction %, energy saved.
- Monitor suppliers and contractors; **identify and mitigate** environmental and social risks; run **sustainability audits** when needed.

Try it yourself: where does each action belong on the checklist?

The MGMT-3105 checklist has four groups. Pick one for every action.

Categories: Procurement and contracts / Design and materials / Operations / People and governance

1. Add a clause requiring suppliers to report CO2e per part _____
2. Require a sustainability audit before approving a new supplier _____
3. Redesign the housing as a single polymer so it can be recycled _____
4. Specify 30 % recycled content in the packaging _____
5. Ship by consolidated sea freight instead of air _____
6. Replace paper travellers on the line with tablets _____

7. Segregate scrap aluminium and plastic into labelled bins _____
8. Report the waste-diversion rate on the monthly KPI sheet _____
9. Hire apprentices from the local community college _____
10. Brief the team on the new recycling station's safety rules _____

Answers: 1 Procurement and contracts; 2 Procurement and contracts; 3 Design and materials; 4 Design and materials; 5 Operations; 6 Operations; 7 Operations; 8 People and governance; 9 People and governance; 10 People and governance.

1.10.1 The circular economy

The circular economy is the alternative to the linear **take — make — use — dispose** model. Instead, material is kept in use through loops of increasing size: maintain and reuse first, then refurbish and remanufacture, then recycle, with disposal as the last resort.

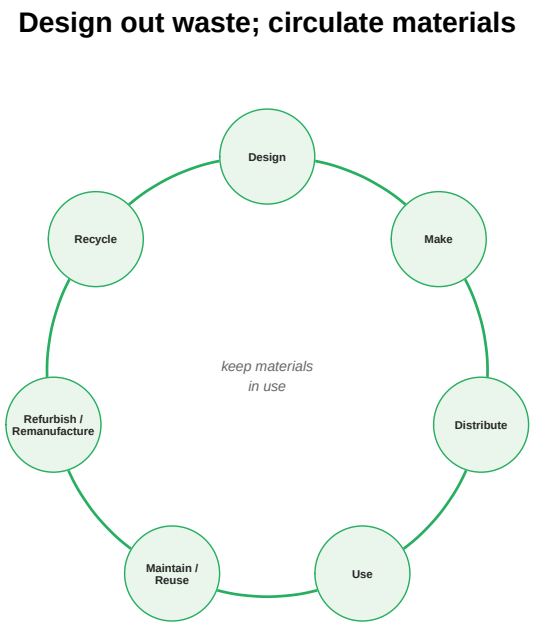


Figure 1.3: The circular economy: keep materials circulating through the tightest loop that still works. Inner loops (maintain, reuse) retain the most value; recycling is the outermost loop before disposal.

Explaining the Circular Economy (Ellen MacArthur Foundation): https://www.youtube.com/watch?v=zCRKvDyyHmI

1.10.2 Try it yourself: a sustainability KPI snapshot

Sustainability only gets managed if it gets measured. The live cell computes a few of the KPIs from the checklist for the RC-vehicle project: carbon footprint per unit, waste-diversion rate, and recycled-content share. Adjust the inputs to your own project assumptions.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: sustainability-kpi-01
units            <- 120000

# Carbon
material_co2_t   <- 71      # tonnes CO2e, cradle-to-gate materials
process_co2_t    <- 18      # tonnes CO2e, moulding + assembly energy
transport_co2_t  <- 26      # tonnes CO2e, inbound + outbound freight

# Waste
process_waste_t  <- 9.0     # tonnes of scrap + packaging generated
waste_diverted_t <- 8.1     # tonnes reused / recycled / to-energy

# Materials
recycled_input_t <- 15.5    # tonnes of recycled feedstock
total_input_t    <- 62.0    # tonnes of material input

total_co2_t <- material_co2_t + process_co2_t + transport_co2_t
diversion   <- 100 * waste_diverted_t / process_waste_t

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(c(Materials = material_co2_t, Process = process_co2_t, Transport = transp
              col = c("#2c3e50", "#2980b9", "#16a085"), ylab = "t CO2e", main = "Carbon

cat(sprintf("Waste KPI status: %s\n", if (diversion >= 90) "PASS" else "!!BELOW TARGET
cat(sprintf("Waste diversion rate  %.1f %% (target >= 90 %%)\n", diversion))
cat(sprintf("Landfilled waste  %.2f t\n", process_waste_t - waste_diverted_t))
cat(sprintf("Carbon footprint  %.1f t CO2e\n", total_co2_t))
cat(sprintf("Carbon per unit  %.3f kg CO2e\n", total_co2_t * 1000 / units))
cat(sprintf("Recycled content  %.1f %% of input\n", 100 * recycled_input_t / total_inpu
```

1.11 Key Takeaways

- **Sustainability** = meeting present needs without compromising future generations' ability to meet theirs. **Sustainable design** applies that

across social, economic, and ecological pillars at once — a "win" on one pillar is not a win.

- The four levers for a product engineer: **recycle, bio-source/biodegrade, reduce material, design for recyclability.**
- The **seven resin codes** tell a sorter what a plastic is; PET (#1), HDPE (#2), and PP (#5) recycle well, #7 (blends/PLA/filled) rarely does. **Post-consumer** recyclate diverts real waste but is hard to process; **post-industrial** regrind is easy.
- **Recyclable and biodegradable can conflict**, and "biodegradable" does little in a dry, compacted landfill — the EcoPlast case shows what happens when that asterisk is ignored.
- A small **material-reduction** decision, multiplied by the production run, moves tonnes of material, energy, and CO2e. The "greenest" material depends on **where in the life cycle the impact lands** — use phase vs. production vs. end of life.
- Consumers give ground on **price and cosmetics** but not on **performance or durability**; **whole-life cost** usually favours the durable design on both dollars and landfill count.
- In **project management**, sustainability is a checklist across procurement, design, operations, and governance — backed by **circular-economy** loops and tracked with **KPIs** (carbon per unit, waste diversion, recycled content).

1.12 Case Study: Meridian R/C — Week 1

CASE STUDY FILE — Week 1: The Body & Frame Material Decision

Meridian R/C Vehicles is racing a deadline neither you nor your sponsor can move. Kestrel Retail Group, a national toy retailer, has told Daniel Cho, Meridian's VP of Product, that if the **Meridian Trailblazer** — a 1/18-scale rally-truck replica — passes durability testing and toy-safety certification, Kestrel's Category Buyer, Renata Ford, will place a **100,000+ unit** production order in time for next year's holiday season. Before a single tool is cut, you and your manufacturing engineer, **Priya Nandakumar**, have to settle the one decision that shapes every cost, mass, and end-of-life number that follows: what is the Trailblazer's body and frame made of?

Priya frames it as a three-way trade study. Steel bodies would weigh about 140 g each and cost roughly $1.20/kg in raw material — cheap, but steel is the heaviest of the three candidates and, for this die-cast toy geometry, the least attractive to recycle back into value. Carbon-fibre composite is the opposite extreme: a 38 g body, the best strength-to-weight ratio by far, but at roughly $28/kg in material alone — and once a thermoset carbon-

fibre part reaches end of life in a child's toy box, there is no practical way to recycle it. Aluminium lands in the middle on every axis: about 70 g per body, $3/kg, and — critically — genuinely recyclable at scale.

Candidate	Body mass	Material cost	End-of-life recyclability
Steel	140 g	$1.20/kg	Heavy; low recyclate value for this design
Aluminium (chosen)	70 g	$3.00/kg	High — genuinely recyclable
Carbon fibre	38 g	$28.00/kg	Very poor — thermoset, effectively unrecyclable

You, Priya, procurement lead **Marcus Webb**, and quality engineer **Sofia Reyes** agree: aluminium is the call for both the body and the frame. It is not the lightest option and it is not the cheapest raw material, but it is the only one that clears the bar on cost *and* recyclability at once — exactly the kind of "no pillar loses" decision the sustainability framework asks for. Carbon fibre does not disappear from the story, though. **Fibro Source**, a carbon-fibre supplier in Sherbrooke, Quebec that is mid-restructuring after a recent French acquisition, stays on the supplier list — not for the core prototype, but as an optional carbon-fibre trim and spoiler package Kestrel could offer later as a premium upsell. Daniel Cho signs off: aluminium body and frame, carbon fibre trim only if the business case for it ever appears.

1.13 Practice Problems

Problem 1: Packaging substitution (food industry)

A food-processing client ships 2.4 million meal trays a year. The current tray is 22 g of black CPET (#1, but often rejected by sorters because black plastic is invisible to optical sorting). A proposed natural-coloured PP (#5) tray weighs 19 g, costs $0.006/g versus $0.005/g for the CPET, and is accepted by the client's regional recycler.

(a) What is the annual change in material mass and material cost?
(b) Give two non-cost arguments for switching, and one argument against.

Solution

```
trays        <- 2.4e6
mass_cpet_g  <- 22
mass_pp_g    <- 19
cost_cpet    <- 0.005
cost_pp      <- 0.006

mass_change_kg <- (mass_pp_g - mass_cpet_g) / 1000 * trays
cost_change    <- (mass_pp_g * cost_pp - mass_cpet_g * cost_cpet) * trays

cat("Annual material mass change :", format(round(mass_change_kg), big.mark = " "), "kg",
    "(a reduction)\n")
cat("Annual material cost change : ",
    sprintf("$%s", format(round(cost_change), big.mark = " ")),
    " (an increase)\n", sep = "")
```

```
Annual material mass change : -7 200 kg (a reduction)
Annual material cost change : $9 600 (an increase)
```

(a) About 7 200 kg less plastic per year, but roughly $9 600/yr more spent on material because PP costs more per gram here and the 3 g saving does not fully offset it.

(b) *For:* the natural-coloured PP is actually recovered by the recycler (black CPET usually is not), and PP trays tolerate reheating better. *Against:* higher material spend, and a tooling change is needed — capital plus a qualification run.

Problem 2: Recycled-aluminium body (automotive/toy)

Switching the RC-car body from primary to **recycled** aluminium keeps the mass at 70 g/unit but changes the cradle-to-gate energy from 155 MJ/kg to 8 MJ/kg and the carbon from 11 kg CO2e/kg to 0.6 kg CO2e/kg. Recycled stock costs $0.25/kg more. Run size 120 000 units.

Estimate the energy saved, the CO2e avoided, and the extra material cost.

Solution

```
mass_kg   <- 70 / 1000
units     <- 120000
kg_total  <- mass_kg * units

energy_saved <- kg_total * (155 - 8)
co2_saved    <- kg_total * (11 - 0.6)
extra_cost   <- kg_total * 0.25

cat("Aluminium in the run :", format(round(kg_total), big.mark = " "), "kg\n")
cat("Production energy saved :", format(round(energy_saved), big.mark = " "), "MJ",
    sprintf("(~%s kWh)\n", format(round(energy_saved / 3.6), big.mark = " ")))
```

```
cat("CO2e avoided :", format(round(co2_saved), big.mark = " "), "kg",
    sprintf("(~%.1f t)\n", co2_saved / 1000))
cat("Extra material cost :",
    sprintf("$%s\n", format(round(extra_cost), big.mark = " ")))
```

```
Aluminium in the run : 8 400 kg
Production energy saved : 1 234 800 MJ (~343 000 kWh)
CO2e avoided : 87 360 kg (~87.4 t)
Extra material cost : $2 100
```

Roughly 1.2 million MJ (about 340 000 kWh) and 87 t of CO2e avoided for about $2 100 of extra material cost — one of the highest-leverage sustainability moves available on this project.

Problem 3: Design-for-recyclability audit (defense)

A defense client requires that field-returned electronic enclosures be **materially recoverable**. You are handed this enclosure: glass-filled nylon (#7) housing, 6 zinc-plated steel screws, a bonded aluminium RFI shield, a polyurethane potting compound over the PCB, and two adhesive asset-tracking labels.

List, in priority order, the four changes that would most improve material recovery, and note the functional risk of each.

Model answer

1. **Replace the potting compound** with a removable conformal coating or a gasketed enclosure. *Risk:* reduced shock/moisture protection — must re-qualify to the environmental spec.
2. **Eliminate the bonded aluminium shield**; use a conductive coating on the nylon or a snap-in shield of the same-family metal. *Risk:* RFI performance must be re-verified.
3. **Swap the 6 steel screws** for moulded snap features or same-metal fasteners. *Risk:* fewer re-assembly cycles; captive-hardware requirement may forbid it.
4. **Drop adhesive labels** for moulded-in or laser-etched marking. *Risk:* contrast/legibility and traceability sign-off.

The housing polymer itself (#7 glass-filled) is hard to change if the structural spec depends on it — note it as a known limitation rather than a quick fix.

Problem 4 (discussion): planned obsolescence vs. sustainable design

"Sustainable design requires that our products last as long as possible. But business decisions may deliberately shorten product life to protect repeat sales — planned obsolescence. Which do you prefer, and how would you reconcile the two in a company that has to stay profitable?"

Discussion

There is no single right answer — this is a position you argue. Strong answers usually land on **service and circular business models**: the manufacturer keeps revenue through repair, upgrades, refurbishment, take-back, or product-as-a-service, so "make it last" and "keep earning" stop being opposites. Weak answers just pick a side and ignore the constraint on the other.

1.14 Further Resources

- **U.S. EPA — Learn About Sustainability:** https://www.epa.gov/sustainability/learn-about-sustainability
- **Brundtland Report (*Our Common Future*, 1987):** https://sustainabledevelopment.un.org/content/documents/5987our-common-future.pdf
- **Ellen MacArthur Foundation — Circular Economy Introduction:** https://www.ellenmacarthurfoundation.org/topics/circular-economy-introduction/overview

The Story of Stuff: https://www.youtube.com/watch?v=9GorqroigqM

Explaining the Circular Economy (Ellen MacArthur Foundation): https://www.youtube.com/watch?v=zCRKvDyyHmI

What Is Sustainable Development?: https://www.youtube.com/watch?v=7V8oFI4GYMY

Chapter 2

Introduction to Project Management and Communication

2.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Distinguish** a project from a process, and explain why project work needs its own discipline.
2. **Describe** the triple constraint (scope, time, cost) and its relationship to quality and resources.
3. **Identify** the five process phases of a project and what happens in each.
4. **Apply** the project manager's decision hierarchy and name the four pillars of the PMI Code of Ethics.
5. **Build** a project communication plan and choose the right method (formal/informal, written/verbal) for a message.
6. **Calculate** the number of communication channels on a team and explain why they grow faster than headcount.
7. **Apply** active- and effective-listening techniques and recognize the listening filters that distort a message.
8. **Select** the appropriate performance report (status, progress, trend, forecast, variance, earned value) for a stakeholder need.

Before we begin: think of a group assignment that went badly

Was the problem that nobody knew *what* to do, or that people knew but never told each other? Most project failures are communication failures wearing a different coat. Keep that example in mind.

2.2 Introduction: only one project in four

Here is a sobering number that opens this course: by common industry estimates, only about **26 % of projects** are delivered successfully while respecting the triple constraint — on scope, on time, on budget. The other three in four slip, overspend, descope, or fail outright.

That is why project management exists as a discipline. Running the RC-vehicle launch from the last chapter is not the same as running a production line. A line is a **process**: repetitive, permanent, tuned once and then kept stable. A **project** is a one-time push toward a unique goal, under a deadline and a budget, and it needs different tools and a different mindset.

> **!** Important
>
> **Case study — Orion Tech (MGMT-3105, Week 2).** A software company built a healthcare app. The team was skilled, but: developers were never clear on requirements and reworked constantly; the PM ran everything through email and buried the team; stakeholders got no status reports; a few voices dominated meetings while others went silent; and tone and urgency were misread between the local team and offshore developers. Deadlines slipped, the budget grew, and the client's satisfaction collapsed. Leadership asked whether the PM had followed any communication protocol at all.

What Is Project Management? (Association for Project Management): https://www.youtube.com/watch?v=Jk-JwtScIlw

2.3 Project versus process

	Project	**Process**
Duration	Temporary, one-time	Permanent, ongoing
Work	Unique, complex, connected activities toward one goal	Repetitive, designed to be repeated
Ends when	The goal is met (by a specific time, within budget, to spec)	It doesn't — it is improved and kept running
Example on our project	*Designing and launching the RC-vehicle replica*	*Moulding 100 000 bodies once the line is running*

Projects help companies cope with **shortened product life cycles**, **narrow launch windows**, **increasingly complex products**, and **global markets**.

The RC-vehicle brief has all four: a holiday launch window, a technical product, and suppliers on two continents.

2.4 The triple constraint

Every project is boxed in by **scope**, **time**, and **cost**, with **quality** sitting in the middle as the measure of whether stakeholders are satisfied. Resources (people, equipment, facilities) are the fourth practical limit.

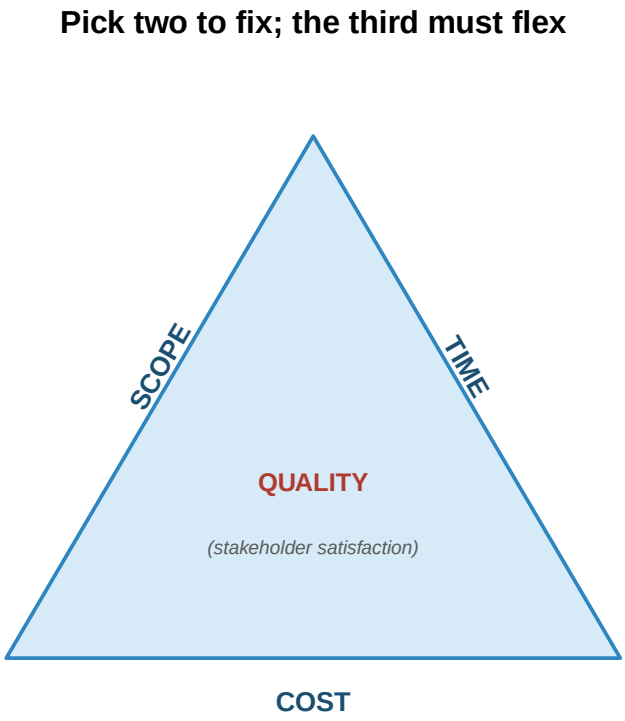


Figure 2.1: The triple constraint. Scope, time, and cost form the sides; quality is the area they enclose. Change one side and at least one other must move, or quality suffers.

Think about it: the retailer moves the RC-vehicle launch four weeks earlier. Which constraint gives?

Time just shrank. To hold the launch you must either **cut scope** (fewer body options, defer a feature), **add cost** (overtime, expedited tooling, a second supplier), or **accept lower quality** (less testing) — and that last one is how

projects like Orion Tech's die. The PM's job is to make that trade explicit and get a decision, not to quietly absorb it.

2.4.1 Try it yourself: put a number on each side of the triangle

"Which constraint gives" is a much easier decision once each option has a dollar figure attached. Live cell — change the weeks compressed, the costs, or the failure probability and see how the three options compare.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: triple-constraint-tradeoff-02
weeks_compressed <- 4

# Option A: hold scope -- pay to compress the schedule
overtime_cost_per_week <- 9000
expedite_fee           <- 15000
cost_of_holding_scope  <- weeks_compressed * overtime_cost_per_week + expedite_fee

# Option B: hold cost -- cut scope (drop one body colourway)
colourway_revenue_at_risk <- 62000   # forecast contribution margin lost

# Option C: hold cost and scope -- accept lower quality (skip one test cycle)
skip_test_prob_failure <- 0.08       # chance a defect reaches the customer
field_failure_cost     <- 240000     # recall/rework cost if it does
expected_quality_cost  <- skip_test_prob_failure * field_failure_cost

fmt <- function(x) format(round(x), big.mark = " ", scientific = FALSE, trim = TRUE)

costs <- c(A = cost_of_holding_scope, B = colourway_revenue_at_risk, C = expected_quali
cheapest <- c(A = "A, pay to compress", B = "B, cut scope", C = "C, skip a test cycle")

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(costs / 1000, names.arg = c("A: compress", "B: cut scope", "C: cut testi
              col = c("#2980b9", "#e67e22", "#8e44ad"), ylim = c(0, max(costs / 1000) *
              ylab = "Cost of flexing ($ thousand)", main = "What each side of the tria
text(bp, costs / 1000, labels = paste0("$", round(costs / 1000, 1), "k"), pos = 3, cex

cat(sprintf("Cheapest side to flex: Option %s\n", cheapest))
cat(sprintf("Option A: pay to compress  $%s\n", fmt(cost_of_holding_scope)))
cat(sprintf("Option B: cut scope  $%s revenue lost\n", fmt(colourway_revenue_at_risk)))
cat(sprintf("Option C: skip a test cycle  $%s expected cost\n", fmt(expected_quality_co
```

```
cat(sprintf("Option C risk  %.0f%% chance of a $%s failure\n",
            skip_test_prob_failure * 100, fmt(field_failure_cost)))
```

2.5 The five process phases

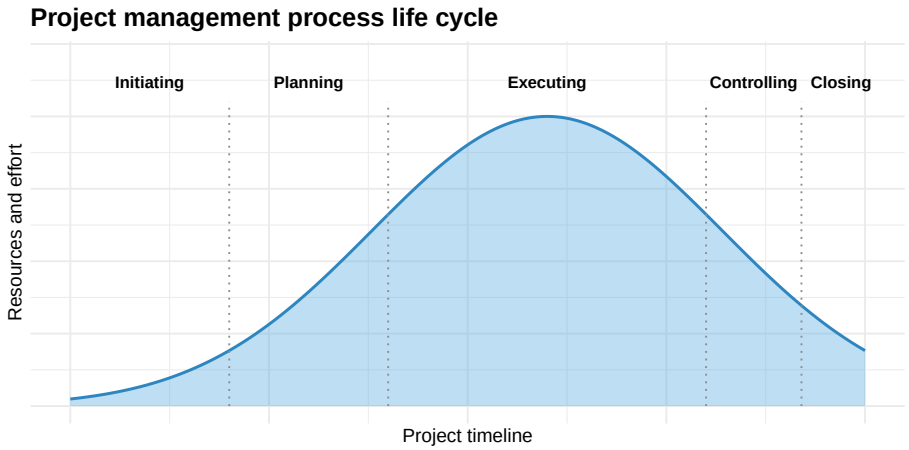


Figure 2.2: Effort and resource use across the project life cycle. Little happens during initiation, effort peaks through execution and control, and it falls away at closing.

Phase	Key work
1. Initiating	Business case, feasibility, project sponsors, **project charter** + sign-off.
2. Planning	Scope statement, WBS, network diagram (AON), budget, risk plan, quality standards, control system, communication protocols. *This phase decides success or failure.*
3. Executing	Execute the plan, monitor progress, quality assurance. The PM shifts from doing to leading.
4. Controlling	Scope/change/schedule/budget/risk control become daily; overlaps execution.

Phase	Key work
5. Closing	Final review of budget, schedule, scope; customer sign-off; **lessons learned**; release resources; final payments. The hardest phase to finish well.

2.5.1 The project manager's decision hierarchy

When decisions conflict, this is the order of consideration:

1. **Safety** (always first)
2. What is best for **the company I represent**
3. What is best for **the project sponsor**
4. What is best for **the customer**
5. What is best for **the suppliers**

And the **PMI Code of Ethics** rests on four pillars: **responsibility, respect, fairness, and honesty**. You can violate ethics without breaking a law — but as an accredited PM you would swear to uphold the code, and breaking it puts your professional standing at risk.

2.6 Project communications

Communication is not a side task. **About 90 % of a project manager's time is spent communicating.** The three communication processes are **plan**, **manage**, and **control**.

2.6.1 The communication plan

A written communication plan is built from the stakeholders' information needs. Typical elements:

- What information is collected, and when
- Who receives it
- How it is gathered and stored
- Limits, if any, on who may tell whom
- Reporting relationships
- Contact information for all stakeholders
- The distribution schedule for each type of communication

Orion Tech fix: what should have been in the plan on day one?

A stakeholder list with each party's information needs; the report types (status, cost, risk, performance) and their frequency; the mix of channels (email for the record, chat for quick updates, dashboards for tracking, video for collaboration); and **feedback loops** so every team member confirms they understood

the requirements. None of that existed — so requirements were guessed at and stakeholders were left in the dark.

2.6.2 The communication model

Every communication has three elements: a **sender**, a **message**, and a **receiver**. The sender **encodes** the message based on the receiver's education, experience, language, and culture, picks the right method, and **confirms it was understood**. The receiver **decodes** carefully and checks their understanding. **Feedback** ("Does that make sense?", a nod, a clarifying question) closes the loop.

2.6.3 Communication channels grow faster than the team

Add one person to a team and you do not add one relationship — you add one for every person already there. The number of two-way channels on a team of n people is:

$$\text{channels} = \frac{n(n-1)}{2}$$

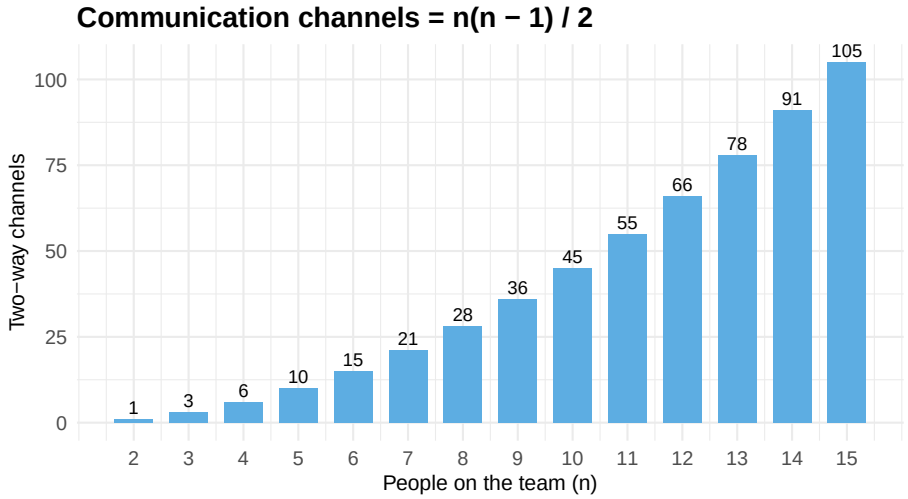


Figure 2.3: Two-way communication channels versus team size. Doubling the team roughly quadruples the channels that have to be kept coherent.

2.6.4 Worked Example 1: channels on the RC-vehicle project

The core project has the PM, 4 team members, an accountant, and a sponsor liaison — **7 people**. The extended stakeholder circle adds 2 supplier contacts,

a retailer contact, and a compliance officer — **11 people**.

Live cell — try different core-team and extended-circle sizes.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: channels-example-02
core <- 7
extended <- 11

ch_core     <- core * (core - 1) / 2
ch_extended <- extended * (extended - 1) / 2

n <- 2:40
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(n, n * (n - 1) / 2, type = "l", lwd = 4, col = "#2c3e50", xlab = "People in the gr
     main = "Channels grow with the square of group size")
points(c(core, extended), c(ch_core, ch_extended), pch = 19, cex = 2.2, col = c("#2980

cat(sprintf("Widening the circle from %d to %d adds %d channels\n", core, extended, ch_
cat(sprintf("Core team channels  %d\n", ch_core))
cat(sprintf("Extended circle channels  %d\n", ch_extended))
```

Four extra people did not add four conversations — they added 34. That is the argument for a **hub-and-spoke** communication plan (routes defined, the PM as the hub) instead of letting everyone talk to everyone about everything.

2.6.5 Try it yourself: channel growth and the cost of one more stakeholder

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: channels-calc-02
team_size <- 9

channels <- function(n) n * (n - 1) / 2

sizes <- team_size + 0:4
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(2:40, channels(2:40), type = "l", lwd = 4, col = "#bdc3c7", xlab = "People in the
     main = "Each added person costs more than the last")
```

```
points(sizes, channels(sizes), pch = 19, cex = 2.2, col = c("#2980b9", rep("#e74c3c", 4)))

cat(sprintf("Adding person #%d alone creates %d new channels\n",
            team_size + 1, channels(team_size + 1) - channels(team_size)))
for (extra in 0:4) {
  n <- team_size + extra
  cat(sprintf("Team of %d  %d channels", n, channels(n)))
  if (extra > 0) cat(sprintf(" (+%d vs. baseline)", channels(n) - channels(team_size)))
  cat("\n")
}
```

2.6.6 Try it yourself: a merger by hand

A team is about to take on extra people. Work out how much the coordination load grows. Each team picks its own **data set**.

Two-way channels before and after a merger

Channels = n(n - 1) / 2. Percentages to the nearest whole number.

- Current team size: 6
- People joining: 3
- Channels before: _____ *(to 0 decimal places)*
- Channels after: _____ *(to 0 decimal places)*
- Increase in headcount: _____ % *(to 1 decimal place)*
- Increase in channels: _____ % *(to 1 decimal place)*

Show the worked solution

Channels = n(n - 1)/2. For data set 1: before, 6 x 5 / 2 = 15; after, 9 x 8 / 2 = 36. Headcount rises 50 % but channels rise 140 %: adding people grows the load faster than the headcount.

2.7 Listening: the mode we use most and understand least

The four modes of communication, by share of a typical day and by how well they are scientifically understood:

Mode	Share	Notes
Listening	~55 %	The most used, the **least** studied and taught
Speaking	~23 %	
Reading	~13 %	

Mode	Share	Notes
Writing	~9 %	

And most of the meaning is **nonverbal**: roughly **93 %** of person-to-person communication is carried by body language (~55 %) and tone of voice (~38 %) — and tone's share rises toward 86 % on the phone, where body language is gone.

- **Active listening:** the receiver signals engagement — agreeing, asking for clarification.
- **Effective listening:** watching the speaker for gesture and expression, thinking before responding, asking questions, repeating back, giving feedback.

2.7.1 The listening filters

Every message passes through three filters before it lands:

- **Selective attention** — we choose what to focus on.
- **Selective interpretation** — we impose our own meaning (shaped by gender, generation, geography, culture, education, occupation).
- **Selective retention** — we choose what to keep, and for how long.

There is also a raw arithmetic problem. People **listen** at about 500 words per minute but **speak** at about 150. That leaves ~350 words per minute of spare capacity — room to tune out.

2.7.2 Try it yourself: the tune-out budget

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: tune-out-02
listen_wpm <- 500
speak_wpm  <- 150
briefing_minutes <- 12

spare_wpm    <- listen_wpm - speak_wpm
spare_words  <- spare_wpm * briefing_minutes
# Time the listener could "leave" if they mentally processed at listening speed:
tune_out_seconds_per_min <- spare_wpm / listen_wpm * 60

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(c(Speaking = speak_wpm, Listening = listen_wpm), col = c("#2980b9", "#16a
```

```
             ylab = "Words per minute", main = "The gap the listener's mind fills")
segments(bp[1], speak_wpm, bp[2], listen_wpm, lty = 2, lwd = 3, col = "#e74c3c")
text(mean(bp), (speak_wpm + listen_wpm) / 2 + 20, labels = paste(spare_wpm, "spare"), cex = 1.3,

cat(sprintf("Spare listening capacity: %d words/min\n", spare_wpm))
cat(sprintf("Spare words over the briefing  %s\n", format(spare_words, big.mark = " ")))
cat(sprintf("Tune-out time available  %.0f seconds per minute\n", tune_out_seconds_per_min))
```

Listening de-railers — which one is Orion Tech's problem?

- **Listening without focus** — mind multitasking.
- **Listening without respect (impatience)** — deciding the other person is wrong and interrupting.
- **Listening without respect (ego)** — hearing only from your own perspective, not theirs.
- **Listening with bias (defensive / adverse)** — building your rebuttal while they talk, or listening only for points to attack.

On Orion Tech it was mostly the first and last: dominant voices building responses instead of listening, quieter voices tuned out, and cross-cultural tone read through the listener's own filter rather than the speaker's intent.

2.8 Choosing a communication method

	Written	**Verbal**
Formal	Complex problems, project plans, charters, long-distance — *formal written*	Presentations and speeches — *formal verbal*
Informal	Memos, emails (careful: email is also a contract), texts, notes — *informal written*	Meetings and conversations — *informal verbal*

Should a PM try to control **all** communication between team members and outsiders? **Yes** — and can they? **No.** The plan defines the important routes; it cannot police every hallway conversation.

2.8.1 Try it yourself: pick the method

Which communication method fits each message?

Pick formal or informal, and written or verbal, for every message.

Categories: Formal written / Formal verbal / Informal written / Informal verbal

1. A signed change order to a supplier _____
2. The project charter, sent for approval _____
3. Presenting the risk register to the steering committee _____
4. A closing speech to the client's executives _____
5. A text: the truck arrives at 3 p.m. _____
6. A short email reminding a colleague a drawing is due Friday _____
7. A hallway chat with a quality technician about a gauge _____
8. The daily 10-minute stand-up with the assembly team _____

Answers: 1 Formal written; 2 Formal written; 3 Formal verbal; 4 Formal verbal; 5 Informal written; 6 Informal written; 7 Informal verbal; 8 Informal verbal.

2.9 Performance reporting

Different stakeholder questions need different reports:

Report	Answers
Status	Where does the project stand right now?
Progress	What has been accomplished?
Trend	Is performance improving or deteriorating over time?
Forecast	What will status and performance be in the future?
Variance	How do actual results compare to the plan?
Earned value	Integrating scope, cost, and schedule into one performance picture (see Chapter 12).

Orion Tech's stakeholders were right to expect regular **status, progress, and forecast** reports — they fund and approve the project, and reports give them transparency, accountability, and early warning.

2.9.1 Try it yourself: which report answers the question?

Which performance report answers each stakeholder question?

Pick the report type for each question.

Categories: Status / Progress / Trend / Forecast / Variance / Earned value

1. Where does the project stand today? _____
2. What has the team finished since the last report? _____
3. Is the defect find-rate getting better or worse over six weeks? _____

4. On the current rate, when will we launch? _____
5. How far is tooling cost from what we budgeted? _____
6. Given what is spent and what is done, are we ahead or behind overall? _____
7. Are issues being closed faster than they are raised? _____
8. Will we finish under budget at completion? _____

Answers: 1 Status; 2 Progress; 3 Trend; 4 Forecast; 5 Variance; 6 Earned value; 7 Trend; 8 Forecast.

2.10 Key Takeaways

- A **project** is temporary and unique; a **process** is permanent and repetitive. Only ~26 % of projects hit the full triple constraint — which is why the discipline exists.
- The **triple constraint** is scope–time–cost, enclosing **quality**. Move one side and another must move, or quality pays.
- Five phases: **initiating, planning, executing, controlling, closing.** Planning decides the outcome; closing is the hardest to finish.
- Decisions are weighed **safety → company → sponsor → customer → suppliers**; ethics rest on **responsibility, respect, fairness, honesty**.
- **~90 %** of a PM's time is communication. Build a written **communication plan** from stakeholder needs, and close every loop with **feedback**.
- Channels grow as $n(n-1)/2$ — faster than headcount. Plan routes; don't let everyone talk to everyone.
- **Listening** is ~55 % of communication and the least trained; ~93 % of meaning is nonverbal. Watch for the **selective attention/interpretation/retention** filters and the de-railers.
- Match the **method** (formal/informal × written/verbal) to the message, and the **report** (status/progress/trend/forecast/variance/EVM) to the stakeholder's question.

2.11 Case Study: Meridian R/C — Week 2

CASE STUDY FILE — Week 2: The Email That Didn't Reach Vancouver

Three weeks after Daniel Cho green-lit the Meridian Trailblazer prototype, Sofia Reyes flagged a durability concern in the early drop-test data: the die-cast aluminium body and frame were flexing more than expected at the wheel-well corners. Priya Nandakumar's fix was straightforward — thicken the aluminium wall from **1.2 mm to 1.5 mm**, a change that would not touch the tooling budget but would add a fraction of a gram per unit. You approved it the same afternoon and asked Marcus Webb to

notify everyone downstream who depended on the old spec sheet.
That is where it went wrong. Marcus fired off the update to the internal engineering distribution list — Priya, Sofia, the CAD vendor, and you — because that was the list he had bookmarked from the Week 1 kickoff. It was the right list for people *building* the part. It was not the right list for people *selling* it. Kestrel Retail Group's Category Buyer, **Renata Ford**, based out of **Vancouver**, had been working from the original 1.2 mm spec sheet to brief her own packaging and merchandising team on box dimensions and weight claims. Nobody had ever added her, or Kestrel's Vancouver office generally, to any distribution list — because no such list existed. Renata found out about the change only when her packaging vendor's proof came back listing a spec that did not match Meridian's shipped sample.
The fallout: a **3-day delay** while Kestrel's packaging vendor reprinted spec sheets and box art, and a **$4,000** rush-reprint charge nobody had budgeted for. Daniel Cho was patient about the money but blunt about the cause: *"This wasn't a design problem. It was a mailing-list problem."*
You rebuilt the communication plan that week. A **weekly status email** now goes to a named, role-based distribution list — not a bookmarked group of whoever was in the room at kickoff. A **biweekly steering call** with Daniel and Kestrel keeps the client in sync in real time instead of by forwarded email. And a **RACI matrix** now spells out, for every deliverable, who is Responsible, Accountable, Consulted, and Informed — so "who else needs to know this" stops being a guess. The timing mattered too: the team was about to grow from 4 people to 7, taking the number of two-way communication channels from $n(n-1)/2 = 6$ to **21** — more than triple the coordination load the new plan now had to carry.

Metric	Before the fix	After the fix
Distribution list	Ad hoc, engineering-only	Named, role-based (RACI-driven)
Client sync	None scheduled	Weekly status email + biweekly steering call
Team size	4	7
Two-way channels $(n(n-1)/2)$	6	21

2.12 Practice Problems

Problem 1: Channel count and a reorganization

The RC-vehicle project currently routes all communication through a 6-person core team. A proposal would merge in 3 supplier engineers as full team members.

(a) How many channels before and after?
(b) By what percentage do channels increase versus the 50 % increase in headcount?

Solution

```r
before <- 6; after <- 9
ch <- function(n) n * (n - 1) / 2
cat("Before:", ch(before), "channels\n")
cat("After :", ch(after), "channels\n")
cat(sprintf("Headcount up %.0f%%, channels up %.0f%%\n",
            100 * (after / before - 1),
            100 * (ch(after) / ch(before) - 1)))
```

```
Before: 15 channels
After : 36 channels
Headcount up 50%, channels up 140%
```

Headcount rises 50 %, channels rise 125 %. Merging them in as full members triples the coordination load of those three people; a defined liaison route would add far less.

Problem 2: Pick the method (automotive, food, defense)

Choose formal/informal and written/verbal for each:

1. Notifying a defense customer that a delivered lot failed an acceptance test.
2. A daily 10-minute stand-up with the assembly team.
3. The project charter for the RC-vehicle launch.
4. Reminding a teammate that a drawing is due Friday.

Answers

1 — **formal written** (contractual weight, complex, long-distance). 2 — **informal verbal** (meeting). 3 — **formal written** (plan/charter). 4 — **informal written** (a text or short email; keep a light record).

Problem 3 (discussion): rebuild Orion Tech's communication system

If you were the PM, redesign the tools, reporting, and meetings to prevent the failure.

Model answer

Write a **communication plan before launch**. Use a mix: **email** for records, **Teams/Slack** for quick updates, **dashboards** for visual tracking, **video calls** for collaboration (so tone and context survive the offshore gap). Run **regular status meetings with an agenda and action items**, and have offshore developers present their own weekly updates so no group dominates. Send stakeholders a **weekly progress report** (status, cost, risk). Build **feedback loops**: every developer confirms, in their own words, what a requirement means

before coding it. Use plain language, avoid idioms, and summarize decisions in writing.

2.13 Further Resources

5 Ways to Listen Better (Julian Treasure, TED): https://www.youtube.com/watch?v=cSohjlYQI2A

10 Ways to Have a Better Conversation (Celeste Headlee, TED): https://www.youtube.com/watch?v=R1vskiVDwl4

Chapter 3

Project Scope, Charter, and Organizational Structures

3.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Evaluate** a candidate project with payback period and net present value (NPV).
2. **Write** a project charter and explain what authority its sign-off grants.
3. **Identify** the five inputs to a charter and the benefits of using one.
4. **Build** a stakeholder register and place stakeholders on a power–interest grid.
5. **Develop** a scope statement — deliverables, acceptance criteria, exclusions, constraints, assumptions — and distinguish product scope from project scope.
6. **Recognize** scope creep and run a change-control process.
7. **Compare** functional, projectized, and matrix (weak/balanced/strong) structures and the trade-offs each imposes on the project manager.

Before we begin: who told you it was okay to start?

On your last group project, was there a moment where everyone agreed *this is what we are building and this is who signed off*? Or did work just… start? The charter is that moment, written down.

3.2 Introduction: nothing official without a charter

Your company must get **Senior Management to approve the Scope and Charter before work begins** on the RC-vehicle replica. That is not bureaucracy — it is the difference between a project and a hobby. *"There is no official project without an approved Project Charter."*

Two questions come first: **should we do this project at all** (selection), and **exactly what are we committing to** (scope). Then the organization's shape — functional, projectized, or matrix — decides how much authority you actually have to deliver it.

> **Note**
>
> **Industry snapshot.** In automotive new-product introduction, a project cannot pass its first gate without a signed charter and a business case showing positive NPV. A food company scoping a new line writes a **statement of work** that names the throughput, the allergen controls, and what is explicitly *out*. A defense contractor's **SOW** is a contract exhibit — every deliverable and exclusion in it is legally enforceable.

What Is Scope in Project Management?: https://www.youtube.com/watch?v=DZr3ZMWGGwc

3.3 Project selection: is it worth doing?

3.3.1 Payback period

The simplest screen: how long until the project's net cash inflows repay the initial outlay?

$$\text{Payback} = \text{years until cumulative cash flow} \geq 0$$

3.3.2 Worked Example 1: payback on the RC-vehicle tooling investment

The project needs **$450 000** up front (tooling, setup, approvals). Once the 100 000-unit contract runs, it returns an estimated **$180 000** of net cash per year.

Live cell — change the outlay, the annual cash flow, or the horizon.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and

press **Run Code** to experiment further.

```
#| label: payback-example-03
initial   <- 450000
annual_cf <- 180000
horizon   <- 5

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", tr

cum <- cumsum(rep(annual_cf, horizon)) - initial
year_paid <- which(cum >= 0)[1]

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(cum / 1000, names.arg = 1:horizon, col = ifelse(cum >= 0, "#16a085", "#e74c3c"),
        xlab = "Year", ylab = "Cumulative cash flow ($ thousand)", main = "Cumulative cash flow")
abline(h = 0, lwd = 2)

if (is.na(year_paid)) {
  cat(sprintf("!!Not paid back within %d years!!\n", horizon))
} else {
  # Fractional year within the payback year:
  prev <- if (year_paid == 1) -initial else cum[year_paid - 1]
  frac <- -prev / annual_cf
  cat(sprintf("Payback period: %.2f years\n", (year_paid - 1) + frac))
}
for (t in 1:horizon) cat(sprintf("End of year %d  %s\n", t, money(cum[t])))
```

Payback is quick to compute but ignores the **time value of money** and anything that happens after the payback point. NPV fixes both.

The same story as a picture: the balance starts at $0, is spent down to **-$450 000** as tooling, setup, and approvals are paid for ahead of **Start of Production (SOP)**, then turns upward once production starts, as $180 000/year of net cash flow closes the gap until it crosses breakeven.

3.3.3 Net present value

NPV discounts every future cash flow back to today's dollars at the organization's required rate of return r, then subtracts the initial outlay. A positive NPV means the project earns more than that hurdle rate.

$$NPV = -C_0 + \sum_{t=1}^{n} \frac{CF_t}{(1+r)^t}$$

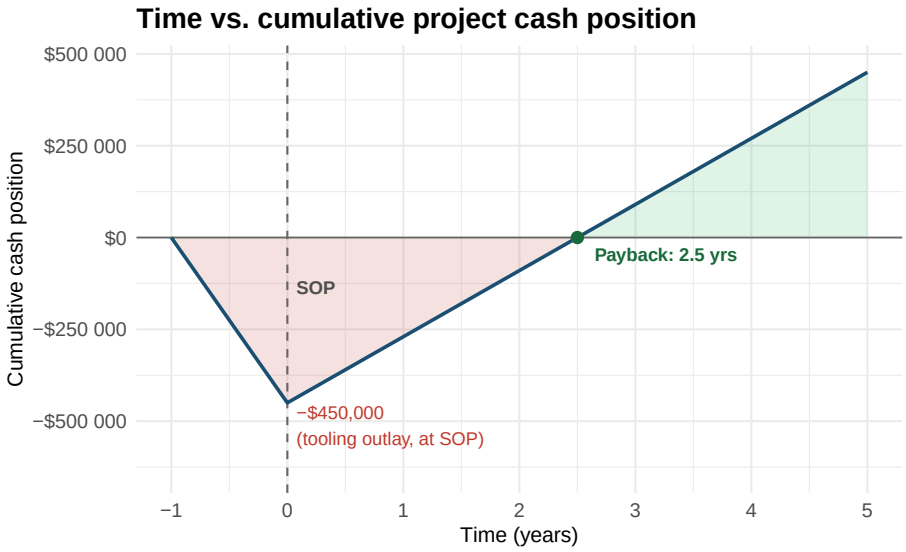


Figure 3.1: Cumulative cash position for the RC-vehicle tooling investment. The balance starts at $0 and is spent down to -$450,000 by Start of Production (SOP); once production begins, $180,000/year of net cash flow closes the gap, crossing breakeven at 2.5 years.

3.3.4 Nominal vs. discounted payback for Example 1

Apply a 12 % hurdle rate to Example 1's own numbers and put the nominal and discounted cash flows side by side:

Table 3.1: Example 1: nominal cash flow and cumulative cash flow vs. the discounted present value and cumulative PV (NPV running total) at a 12% hurdle rate.

Year	Net cash flow	Cumulative cash flow	Discount factor @ 12%	Present value	Cumulative I
0	-$450 000	**-$450 000**	1.000	-$450 000	**-$450 0**
1	$180 000	**-$270 000**	0.893	$160 714	**-$289 2**
2	$180 000	**-$90 000**	0.797	$143 495	**-$145 7**
3	$180 000	**$90 000**	0.712	$128 120	**-$17 6**
4	$180 000	**$270 000**	0.636	$114 393	**$96 7**
5	$180 000	**$450 000**	0.567	$102 137	**$198 8**

The **nominal** cumulative cash flow crosses zero between year 2 and year 3 — the 2.5-year payback already computed. Once each year's cash flow is discounted at 12 %, the **cumulative PV** does not cross zero until between year 3 and year 4:

a **discounted payback of about 3.15 years**. Those extra 0.65 years are the cost of the time value of money — and the year-5 cumulative PV, **$198 860**, is exactly the project's NPV.

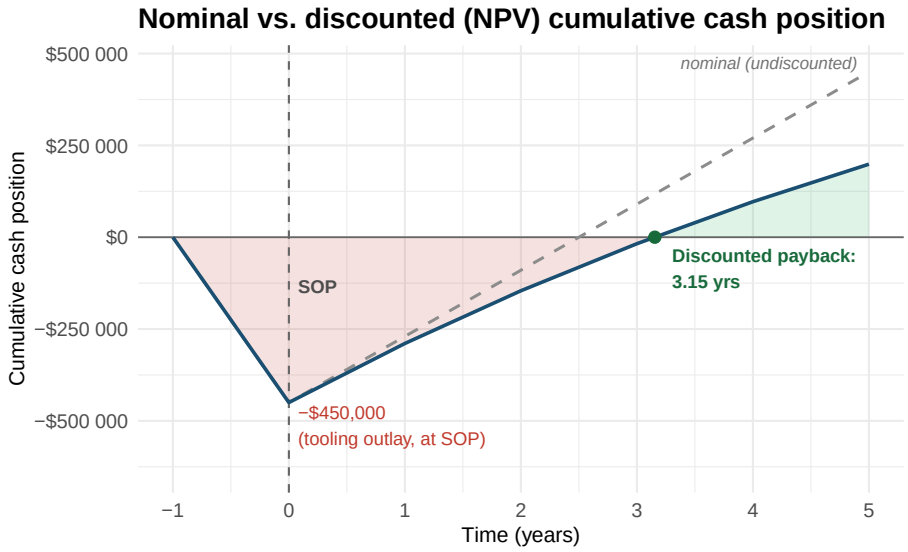


Figure 3.2: Discounted breakeven for Example 1. The dashed grey line is the nominal cumulative cash flow from the earlier figure; the solid navy line discounts each year's cash flow at the 12% hurdle rate before accumulating it, pushing the crossing point from 2.5 years out to about 3.15 years.

3.3.5 Try it yourself: NPV of the project

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: npv-calc-03
initial   <- 450000
cash_flows<- rep(180000, 5)   # years 1..5, as in Worked Example 1
r         <- 0.12   # required rate of return

t   <- seq_along(cash_flows)
pv  <- cash_flows / (1 + r)^t
npv <- sum(pv) - initial

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", tr
```

```
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(rbind(cash_flows, pv) / 1000, beside = TRUE, names.arg = t, col = c("#bd
              main = "Cash flow (grey) vs. present value (blue)")

if (npv > 0) {
  cat(sprintf("NPV %s: ACCEPT (beats the %s%% hurdle)\n", money(npv), format(100 * r))
} else {
  cat(sprintf("NPV %s: !!REJECT!! (misses the %s%% hurdle)\n", money(npv), format(100 *
}
for (i in t) cat(sprintf("Year %d present value  %s\n", i, money(pv[i])))
cat(sprintf("Sum of PVs  %s\n", money(sum(pv))))
cat(sprintf("Less outlay  %s\n", money(initial)))
```

What happens to NPV if the hurdle rate rises to 18 %?

Raise `r` in the cell and re-run. Higher discount rates punish cash flows that arrive later, so NPV falls — a project that clears a 12 % hurdle may fail an 18 % one. This is why a company with expensive capital (or a risky project that demands a risk premium) is choosier about which projects it starts. Course Learning Outcome 1 is exactly this: *communicate the philosophy of project selection using the NPV method.*

3.3.6 Try it yourself: a discounting table by hand

Build the year-by-year table for a fresh project, then read off its NPV and discounted payback. Amounts are in $ thousand. Each team picks its own **data set**.

Discounted cash flow table

The required rate of return is in the Rate column. Year 0 is the outlay.

Year	Rate	Cash flow (k)	$Discount factor$	PV ($k)	Cumulative $present value$ (k)
0.0	10 %	-320.0			
1.0	10 %	110.0			
2.0	10 %	120.0			
3.0	10 %	130.0			
4.0	10 %	90.0			
5.0	10 %	70.0			

- NPV at the end of year 5: _____ $k *(to 1 decimal place)*
- Discounted payback: _____ years *(to 2 decimal places)*

Show the worked solution

For data set 1 (outlay $320k, r = 10 %): the present values are 100.0, 99.2, 97.7, 61.5 and 43.5, so the cumulative PV runs -220.0, -120.8, -23.2, +38.3 and +81.8. The NPV is **$81.8k**. The cumulative PV turns positive in year 4: discounted payback = 3 + 23.2 / 61.5 = **3.38 years**. The nominal payback would be shorter; the gap is the cost of the time value of money.

3.4 The project charter

> **PMI:** developing a document that formally **authorizes** a project and documents the initial requirements that satisfy the stakeholders' needs and expectations. **Approval of the charter formally initiates the project.**

3.4.1 Five inputs to a charter

1. **Project Statement of Work** — the high-level "project brief": business need and product scope, fewer details than the scope statement.
2. **Business Case** — why the project is being done (the NPV/payback story).
3. **Contract** (if the client is external).
4. **Enterprise Environmental Factors (EEFs)** — conditions you cannot control (market, regulation, weather).
5. **Organizational Process Assets (OPAs)** — the company's plans, processes, policies, and accumulated knowledge.

3.4.2 What a charter contains

Title and description; business case; pre-assigned resources; stakeholders and their known requirements; product deliverables and the end result; measurable objectives tied to strategy; a **summary** milestone schedule; a **summary** budget (gross only — the real budget comes after planning); approval requirements and who signs off; high-level risks; and **signatures**.

3.4.3 Why bother

- Stakeholders are clearly defined
- Communication channels are defined
- Roles and responsibilities are outlined
- Scope is defined — helping prevent **scope creep**
- Better project processes in later phases
- Higher probability of success

- **Buy-in** from the team

The charter is a **living document**: update it as the project moves. Used well, it is *"the single most important tool the project manager uses to manage the expectations of the sponsor and all other stakeholders."*

3.5 Stakeholders

A **stakeholder** is any individual or group with an active interest in the project who can affect it.

Internal	External
Senior management, accountant, team members, IT department	Client, competitors, suppliers, environmental agencies, government agencies, research labs

Managing them is a loop: **identify → gather information → determine strengths and weaknesses → predict behaviour → build an engagement strategy → implement**. The point of engagement is to **increase support and reduce resistance** — by listening, informing, and helping stakeholders along.

3.5.1 Try it yourself: score your own stakeholders onto the grid

Rate each stakeholder's **power** and **interest** on a 1–10 scale and the quadrant follows automatically — the same rule the figure above was built from. Live cell — edit the table or add rows for your own project.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: stakeholder-score-03
stakeholders <- data.frame(
  who      = c("Kestrel Retail Group", "Daniel Cho (sponsor)", "MCS (motor supplier)",
               "Internal QA", "Fibro Source (trim supplier)"),
  power    = c(9, 9, 5, 5, 2),
  interest = c(8, 8, 7, 5, 2)
)

stakeholders$quadrant <- with(stakeholders, ifelse(
  power >= 5 & interest >= 5, "Manage closely",
  ifelse(power >= 5 & interest < 5, "Keep satisfied",
```

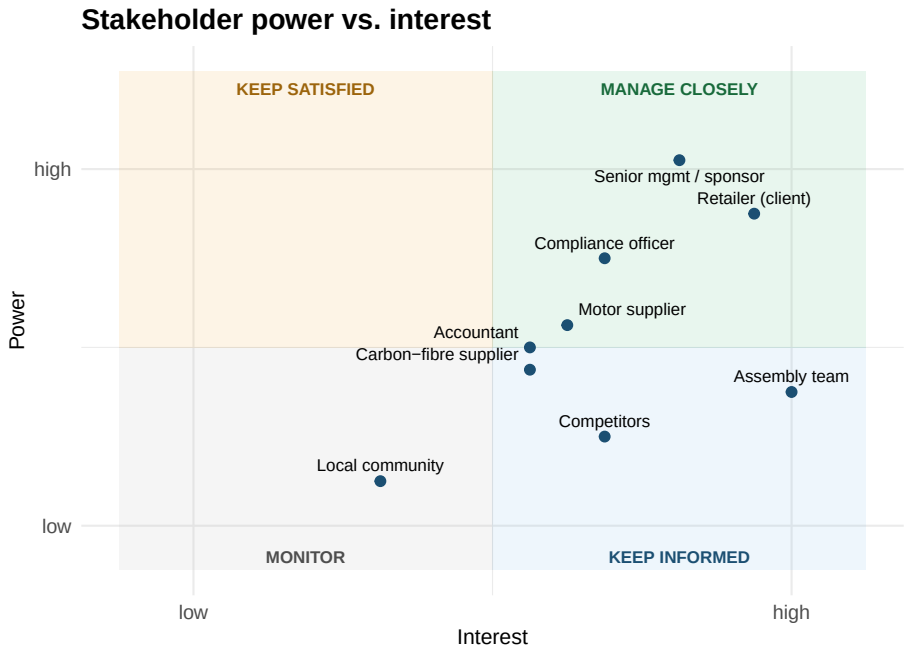


Figure 3.3: Power-interest grid for the RC-vehicle project. Effort goes where power and interest are both high; low-low stakeholders are monitored, not managed.

```
  ifelse(power < 5 & interest >= 5, "Keep informed", "Monitor"))))

n_close <- sum(stakeholders$quadrant == "Manage closely")
cat(sprintf("%d of %d stakeholders to manage closely\n", n_close, nrow(stakeholders)))
for (i in seq_len(nrow(stakeholders)))
  cat(sprintf("%s  %s\n", stakeholders$who[i], stakeholders$quadrant[i]))
cat("Change the power/interest scores (1-10) and the quadrant assignment updates.\n")
```

3.5.2 Try it yourself: place each stakeholder on the grid

Which quadrant is each stakeholder in?

Scores are out of 10; 5 and above counts as high. Pick the quadrant for each stakeholder.

Categories: Manage closely / Keep satisfied / Keep informed / Monitor

1. City building inspector (power 8, interest 3) _____
2. Assembly team lead (power 4, interest 9) _____
3. Retail buyer (power 9, interest 9) _____
4. Neighbouring business (power 2, interest 3) _____
5. Environmental agency (power 7, interest 2) _____
6. Motor supplier (power 6, interest 8) _____
7. Local newspaper (power 3, interest 6) _____
8. Finance director (power 9, interest 4) _____

Answers: 1 Keep satisfied; 2 Keep informed; 3 Manage closely; 4 Monitor; 5 Keep satisfied; 6 Manage closely; 7 Keep informed; 8 Keep satisfied.

3.6 The scope statement

Written early in **planning**, the scope statement is more detailed than the charter and should include:

- The **project statement** (what will be done)
- **Acceptance criteria** and the **validation/testing plan**
- **Deliverables**
- **Exclusions** (what is explicitly *not* in scope)
- **Constraints** and **assumptions**
- Project coordination
- **Signed-off** review with the customer and with corporate management

Product scope = the features and functions of the thing you are building. **Project scope** = the work needed to deliver it. A scope statement covers both, plus boundaries and references to specs. *As the project progresses, scope should get clearer and more specific — and changes here should be avoided, though not forbidden.*

3.6.1 The scope management processes

Plan scope → collect requirements → define scope → create WBS → validate scope (at closing) → control scope (throughout). The scope management plan is often run as an extension of **Management by Objectives (MBO)**: set unambiguous, realistic objectives; check periodically whether they are being met; take corrective action.

3.7 Scope creep

Scope creep is the uncontrolled growth of requirements over the project's life:

- one deliverable quietly becomes five
- three "essential" features become ten
- mid-project, the customer's needs change and requirements are reopened

Its usual causes are **stakeholders changing requirements** and **internal miscommunication**. It is *not always bad* — customer needs genuinely evolve, and refusing all change can deliver the wrong product. The problem is **uncontrolled** change. A good PM **expects it and plans for it** with a **change-control process**: every change request is logged, its impact on scope, schedule, and cost is assessed, and it is approved or rejected before work changes.

3.7.1 Try it yourself: the cost of an "innocent" change

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: scope-change-03
# A stakeholder asks to add a second body colourway "while we're at it".
extra_design_hours   <- 60
extra_tooling_cost   <- 28000
designer_rate        <- 46
indirect_rate        <- 0.35
schedule_days_added  <- 9
penalty_per_day_late <- 1500     # retailer contract clause

direct  <- extra_design_hours * designer_rate + extra_tooling_cost
loaded  <- direct * (1 + indirect_rate)
sched   <- schedule_days_added * penalty_per_day_late

fmt <- function(x) paste0("$", format(round(x), big.mark = " ", scientific = FALSE, trim = TRUE))

parts <- c(Direct = direct, Indirect = loaded - direct, "Late penalty" = sched)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
```

```
bp <- barplot(parts / 1000, col = c("#2980b9", "#8e44ad", "#e74c3c"), ylim = c(0, max(p
              main = "Where the cost of the change lands")
text(bp, parts / 1000, labels = paste0("$", round(parts / 1000, 1), "k"), pos = 3, cex

cat(sprintf("Total impact of one 'small' change: %s\n", fmt(loaded + sched)))
cat(sprintf("Direct cost  %s\n", fmt(direct)))
cat(sprintf("Fully burdened (x%.2f)  %s\n", 1 + indirect_rate, fmt(loaded)))
cat(sprintf("Schedule penalty (%g days)  %s\n", schedule_days_added, fmt(sched)))
```

3.8 Organizational culture

Every organization develops its own outlook, policies, habits of thought, and norms — *"as unique as a fingerprint."* Culture is *the solution to problems that has worked consistently for a group and is therefore taught to new members as the correct way to perceive and think.* It is shaped by **technology, environment, geographic location, reward systems, rules and procedures, key organizational members, and critical incidents.**

Culture affects projects through:

- **Department interaction** — cooperative cultures run projects far better than adversarial ones.
- **Employee commitment to goals** — teams need to understand the *why* to be motivated.
- **Project planning** — some cultures tolerate padding estimates but not being late.
- **Performance evaluation** — a culture that rewards innovation breeds aggressive, high-payoff project management (with occasional big losses).

3.9 Organizational structures

How the lines of authority and communication are arranged decides **when people are available** and **how you get them**.

Structure	Strengths for the PM	Weaknesses for the PM
Functional (grouped by specialty; one clear boss; each department runs its own projects)	No restructuring needed; deep expertise; everyone knows their place	Many layers to reach people; weak customer focus; projects drag because staff have "day jobs" and other priorities

Structure	Strengths for the PM	Weaknesses for the PM
Projectized (the company just does projects; team co-located)	One reporting line; easy communication; fast decisions; strong PMs; rapid market response	Teams are expensive to set up and maintain; loyalty to the project over the organization; hard to keep a shared pool of expertise; team anxiety about "what next" when the project ends
Matrix (blend; classified **weak / balanced / strong** by PM influence)	Suited to dynamic environments; values both project and functional efficiency; coordinates across units; maximizes scarce shared resources	**Two bosses**; heavy time spent negotiating shared resources; frustrating for staff caught between project and functional demands

Which structure is the RC-vehicle project running in?

Almost certainly a **matrix** — the company has an existing manufacturing line and functional departments (design, quality, purchasing), and it borrows people into the project. That means the PM shares the design engineers with their functional manager and must negotiate their time. Expect the "administrative conflict" of Chapter 10: every matrix team member has two bosses.

3.9.1 Try it yourself: which structure is it?

Which organizational structure does each description fit?

Pick the structure each description fits best.

Categories: Functional / Projectized / Weak matrix / Balanced matrix / Strong matrix

1. Each department runs its own projects; any PM is a part-time coordinator inside one department. _____
2. The company only does projects; the whole team is co-located and reports to the PM. _____
3. Staff report to functional managers; the PM mainly coordinates and expedites. _____
4. The PM and the functional managers share authority and negotiate over people. _____
5. A full-time PM with a project office controls the budget and assignments; functions supply specialists. _____

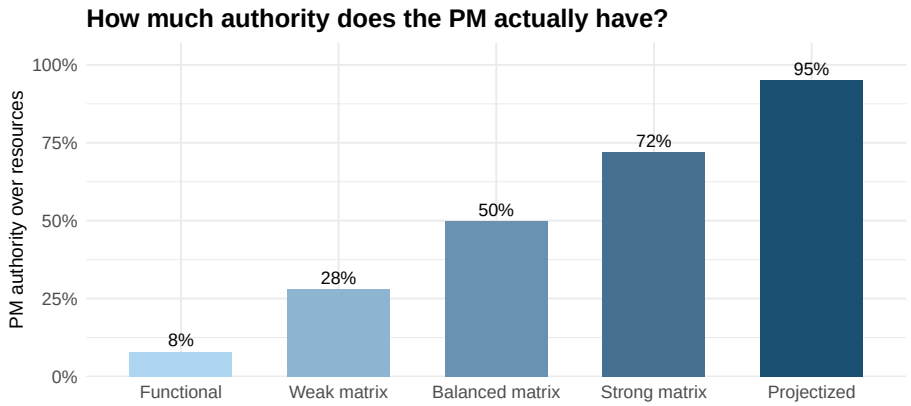


Figure 3.4: Project manager authority across organizational structures. It rises from almost none in a functional organization to near-total in a projectized one, with the matrix band in between.

6. The main worry is team anxiety about 'what next' when the project ends. _____
7. Projects drag because staff have day jobs and other priorities. _____
8. Every team member has two bosses, and the PM and functional manager negotiate for their time as equals. _____

Answers: 1 Functional; 2 Projectized; 3 Weak matrix; 4 Balanced matrix; 5 Strong matrix; 6 Projectized; 7 Functional; 8 Balanced matrix.

3.10 Key Takeaways

- **Select** projects on merit: **payback** is quick but ignores the time value of money; **NPV** discounts every future cash flow and is the course's standard (CLO 1). Positive NPV = beats the hurdle rate.
- The **charter** *formally initiates* the project. Inputs: SOW, business case, contract, EEFs, OPAs. It defines stakeholders, roles, and scope, and earns team **buy-in**. No approved charter, no official project.
- Manage **stakeholders** on a **power–interest grid**: manage the high-power/high-interest closely, monitor the rest.
- The **scope statement** covers **product and project scope**, plus acceptance criteria, exclusions, constraints, and assumptions, and is signed off.
- **Scope creep** is uncontrolled requirement growth. Change is normal; *uncontrolled* change is the enemy — run every request through **change control** for a cost/schedule impact and a decision.
- **Culture** shapes project outcomes through department cooperation, commitment, planning honesty, and what performance evaluation rewards.

- **Structure** sets the PM's real authority: near zero in **functional**, near total in **projectized**, negotiated in the **matrix** (weak/balanced/strong) — where everyone has two bosses.

3.11 Case Study: Meridian R/C — Week 3

CASE STUDY FILE — Week 3: The Charter Lands on Your Desk

This week the paperwork catches up with the promise. Daniel Cho, Meridian's VP of Product and the project's sponsor, signs off on the project charter for the Meridian Trailblazer, the 1/18-scale rally-truck replica that Kestrel Retail Group has all but promised to order in bulk — 100,000 units or more for next year's holiday season — if the prototype survives durability testing and toy-safety certification. Cho's signature does more than start the clock: it sets a hard ceiling. You are authorized to spend **up to $800,000** to get a certified, sellable prototype in front of Kestrel. Every budget you build from here — the detailed estimate due in a few weeks, the crashing decisions later, the earned-value tracking after that — has to fit under that number.

The charter also settles a structural question you've been dancing around since the aluminium-vs-steel-vs-carbon-fibre debate in Week 1: Meridian is not spinning up a dedicated project team. You are running a **matrix organization**. Priya Nandakumar keeps her desk in Manufacturing Engineering and is loaned to you for body and frame tooling. Marcus Webb still reports to Procurement and negotiates your supplier contracts on top of his other accounts. Sofia Reyes stays inside Quality and splits her week between your durability and certification testing and her department's other work. None of the three has you as their only boss — and neither do you have them full time.

With the charter signed, you sit down and sketch a power-interest grid to decide where your attention goes this quarter:

Stakeholder	Power	Interest	Quadrant
Kestrel Retail Group	High	High	Manage closely
Daniel Cho (sponsor)	High	High	Manage closely
MCS (motor supplier)	Medium	High	Keep informed
Paragon (motor supplier)	Medium	High	Keep informed
Internal QA department	Medium	Medium	Keep satisfied
Fibro Source (trim supplier)	Low	Low	Monitor

Kestrel and Cho land in the top-right corner — high power, high interest — because either one can stall or kill the project. MCS and Paragon, your two candidate motor suppliers, sit at medium power but high interest: they care intensely about winning the contract, so you keep them informed even though neither can unilaterally derail you. Internal QA, medium on both axes, gets kept satisfied: not a daily conversation, but not ignored either. Fibro Source, supplying only the optional carbon-fibre trim package, is low power and low interest — worth monitoring, nothing more.

3.12 Practice Problems

Problem 1: Payback vs. NPV disagree

Project A: $300 000 outlay, then $150 000/yr for 3 years, nothing after. Project B: $300 000 outlay, then $90 000/yr for 6 years. Hurdle rate 10 %. Which has the shorter payback, and which has the higher NPV?

Solution

```
npv <- function(c0, cfs, r) -c0 + sum(cfs / (1 + r)^seq_along(cfs))
A <- c(150000, 150000, 150000)
B <- rep(90000, 6)
cat(sprintf("A: payback 2.0 yr, NPV $%s\n", format(round(npv(3e5, A, 0.10)), big.mark =
cat(sprintf("B: payback ~3.3 yr, NPV $%s\n", format(round(npv(3e5, B, 0.10)), big.mark
```

```
A: payback 2.0 yr, NPV $73 028
B: payback ~3.3 yr, NPV $91 973
```

A pays back faster; B has the higher NPV because its cash flows keep coming for three more years. Payback alone would pick the worse project.

Problem 2: Charter or scope statement?

Which document does each belong in?

1. “Gross budget: approximately $650 000.”
2. “Body colourways: Racing Red and Gunmetal only; no custom colours.”
3. “Sponsor: VP Operations. Signs off at Gate 1.”
4. “Drop test: survives 10 drops from 1.5 m onto concrete, no functional damage.”

Answers

1. **Charter** (summary budget).
2. **Scope statement** (exclusions).
3. **Charter** (approval/signatures).
4. **Scope statement** (acceptance criteria).

Problem 3 (discussion): the functional manager won't release the good engineer

In a weak matrix, your best-choice design engineer's functional manager keeps assigning them to department work and offers you a junior instead. Using the culture and structure ideas in this chapter, explain why this happens and what you can do.

Discussion points

The functional manager is **evaluated on their department's output**, so they hoard their strongest people — a reward-system effect. In a weak matrix you have little formal authority to override it. Levers: escalate to the sponsor (whose NPV depends on the project), negotiate a part-time or time-boxed assignment, trade a deliverable the functional manager cares about, or get the resource commitment written into the **charter** so it is a documented agreement rather than a favour.

3.13 Further Resources

What Is Scope Creep? (Explained in 3 Minutes): https://www.youtube.com/watch?v=30a6zZP5EVI

Scope Creep in Project Management: Causes, Examples, Prevention: https://www.youtube.com/watch?v=HHey0EXZR0w

Chapter 4

Leadership and the Project Manager

4.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Distinguish** leadership from management in a project context.
2. **Explain** the four foundations of a project partnership and Bennis's four competencies of the "new project leader."
3. **Match** a leadership style (command & control, relating, democratic, hands-on, coaching) to a team situation, and state each style's downside.
4. **Score** a leadership self-assessment and interpret the result.
5. **Describe** the stages of team development and what the leader does in each.
6. **Apply** motivation theory — Maslow, Herzberg, and expectancy — to diagnose a performance problem.
7. **Identify** the five components of emotional intelligence and why they matter to a project leader.

Before we begin: recall a team you would follow anywhere

Who led it, and what did they *do* — not what title they held? Most people describe behaviour, not authority. That is the whole chapter in one observation.

4.2 Introduction: the project manager as mini-CEO

Leading the RC-vehicle project means more than tracking a Gantt chart. The brief has your team learning a **new assembly line** with a real learning curve,

working with an **unproven supplier**, and racing a **holiday deadline**. The PM *"encapsulates the role of a mini-CEO"* — expected to manage holistically from start to finish: acquiring resources, motivating and building the team, holding a vision *and* fighting fires, and communicating relentlessly. *"A PM cannot get up in the morning and think 'I don't want to communicate today.'"*

> **Note**
>
> **Industry snapshot.** On an automotive launch, the line lead who can only give orders stalls the moment a novel problem appears; the one who has built a team that raises issues early hits the job-one date. In food processing, a sanitation crew that understands *why* the changeover procedure matters follows it at 3 a.m. when no one is watching. Leadership is what fills the gap between the procedure and reality.

Why Good Leaders Make You Feel Safe (Simon Sinek, TED): https://www.youtube.com/watch?v=lmyZMtPVodo

4.3 Leadership versus management

Leadership is *the ability to inspire confidence and support among the people needed to achieve organizational goals.* A **manager** holds a title that lets them *plan, organize, direct, and control* others. Leadership may be part of the manager's job, but the other roles are more administrative.

Leaders create a sense of **partnership**. Four things build it:

Foundation	What it means
Exchange of purpose	Every member knows the project's goals and vision.
A right to say "no"	Members can voice professional disagreement. *Losing an argument is acceptable; losing the right to disagree is not.*
Joint accountability	Every member owns the project's outcomes — good or bad.
Absolute honesty	All information becomes community information.

4.3.1 Bennis's four competencies of the new project leader

1. **The power of appreciation** — recognize and reward others' talent; your job is not to outshine the team but to let it develop.

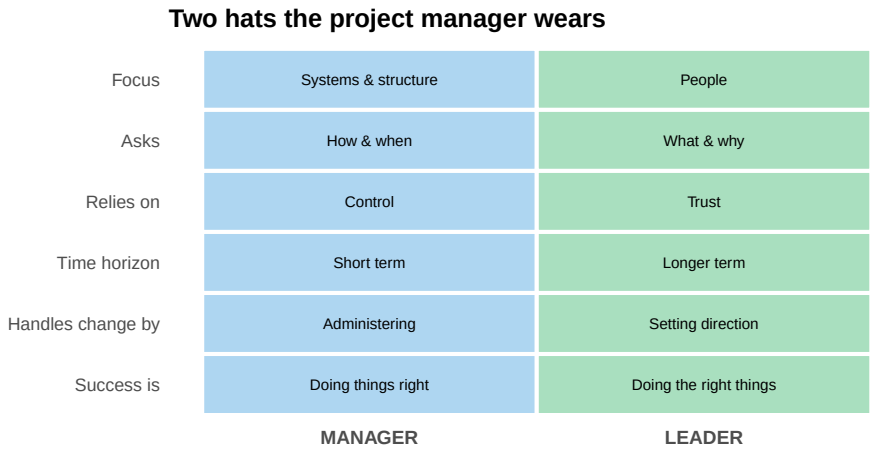


Figure 4.1: Manager and leader emphases. Neither column is 'bad' — a project manager needs both, but leans on the right-hand behaviours during execution.

2. **Keep reminding people what's important** — amid the noise of problems, keep everyone's eye on the prize.
3. **Generate and sustain trust** — through authenticity, fairness, honesty, and caring. Honesty is the trait people look for most.
4. **The leader and the led are intimate allies** — leadership supports the team's effort; it does not dominate it.

4.4 The five leadership styles

Effective PMs switch styles to fit the person and the situation.

Style	Use it when	The downside if overused
Command & control	You know the best way and the other person doesn't; beginners; a crisis	Team feels devalued; learned helplessness; kills creativity and initiative
Relating	Building rapport, bonding the team, focusing people on common goals	Can't make tough calls or take charge in a crisis; seen as weak
Democratic	*Planning* — getting buy-in and reality-based plans from those who will execute	*Execution* often needs autocracy; consensus-building wastes a critical deadline

Style	Use it when	The downside if overused
Hands-on	You must step in and model the way to get a stuck task done	Disempowering; low morale; you become the bottleneck
Coaching	Long-term development; asking questions that lead people to their own solutions	Slow for short-term objectives — but it is the style most managers most need to build

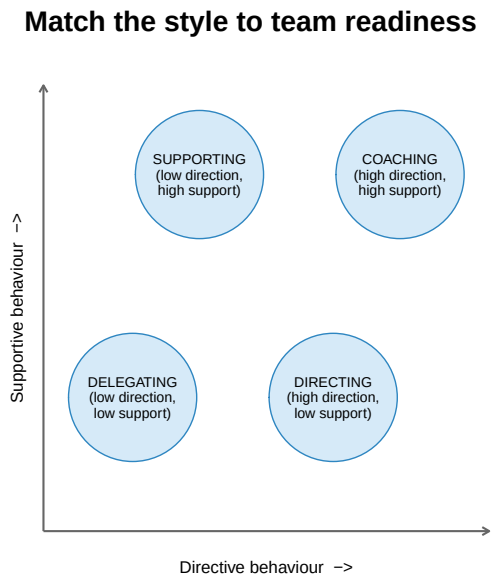


Figure 4.2: Situational leadership: match the style to how ready the team is. New or struggling teams need direction; capable, committed teams need delegation.

4.4.1 Try it yourself: pick the style

Which leadership style fits each moment?

Pick the style that fits best for each situation.

Categories: Command & control / Relating / Democratic / Hands-on / Coaching

1. A fire-alarm panel fault at 2 a.m.; the line must restart in 20 minutes.

2. Kick-off of planning: you need buy-in from the people who will do the work. _____
3. Two engineers have barely spoken since a design dispute, and the goal is slipping from view. _____
4. A task is stuck, so you sit down and show how the weld sequence is done. _____
5. A promising junior stalls on supplier negotiation; you ask questions until they find an approach. _____
6. A brand-new co-op student has never used the company's PLM system. _____
7. The team is bruised after a missed milestone and needs its trust rebuilt. _____
8. A capable designer wants to grow, and the deadline is months away. _____

Answers: 1 Command & control; 2 Democratic; 3 Relating; 4 Hands-on; 5 Coaching; 6 Command & control; 7 Relating; 8 Coaching.

4.4.2 Try it yourself: which quadrant is this team member in?

Rate the person's **competence** on the specific task and their **commitment** to doing it, both 0–1, and the situational-leadership quadrant above follows directly. Live cell — try Owen from the case study below, then a veteran who is skilled but burning out.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: situational-quadrant-04
team_member <- "Owen (co-op student, mounting-tab idea)"
competence  <- 0.3   # 0-1: skill/experience on this specific task
commitment  <- 0.8   # 0-1: confidence and motivation to do it

# Readiness model: a newcomer who is keen but unskilled needs direction;
# one who is losing confidence as the work gets hard needs coaching.
style <- if (competence < 0.5 && commitment >= 0.5) {
  "Directing (high direction, low support)"
} else if (competence < 0.5 && commitment < 0.5) {
  "Coaching (high direction, high support)"
} else if (competence >= 0.5 && commitment < 0.5) {
  "Supporting (low direction, high support)"
} else {
  "Delegating (low direction, low support)"
```

```
}

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(NA, xlim = c(0, 1), ylim = c(0, 1), xlab = "Competence on this task", ylab = "Comm
     main = "Situational leadership quadrant")
abline(v = 0.5, h = 0.5, col = "#bdc3c7", lwd = 2)
text(c(0.25, 0.25, 0.75, 0.75), c(0.25, 0.75, 0.25, 0.75),
     labels = c("Coaching", "Directing", "Supporting", "Delegating"), col = "#7f8c8d",
points(competence, commitment, pch = 19, cex = 3, col = "#e74c3c")

cat(sprintf("Recommended style: %s\n", style))
cat(sprintf("Team member  %s\n", team_member))
cat(sprintf("Competence  %.1f\n", competence))
cat(sprintf("Commitment  %.1f\n", commitment))
```

4.4.3 Try it yourself: the leadership self-assessment

The course quiz has 18 statements rated 1 (strongly disagree) to 5 (strongly agree). Items **1, 4, 7, 10, 13, 16** sum to a **command & control** score; items **2, 5, 8, 11, 14, 17** to a **democratic** score; items **3, 6, 9, 12, 15, 18** to a **relating** score. Enter your 18 answers and run.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: leadership-score-04
# Replace with your own 1-5 answers to items 1..18, in order:
responses <- c(2, 4, 3, 2, 5, 4,   # items 1-6
               2, 4, 4, 2, 4, 4,   # items 7-12
               2, 5, 2, 2, 4, 3)   # items 13-18

cc  <- sum(responses[c(1, 4, 7, 10, 13, 16)])
dem <- sum(responses[c(2, 5, 8, 11, 14, 17)])
rel <- sum(responses[c(3, 6, 9, 12, 15, 18)])

scores <- c("Command & Control" = cc, "Democratic" = dem, "Relating" = rel)
cat(sprintf("Your dominant style: %s\n", names(which.max(scores))))
for (nm in names(scores)) cat(sprintf("%s  %g / 30\n", nm, scores[nm]))
cat("Remember: the best PMs deliberately use all three depending on the situation.\n")
```

4.5 Building the team

Teams develop through predictable stages, and the leader's job changes with each:

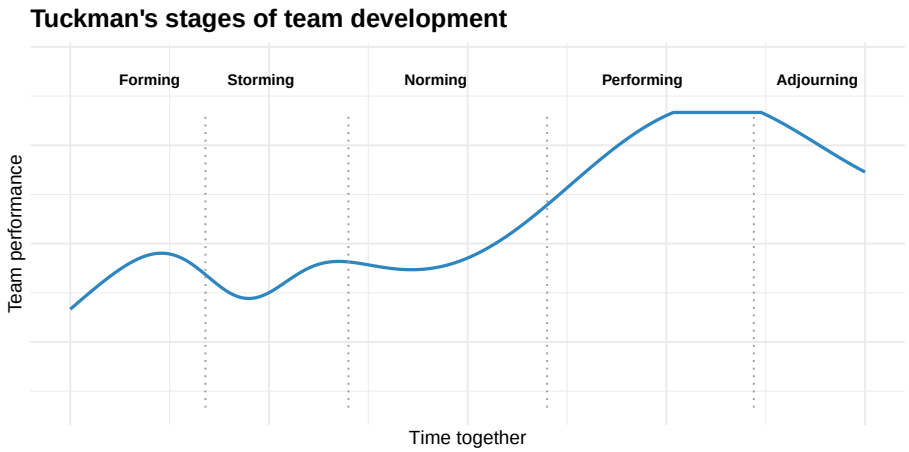


Figure 4.3: Team performance through Tuckman's stages. Performance dips during storming before rising; a leader who expects the dip manages it instead of panicking.

- **Forming** — polite, uncertain. Leader: **direct** — clarify goals, roles, the charter.
- **Storming** — conflict over approach and authority; performance dips. Leader: **coach** — surface conflict, don't suppress it.
- **Norming** — norms settle, trust builds. Leader: **support**.
- **Performing** — the team runs itself. Leader: **delegate**.
- **Adjourning** — the project ends; recognize contributions and manage the transition (see Chapter 13).

4.5.1 Try it yourself: which stage is the team in?

Which stage of team development does each observation show?

Pick the stage for each observation.

Categories: Forming / Storming / Norming / Performing / Adjourning

1. Everyone is polite and waits to be told what to do. _____
2. Two engineers argue over whose approach to follow, and output dips. _____
3. The team agrees how meetings run and how work is reviewed. _____

4. The team solves its own problems; the leader mostly removes obstacles. _____
5. The sample has shipped; the PM thanks everyone and handles reassignments. _____
6. The leader's first job is to clarify goals, roles and the charter. _____
7. Trust is building and norms have settled. _____
8. Whole workstreams are delegated to the team. _____

Answers: 1 Forming; 2 Storming; 3 Norming; 4 Performing; 5 Adjourning; 6 Forming; 7 Norming; 8 Performing.

4.6 Motivating the team

4.6.1 Worked Example 1: Herzberg — is this a hygiene problem or a motivator problem?

Herzberg splits job factors into **hygiene factors** (pay, conditions, policy, supervision — their *absence* causes dissatisfaction, but their presence does not motivate) and **motivators** (achievement, recognition, the work itself, responsibility, growth — these drive engagement).

Live cell — re-classify the complaints, or add your own, and see which get fixed first.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: herzberg-example-04
complaints <- data.frame(
  issue = c("Cramped, hot project room", "No recognition for hitting the drop-test mil
            "Confusing expense policy", "Same repetitive CAD cleanup every week",
            "Parking is far from the building"),
  type = c("Hygiene", "Motivator", "Hygiene", "Motivator", "Hygiene")
)

fix_first  <- complaints$issue[complaints$type == "Hygiene"]
then_build <- complaints$issue[complaints$type == "Motivator"]

cat(sprintf("%d hygiene issues to fix first, %d motivators to build\n",
            length(fix_first), length(then_build)))
for (x in fix_first)  cat("Fix first  ", x, "\n", sep = "")
for (x in then_build) cat("Then add  ", x, "\n", sep = "")
cat("Fixing hygiene stops dissatisfaction; only motivators raise engagement.\n")
```

4.6.2 Try it yourself: hygiene or motivator?

Is each complaint or wish a hygiene factor or a motivator?

Hygiene factors prevent dissatisfaction; motivators raise engagement.

Categories: Hygiene factor / Motivator

1. Broken heating in the project room _____
2. Being given ownership of the test plan _____
3. A confusing travel-expense policy _____
4. Public recognition for hitting the drop-test milestone _____
5. Slow, shared CAD licences _____
6. A chance to present results to the client _____
7. Noisy open-plan seating _____
8. The same repetitive data entry every week, with no variety _____

Answers: 1 Hygiene factor; 2 Motivator; 3 Hygiene factor; 4 Motivator; 5 Hygiene factor; 6 Motivator; 7 Hygiene factor; 8 Motivator.

4.6.3 Try it yourself: expectancy theory

Vroom's expectancy model says motivation is the product of three beliefs, each 0–1: **Expectancy** (effort → performance), **Instrumentality** (performance → reward), and **Valence** (how much the person wants the reward).

$$\text{Motivation} = E \times I \times V$$

Because it is a product, **any factor near zero kills motivation**.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: expectancy-04
team_member <- data.frame(
  name          = c("Priya", "Sam", "Dev"),
  expectancy    = c(0.9, 0.8, 0.4),   # "if I try, I can hit the target"
  instrumentality = c(0.8, 0.3, 0.9), # "hitting it gets me the reward"
  valence       = c(0.7, 0.9, 0.8)    # "I want that reward"
)
team_member$motivation <- with(team_member, expectancy * instrumentality * valence)

weakest <- team_member$name[which.min(team_member$motivation)]
cat(sprintf("Lowest motivation: %s\n", weakest))
factors <- c("Expectancy", "Instrumentality", "Valence")
for (i in seq_len(nrow(team_member))) {
```

```
  ev <- unlist(team_member[i, c("expectancy", "instrumentality", "valence")])
  cat(sprintf("%s  %.2f (weakest link: %s)\n", team_member$name[i],
              team_member$motivation[i], factors[which.min(ev)]))
}
cat("The weakest of E, I, V is where the leader intervenes.\n")
```

4.6.4 Try it yourself: expectancy by hand

Compute each person's motivation as E x I x V, then find who is least motivated. Each team picks its own **data set**.

Motivation = E x I x V

Three decimal places.

- Ana: E = 0.9, I = 0.6, V = 0.8
- Ben: E = 0.5, I = 0.9, V = 0.9
- Cal: E = 0.8, I = 0.2, V = 0.9
- Ana : motivation: _____ *(to 3 decimal places)*
- Ben : motivation: _____ *(to 3 decimal places)*
- Cal : motivation: _____ *(to 3 decimal places)*
- Lowest motivation score: _____ *(to 3 decimal places)*

Show the worked solution

Multiply the three factors for each person. In data set 1, Cal's motivation is 0.8 x 0.2 x 0.9 = 0.144: instrumentality is the weak factor, so the leader's job is to make the link from performance to reward visible. Because the model is a product, one factor near zero drags the whole score down.

4.7 Emotional intelligence

Goleman: *the most effective leaders all have a high degree of emotional intelligence* — IQ and technical skill are just the entry ticket. Five components:

- **Self-awareness** — a clear view of your strengths, weaknesses, drives, and ego needs.
- **Self-regulation** — keeping yourself under control; think before you act.
- **Motivation** — driven to your own potential *and* to drawing the best from each team member.
- **Empathy** — weighing team members' feelings when making a decision.
- **Social skill** — managing relationships with purpose.

4.8 Key Takeaways

- **Management** administers systems; **leadership** inspires people. A PM needs both and leans on leadership during execution.
- Partnership rests on **exchange of purpose, the right to say no, joint accountability, and absolute honesty**. Bennis: **appreciate, remind what matters, build trust, be an ally.**
- Switch among the **five styles** — command & control, relating, democratic, hands-on, coaching — by situation. Plan democratically; often execute autocratically. Every style overused has a cost.
- Teams move through **forming → storming → norming → performing → adjourning**; performance dips in storming — expect it and coach through it.
- Diagnose motivation with **Herzberg** (fix hygiene to stop dissatisfaction, add motivators to raise engagement) and **expectancy** ($E \times I \times V$ — any factor near zero kills it).
- **Emotional intelligence** — self-awareness, self-regulation, motivation, empathy, social skill — separates effective project leaders from merely competent ones.

4.9 Case Study: Meridian R/C — Week 4

CASE STUDY FILE — Week 4: The Whiteboard Moment

Four weeks into the Meridian Trailblazer prototype, the team gathers for its regular Tuesday design review — Priya Nandakumar (tooling), Marcus Webb (procurement), Sofia Reyes (quality and test), and Owen Matsuda, a co-op student newly rotated in to shadow the tooling work. The charter Daniel Cho signed in Week 3 is in hand, the aluminium body-and-frame decision from Week 1 is locked, and the team has moved on to something less settled: which motor and controller supplier to commit to, MCS or the newer Paragon. It is still an early, informal debate — no quotes on the table yet — but it is already loud. Marcus keeps steering the conversation toward Paragon's lower sticker price; Sofia keeps pulling it back to qualification risk. Positions are hardening, side conversations are breaking out, and twice now someone has cut off someone else mid-sentence. You recognize it the moment it starts: this is Tuckman's storming, arriving right on schedule. It is uncomfortable, but it is not a sign anything has gone wrong — yet.

Into that tension, Owen raises a hand and offers an idea about the frame's mounting-tab geometry, something he saw work on a project during a previous rotation. Priya, mid-argument with Marcus and not really listening, cuts him off — "that's not how we do it here" — and moves straight back to the supplier debate. The room goes quiet for a beat. The mounting-tab

idea was not the real loss; what a new team member just learned is that raising a suggestion here costs something.
You do not correct it in the room. You address it afterward, with each person separately, because the two problems call for two different tools. With Owen, you use a coaching style: you ask him to walk you through the mounting-tab idea in full, help him sharpen it into two concrete questions, and set aside five minutes at the next stand-up for him to raise it himself, in front of the team. With Priya, whose tooling judgment has been sound since the Week 1 material call and who does not need you supervising her every interaction, you use a delegating style: a short, low-drama flag that new voices need room to speak in these early weeks, then trust her to follow up with Owen in her own way. Neither conversation is a lecture. Both are sized to how ready each person already is — direction for the newcomer still finding his footing, room for the veteran who already knows how to lead.

4.10 Practice Problems

Problem 1: Pick the style

Which leadership style fits each moment on the RC-vehicle project?

1. Kicking off planning for the WBS with the whole team.
2. A moulding press fails at 2 a.m. and the launch date is at risk.
3. A capable senior designer is bored and thinking of leaving.
4. A brand-new co-op student has never used the company's PLM system.
5. The team has gelled and is hitting milestones on its own.

Answers

1. **Democratic** (buy-in, reality-based plan).
2. **Command & control** (crisis; take charge).
3. **Coaching** (long-term development; varied, owned work).
4. **Command & control** initially, moving to **coaching**.
5. **Delegating / supporting** — stay out of the way.

Problem 2: Expectancy diagnosis

A skilled technician has stopped putting in effort on validation testing. In a one-on-one you learn they are certain the extra effort won't change the delivery date because "purchasing will be late anyway." Which expectancy factor is broken, and what do you do?

Answer

Instrumentality — they no longer believe performance leads to the outcome. Restoring it means making the link visible again: show how the test results

actually gate the next milestone, remove the purchasing blocker or re-sequence around it, and close the loop when their work does move the date.

Problem 3 (discussion): the hands-on trap

You are the strongest CAD modeller on the team, and every time someone struggles you take the model over and finish it yourself. The work gets done and is high-quality. Why is this a leadership failure, and what should you do instead?

Discussion points

Taking over is the **hands-on** style overused: it is disempowering, tanks morale, stops people learning, and makes you the bottleneck — the project's throughput is now capped by your personal hours. The **coaching** move is to ask questions that lead the person to the fix, sit with them while they do it, and follow up — slower once, faster forever.

4.11 Further Resources

Why Leaders Eat Last (Simon Sinek, 99U): https://www.youtube.com/watch?v=ReRcHdeUG9Y

Chapter 5

Work Breakdown Structures (WBS)

5.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Explain** what a WBS is and how it links scope, schedule, and budget.
2. **Decompose** a project into deliverables, work packages, and activities.
3. **Apply** the 100 % rule, the mutually-exclusive rule, and "plan outcomes, not actions."
4. **Choose** an appropriate level of decomposition using the 8/80 and single-reporting-period rules of thumb.
5. **Number** a WBS and write a WBS dictionary entry for a work package.
6. **Roll up** cost and effort from work packages to the total project.
7. **State** the six requirements of a valid project contract and the purpose of work authorization.

Before we begin: how do you eat an elephant?

One bite at a time. *"Project managers cannot manage projects. Project managers manage pieces of projects."* The WBS is how you cut the elephant into bites.

5.2 Introduction: from a scope statement to something you can manage

You have a signed scope statement for the RC-vehicle replica. It is still far too big to schedule, budget, or assign. The **work breakdown structure** is the bridge: it takes the total scope and breaks it into **deliverables**, then **work**

packages, then **activities** small enough that one person can own each one and you can put a real number on it.

> **Definition.** A WBS is a **deliverable-oriented** grouping of project components that organizes and defines the **total scope** of the project.

A well-built WBS lets the team **plan every step**, **collect and organize** the information each task needs, and **assign** manageable chunks to team members. It also enables the **WBS dictionary**, which pins down the requirements for each task.

> **i** Note
>
> **Industry snapshot.** An automotive program WBS separates *body*, *powertrain integration*, *electrical*, *validation*, and *launch* — each a deliverable, not an action. A food-plant install WBS breaks out *utilities*, *process equipment*, *controls*, *sanitation qualification*, and *commissioning*. A defense contract WBS follows a mandated standard (MIL-STD-881) so the customer can track cost against the exact same structure the contractor uses.

Work Breakdown Structure (WBS) Explained: https://www.youtube.com/watch?v=B8q4uz5XiZc

5.3 The hierarchy

```
Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0
i Please use the `linewidth` argument instead.
```

- **Level 1** — the project. *What is the project?*
- **Level 2** — the summary level: the project's major deliverables.
- **Level 3** — the start of the **work package** level (more levels if the package is complex).

The same content can be shown as a **tree** (good for communicating structure) or a **tabular / indented list** (good for working). Both carry the numbering.

5.4 Design principles

5.4.1 The 100 % rule

The WBS includes **100 % of the work** in the scope — all deliverables, internal, external, and interim — and **no more**. At every level, the **sum of the children equals the parent**, and nothing outside the scope appears.

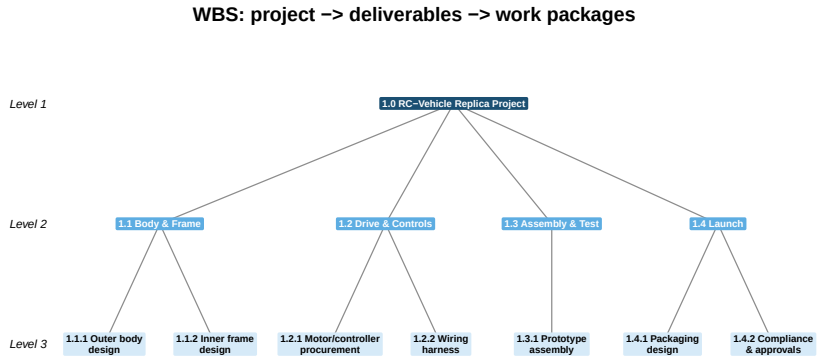


Figure 5.1: A partial WBS for the RC-vehicle project. Level 1 is the project; Level 2 the major deliverables; Level 3 the work packages. Numbering encodes the hierarchy.

5.4.2 Worked Example 1: does this WBS roll up to 100 %?

The "1.1 Body & Frame" deliverable is estimated at **340 hours**. Its four work packages are estimated at 120, 90, 70, and 60 hours.

Live cell — change the parent estimate or the work-package hours and watch the 100 % check.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: wbs-rollup-example-05
parent_hours <- 340
h_outer  <- 120   # 1.1.1 Outer body design
h_inner  <- 90    # 1.1.2 Inner frame design
h_select <- 70    # 1.1.3 Material selection
h_review <- 60    # 1.1.4 Design review & release

children_hours <- c("1.1.1 Outer body design" = h_outer,
                    "1.1.2 Inner frame design" = h_inner,
                    "1.1.3 Material selection" = h_select,
                    "1.1.4 Design review & release" = h_review)

child_sum <- sum(children_hours)
```

```
# Chart: the four children stacked beside the parent estimate
stack <- rbind(cbind(children_hours, 0), c(0, parent_hours))
colnames(stack) <- c("Children", "Parent")
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(stack, col = c("#2980b9", "#16a085", "#8e44ad", "#e67e22", "#34495e"), ylim = c
        main = "Do the children add up to the parent?")
abline(h = parent_hours, lty = 2, lwd = 3, col = "#e74c3c")

if (child_sum == parent_hours) {
  cat("OK: the children account for exactly 100% of the parent\n")
} else if (child_sum < parent_hours) {
  cat(sprintf("!!GAP: %g h of the parent's scope is in no work package!!\n", parent_hou
} else {
  cat(sprintf("!!OVER: the children exceed the parent by %g h (scope crept in)!!\n", ch
}
cat(sprintf("Sum of children  %g h\n", child_sum))
cat(sprintf("Parent estimate  %g h\n", parent_hours))
cat(sprintf("Children as share of parent  %.1f%%\n", 100 * child_sum / parent_hours))
```

Here the children sum to 340 — the deliverable is fully decomposed with nothing missing and nothing extra.

5.4.3 Mutually exclusive

No overlap between WBS elements. Overlap causes duplicated work, confusion over who is responsible, and messy cost accounting. If element names are ambiguous, the **WBS dictionary** disambiguates them.

5.4.4 Plan outcomes, not actions

Define elements as **results** (“Training module completed”), not **actions** (“write script”, “record video”, “upload”). Action lists always capture too many or too few steps — breaking the 100 % rule — and they over-prescribe *how* the work is done, killing the team’s ingenuity.

5.4.5 Try it yourself: outcome or action?

Is each WBS entry an outcome or an action?

Outcomes name a result; actions name a task.

Categories: Outcome (a good WBS element) / Action (rewrite it)

1. Compressor skid commissioned _____
2. Write the test report _____
3. Supplier selected and contracted _____
4. Call three vendors _____

5. Drawings released to manufacturing _____
6. Run the line for two shifts _____
7. Training module completed _____
8. Record the demo video _____

Answers: 1 Outcome (a good WBS element); 2 Action (rewrite it); 3 Outcome (a good WBS element); 4 Action (rewrite it); 5 Outcome (a good WBS element); 6 Action (rewrite it); 7 Outcome (a good WBS element); 8 Action (rewrite it).

5.4.6 Try it yourself: does it roll up to 100 %?

What is wrong with each decomposition, if anything?

Add the children and compare with the parent. Pick one verdict per card.

Categories: OK / Gap: scope missing / Over: children exceed parent / Overlap: not mutually exclusive

1. Parent 200 h; children 80 + 70 + 50 h _____
2. Parent 300 h; children 120 + 100 + 60 h _____
3. Parent 150 h; children 70 + 60 + 40 h _____
4. 'Prototype assembly' and 'Assemble prototype and rework' are listed under the same parent _____
5. Parent 420 h; children 150 + 140 + 130 h _____
6. Parent 90 h; children 40 + 30 h _____
7. Two packages both include 'prepare test fixtures' in their descriptions _____
8. Parent 250 h; children 100 + 90 + 80 h _____

Answers: 1 OK; 2 Gap: scope missing; 3 Over: children exceed parent; 4 Overlap: not mutually exclusive; 5 OK; 6 Gap: scope missing; 7 Overlap: not mutually exclusive; 8 Over: children exceed parent.

5.5 Level of detail

A work package at the **activity level** is a task that:

- can be **realistically and confidently estimated**;
- makes no practical sense to break down further;
- produces a **measurable deliverable**; and
- forms a unique package that **could be outsourced or contracted out**.

Rules of thumb for how far to decompose:

- **80-hour rule** — no work package (single deliverable at the lowest level) exceeds ~80 hours of effort.
- **One reporting period** — no package is longer than a single progress-reporting period (if you report monthly, nothing runs longer than a month).

- **8-hour floor** — generally do not define a task below ~8 hours; below that you are micromanaging.
- **"If it makes sense"** — apply common sense.

5.5.1 Try it yourself: decomposition check

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: wbs-decomp-05
packages <- data.frame(
  id    = c("1.1.1", "1.2.1", "1.3.1", "1.3.4", "1.4.2"),
  name  = c("Outer body design", "Motor/controller procurement",
            "Prototype assembly", "Tighten one fastener",
            "Full regulatory approval campaign"),
  hours = c(64, 40, 72, 1.5, 260)
)

flag <- ifelse(packages$hours > 80, "TOO BIG (> 80 h, split it)",
        ifelse(packages$hours < 8,  "TOO SMALL (< 8 h, roll it up)",
               "ok"))

n_bad <- sum(flag != "ok")
cat(sprintf("%s%d of %d packages need re-sizing%s\n",
            ifelse(n_bad > 0, "!!", ""), n_bad, nrow(packages), ifelse(n_bad > 0, "!!"
for (i in seq_len(nrow(packages)))
  cat(sprintf("%s %s  %s\n", packages$id[i], packages$name[i],
              ifelse(flag[i] == "ok", "ok", paste0("!!", flag[i], "!!"))))
```

5.5.2 Try it yourself: does it fit inside one reporting period?

The 8/80 rule checks **effort hours**; the one-reporting-period rule checks **calendar span** against how often you report — a different failure mode. A package can be well under 80 hours and still run longer than your reporting cycle if it is thin, spread-out work. Live cell — change the reporting cadence or a package's span and see which packages need to be split or tracked by percent-complete instead.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: wbs-reporting-period-05
reporting_cycle_days <- 14   # biweekly status reports

span_concept <- 10    # 1.1.1 Concept CAD
span_motor   <- 25    # 1.2.1 Motor/controller procurement
span_drop    <- 5     # 1.5.1 Drop/durability test
span_cert    <- 12    # 1.5.2 Toy-safety certification

packages <- data.frame(
  id        = c("1.1.1", "1.2.1", "1.5.1", "1.5.2"),
  name      = c("Concept CAD", "Motor/controller procurement",
                "Drop/durability test", "Toy-safety certification"),
  span_days = c(span_concept, span_motor, span_drop, span_cert)
)
packages$too_long <- packages$span_days > reporting_cycle_days
n_bad <- sum(packages$too_long)

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(packages$span_days, names.arg = packages$id, col = ifelse(packages$too_long, "#e74c3c", "
        ylim = c(0, max(packages$span_days, reporting_cycle_days) * 1.15),
        ylab = "Span (days)", main = "Work-package span vs. reporting cycle")
abline(h = reporting_cycle_days, lty = 2, lwd = 3, col = "#2c3e50")

if (n_bad > 0) {
  cat(sprintf("!!%d of %d packages need attention!!\n", n_bad, nrow(packages)))
} else {
  cat(sprintf("All %d packages fit within one reporting cycle\n", nrow(packages)))
}
cat(sprintf("Reporting cadence  every %d days\n", reporting_cycle_days))
for (i in seq_len(nrow(packages))) {
  verdict <- if (packages$too_long[i]) {
    sprintf("!!%g days: split or report %% complete!!", packages$span_days[i])
  } else {
    sprintf("%g days: fits within one cycle", packages$span_days[i])
  }
  cat(sprintf("%s %s  %s\n", packages$id[i], packages$name[i], verdict))
}
```

5.5.3 Try it yourself: is the package the right size?

Is each work package the right size? (status is reported weekly)

Apply the 8 h floor, the 80 h ceiling and the one-reporting-period rule.

Categories: About right / Too big: split it / Too small: roll it up / Longer than a reporting period

1. Fixture design: 40 h over 5 days _____
2. Full environmental test campaign: 210 h over 3 weeks _____
3. Tighten the guard screws: 1 h _____
4. Supplier qualification audit: 30 h of effort spread over 3 weeks of waiting for replies _____
5. Wiring harness layout: 72 h over 5 days _____
6. Label the sample boxes: 2 h _____
7. Complete tooling design: 150 h _____
8. Prototype shake-down: 24 h over 4 weeks of intermittent runs _____

Answers: 1 About right; 2 Too big: split it; 3 Too small: roll it up; 4 Longer than a reporting period; 5 About right; 6 Too small: roll it up; 7 Too big: split it; 8 Longer than a reporting period.

5.6 Why the WBS matters

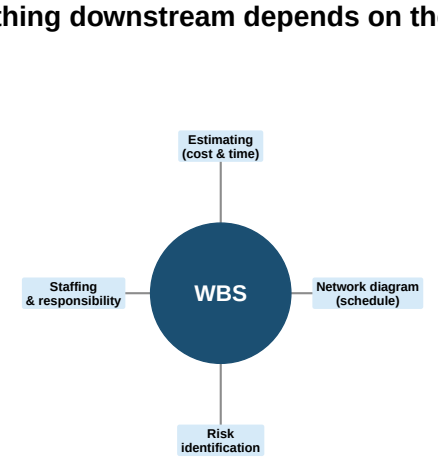


Figure 5.2: The WBS feeds four downstream planning activities. Get it wrong and the estimate, the schedule, the staffing plan, and the risk register are all built on sand.

5.7 The WBS dictionary

The dictionary is used to **control what work is done and when**, **prevent scope creep**, and **capture the effort** for each task. Physically, teams post

the work packages on a wall in logical work-flow order; **removing them in numerical order generates the dictionary**. Each entry — a **task description** — records:

- Task description (what work is authorized)
- Quantified objectives and deliverables (cross-referenced to the charter)
- Product description; acceptance criteria (how the owner checks their own work)
- Assumptions; resources assigned; duration; cost; due date
- Interdependencies (before this task / after this task)
- Project manager approval and date

5.8 Work authorization and contracts

Once the scope is decomposed into work packages, stakeholders formally agree on project content — the **work authorization**. If the client is external, this includes the **contract sign-off**.

A valid project-management contract requires all six of:

1. The legal **right** to contract
2. Legal **age** (18 in Ontario and Quebec; 19 or 21 elsewhere in Canada)
3. **Sound mind** at signing
4. Valid, legal **consideration** for both parties
5. A **legal scope of work**
6. **Offer and acceptance**

Contracts also commonly contain: excusable delays; allowable costs; **liquidated damages** (non-performance / penalty clauses); inspection criteria; responsibility for correcting defects; and dispute-resolution mechanisms.

5.8.1 Try it yourself: cost roll-up from the WBS

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: wbs-cost-rollup-05
# Work-package costs (direct), grouped by Level-2 deliverable.
wp <- data.frame(
  deliverable = c(rep("1.1 Body & Frame", 3),
                  rep("1.2 Drive & Controls", 2),
                  rep("1.3 Assembly & Test", 3),
                  rep("1.4 Launch", 2)),
  package = c("Outer body", "Inner frame", "Material selection",
              "Motor/controller", "Wiring harness",
```

```
              "Prototype assembly", "Drop test", "EMC test",
              "Packaging design", "Approvals"),
  cost = c(18000, 14000, 6000, 140000, 9000, 21000, 12000, 15000, 8000, 6000)
)

money <- function(x) paste0("$", format(x, big.mark = " ", scientific = FALSE, trim = T

by_deliverable <- tapply(wp$cost, wp$deliverable, sum)
cat("Level 1 (total project direct cost): ", money(sum(wp$cost)), "\n", sep = "")
for (d in names(by_deliverable))
  cat(sprintf("Level 2: %s  %s\n", d, money(by_deliverable[[d]])))
cat("Every dollar is counted once, and all of them, because the WBS obeys the 100% and
```

5.8.2 Try it yourself: roll the hours up the tree

The leaf work packages carry their hours. Work up the tree: each deliverable is the sum of its packages, and the project is the sum of the deliverables. Each team picks its own **data set**.

Charging-station kit: WBS hours

Level 1 is the project, Level 2 the deliverables, Level 3 the work packages.

Box	Values	Inputs from below
1.0 Charging-station kit	Hours = _____	1.1 Enclosure, 1.2 Electronics, 1.3 Test and release
1.1 Enclosure	Hours = _____	1.1.1 Housing design, 1.1.2 Tooling quote, 1.1.3 Prototype housing
1.1.1 Housing design	Hours = 60 h	-
1.1.2 Tooling quote	Hours = 35 h	-
1.1.3 Prototype housing	Hours = 70 h	-
1.2 Electronics	Hours = _____	1.2.1 PCB layout, 1.2.2 Firmware
1.2.1 PCB layout	Hours = 80 h	-
1.2.2 Firmware	Hours = 95 h	-
1.3 Test and release	Hours = _____	1.3.1 Compliance test, 1.3.2 Release review
1.3.1 Compliance test	Hours = 50 h	-

Box	Values	Inputs from below
1.3.2 Release review	Hours = 30 h	-

- Share of the project in 1.2 Electronics: _____ % *(to 1 decimal place)*

Show the worked solution

For data set 1: 1.1 = 60 + 35 + 70 = 165 h; 1.2 = 80 + 95 = 175 h; 1.3 = 50 + 30 = 80 h; the project is 165 + 175 + 80 = **420 h**. The electronics branch is 175 / 420 = 42 % of the work. The roll-up only works because every hour is in exactly one package: the 100 % rule and mutual exclusivity.

5.9 Key Takeaways

- A **WBS** is a **deliverable-oriented** decomposition of the **total scope** into deliverables → work packages → activities. PMs manage the pieces.
- **Numbering** encodes the hierarchy: Level 1 project, Level 2 deliverables, Level 3+ work packages.
- Obey the three design rules: **100 % rule** (children sum to the parent, never more), **mutually exclusive** (no overlap), and **plan outcomes, not actions**.
- Size work packages with the **8/80 rule** and the **one-reporting-period** rule; below ~8 h you are micromanaging, above ~80 h you cannot estimate confidently.
- The WBS feeds **estimating, scheduling, staffing, and risk** — errors propagate to all four.
- The **WBS dictionary** locks down each task's description, acceptance criteria, resources, and interdependencies, and helps prevent scope creep.
- **Work authorization** formalizes agreement on content; a valid contract needs all **six** legal elements and usually carries **liquidated-damages** and dispute clauses.

5.10 Case Study: Meridian R/C — Week 5

CASE STUDY FILE — Week 5: Cutting the Trailblazer Into Bites

You're three weeks into the Meridian Trailblazer prototype, and Daniel Cho's $800,000 charter ceiling is signed. It's time to stop describing the project and start decomposing it. This week you sat down with Priya Nandakumar (tooling), Marcus Webb (procurement), and Sofia Reyes (quality and test) to turn "build a prototype that passes Kestrel Retail Group's

durability testing and toy-safety certification" into a WBS the team can actually schedule, budget, and staff.

You started at Level 1 with **1.0 Meridian Trailblazer Prototype** and split it into seven Level-2 deliverables, each owned by a named function so nobody can later claim a piece of scope belongs to someone else:

- **1.1 Design & Engineering** — 1.1.1 Concept CAD, 1.1.2 Detailed design release
- **1.2 Tooling & Fabrication** — 1.2.1 Body tooling, 1.2.2 Frame tooling
- **1.3 Motor & Controls Integration**
- **1.4 Prototype Assembly**
- **1.5 Testing & Certification** — 1.5.1 Drop/durability test, 1.5.2 Toy-safety certification
- **1.6 Packaging & Client Sample Shipment**
- **1.7 Project Management** (cross-cutting)

Notice what this structure is *not*. It is not "call the tooling vendor" or "email Sofia the test plan" — every element names a deliverable or result (a released design, a qualified tool, an integrated drive system, a certified sample) rather than an action, exactly as the outcomes-not-actions rule demands. Priya pushed back once, wanting to fold "1.2 Tooling & Fabrication" and "1.3 Motor & Controls Integration" into one bucket since her team touches both — but that would blur ownership between tooling (hers) and the MCS/Paragon procurement decision (Marcus's), so you kept them mutually exclusive instead.

You also ran the 100% rule on 1.2: body tooling (1.2.1) and frame tooling (1.2.2) are the *only* children of Tooling & Fabrication — nothing about motor sourcing or assembly labour is allowed to hide inside that branch, and nothing about tooling is allowed to leak into 1.3 or 1.4. And you sized 1.7 Project Management as its own Level-2 element specifically so PM effort never gets smuggled into a technical deliverable's hours — the same mistake you'll later coach a student through in the practice problems. Sofia flagged that 1.5's two children stay far enough apart in scope — durability is physical stress testing, certification is regulatory sign-off — that a failure in one doesn't get miscoded as a failure in the other when Kestrel's 100,000-unit decision comes down to those two checkboxes.

5.11 Practice Problems

Problem 1: Fix the 100 % rule violation

The "1.3 Assembly & Test" deliverable is estimated at 300 h. Its packages are listed as: Prototype assembly (110 h), Drop test (60 h), EMC test (70 h), Prototype assembly rework (40 h), Write the test report (30 h), *and* "General project management (50 h)."

What is wrong, and how do you fix it?

Solution

The children sum to 360 h — 60 h **over** 100 % of the parent. Two problems: "Prototype assembly rework" overlaps "Prototype assembly" (not mutually exclusive — fold rework into the assembly package or make it a contingency), and "General project management" is **not part of this deliverable's scope** — PM effort belongs in its own Level-2 element (e.g. "1.0.1 Project Management"), not smuggled into a technical deliverable.

Problem 2: Outcome or action?

Rewrite these action-style entries as outcome-style WBS elements:

1. "Call three carbon-fibre suppliers and get quotes"
2. "Run the mould 20 times and measure the parts"
3. "Email the drawings to the customer for approval"

Model answers

1. "Carbon-fibre supplier selected and quoted."
2. "Moulding process capability established."
3. "Customer design approval obtained."

Each is a result you can mark done or not done, and it does not dictate the method.

Problem 3 (discussion): how deep should the RC-vehicle WBS go?

Your draft WBS has a work package "Design the outer body" estimated at 180 h and spanning 6 weeks, and another, "Deburr the frame jig", estimated at 2 h. Using the rules of thumb, what do you do with each?

Discussion points

"Design the outer body" breaks both the 80-hour rule and (if you report every 2 weeks) the single-reporting-period rule — split it into concept, detailed CAD, and design-release packages. "Deburr the frame jig" is below the 8-hour floor — roll it into the jig-fabrication package rather than tracking it as its own line.

5.12 Further Resources

What Is a Work Breakdown Structure? (Online PM Courses): https://www.youtube.com/watch?v=akO2Lf1fHmM

Work Breakdown Structure Basics: https://www.youtube.com/watch?v=_7R_jjxVpK4

Chapter 6

Network Diagrams and Activity-on-Node (AON)

6.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Build** an activity-on-node (AON) network from a precedence table.
2. **Apply** the rules for constructing a valid network diagram.
3. **Perform** the forward pass to find early start (ES) and early finish (EF).
4. **Perform** the backward pass to find late start (LS) and late finish (LF).
5. **Calculate** total float and identify the **critical path**.
6. **Compute** a three-point (PERT) activity duration.
7. **Choose** a strategy to shorten the critical path when the project is too long.
8. **Translate** a solved AON network into a Gantt chart.

Before we begin: which task can slip without moving the deadline?

On any project some tasks have slack and some do not. Guessing wrong — pouring effort into a task with float while a zero-float task quietly slips — is one of the most common ways schedules die. The network diagram tells you which is which.

6.2 Introduction: when can we launch?

The RC-vehicle WBS gives you a list of activities and their durations. It does *not* tell you how long the whole project takes, because the activities are not all independent — you cannot assemble a prototype before the body is designed. A **network diagram** captures those dependencies and answers the question the

sponsor actually cares about: **what is the earliest the project can finish, and which activities decide that date?**

> **Note**
>
> **Industry snapshot.** Every automotive program plan is a critical-path network: the "job one" date is the length of the longest dependent chain. A food-plant commissioning schedule uses the same math to decide whether sanitation qualification can overlap controls checkout. A defense contractor reports critical-path status to the customer monthly — slipping a critical task is a contractual event.

Forward and Backward Pass -> Project Duration and Critical Path (Engineer4Free): https://www.youtube.com/watch?v=4oDLMs11Exs

6.3 From WBS to precedence table

For each activity you need its **duration** and its **immediate predecessors** — the activities that must finish before it can start. Rules for a valid AON network:

1. Determine precedence ordering **before** drawing.
2. Networks flow **left to right**.
3. An activity cannot begin until **all** connected predecessors are complete.
4. Arrows show **precedence and logical flow** (they may cross, but minimize it).
5. Every activity has a **unique identifier**.
6. **No looping** back to an earlier activity.
7. It is common (not required) to start from a **single start node**…
8. …and end at a **single end node**.

6.3.1 Try it yourself: is this network valid?

The rules above are easy to recite and easy to break. Decide which rule, if any, each description breaks. (More than one start node, or more than one end node, is *allowed* — rules 7 and 8 only say it is common.)

Valid network, or which rule does it break?

Pick one for every card.

Categories: Valid / Loop (rule 6) / Duplicate ID (rule 5) / Starts too early (rule 3)

1. Activity C lists B as its predecessor, and B lists C as its predecessor. _____
2. Two different activities are both labelled F: a 'Paint' and a 'Fit-out'. _____

3. H needs both D and E to finish, but the network has H starting as soon as D finishes. _____
4. A and B both have no predecessors, and both feed C. _____
5. B and C both start when A finishes, and run in parallel. _____
6. F starts only when both D and E are complete. _____
7. G follows F, F follows H, and H follows G. _____
8. Two separate end activities, K and L, with nothing after either. _____

Answers: 1 Loop (rule 6); 2 Duplicate ID (rule 5); 3 Starts too early (rule 3); 4 Valid; 5 Valid; 6 Valid; 7 Loop (rule 6); 8 Valid.

6.4 AON node anatomy

Each activity is a box with six numbers:

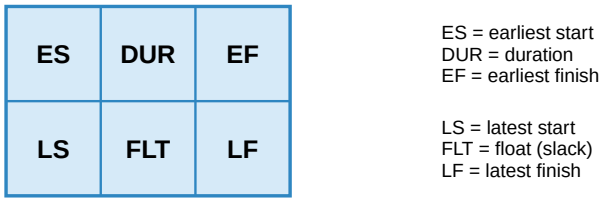


Figure 6.1: The six-number activity node. Forward pass fills the top row (ES, DUR, EF); backward pass fills the bottom (LS, FLT, LF).

An activity with 2+ predecessors flowing **in** is a **merge**; one with 2+ successors flowing **out** is a **burst**.

6.5 Worked Example: the RC-vehicle sub-project network

Ten activities (a simplified slice of the project):

Activity	Duration (days)	Predecessors

A	5	
B	8	A
C	3	B
D	4	A
E	5	C
F	7	C
G	6	D,E
H	8	F
I	6	G
J	5	H,I

6.5.1 Forward pass (ES, EF)

Move left to right. $EF = ES + DUR$. Carry each EF forward as the successor's ES — **except at a merge, where the ES is the largest incoming EF.**

6.5.2 Backward pass (LS, LF)

Move right to left from the project finish. $LS = LF - DUR$. Carry each LS back as the predecessor's LF — **except at a burst, where the LF is the smallest outgoing LS.**

6.5.3 Float and the critical path

$$FLT = LS - ES = LF - EF$$

Activities with **zero float** cannot slip without delaying the whole project — together they form the **critical path**.

Live cell — a base-R critical-path solver. Run it, then change a duration or a predecessor in `acts` and watch the project duration and the critical path move.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: cpm-solve-06
# The ten activities: id, duration, and comma-separated predecessors.
acts <- data.frame(
  id = LETTERS[1:10],
  dur = c(5, 8, 3, 4, 5, 7, 6, 8, 6, 5),
  pred = c("", "A", "B", "A", "C", "C", "D,E", "F", "G", "H,I"),
  stringsAsFactors = FALSE
)
```

```
# A small critical-path solver (base R only).
cpm <- function(df) {
  df$pred <- strsplit(gsub(" ", "", df$pred), ",")
  n <- nrow(df); id <- df$id
  ES <- EF <- LS <- LF <- setNames(rep(NA_real_, n), id)

  # Forward pass (ids are already in a valid topological order here)
  for (i in seq_len(n)) {
    p <- df$pred[[i]]; p <- p[p != ""]
    ES[i] <- if (length(p) == 0) 0 else max(EF[p])
    EF[i] <- ES[i] + df$dur[i]
  }
  proj <- max(EF)

  # Backward pass
  for (i in rev(seq_len(n))) {
    succ <- id[sapply(df$pred, function(pp) id[i] %in% pp)]
    LF[i] <- if (length(succ) == 0) proj else min(LS[succ])
    LS[i] <- LF[i] - df$dur[i]
  }

  data.frame(Activity = id, Dur = df$dur,
             ES = ES, EF = EF, LS = LS, LF = LF,
             Float = LS - ES,
             Critical = ifelse(LS - ES == 0, "  <- critical", ""))
}

res <- cpm(acts)
cat(sprintf("Project duration: %g days\n", max(res$EF)))
cat("Critical path  ", paste(res$Activity[res$Float == 0], collapse = " - "), "\n", sep = "")
for (i in seq_len(nrow(res)))
  cat(sprintf("%s  start %g, finish %g, float %g%s\n", res$Activity[i], res$ES[i], res$EF[i],
              res$Float[i], ifelse(res$Float[i] == 0, "  !!critical!!", "")))
```

The whole forward/backward/float sequence is one connected calculation — change any duration and every downstream number moves — so work it as a block, not as isolated steps.

6.5.4 Try it yourself: fill in the AON node boxes

Now do the math yourself, by hand, on a fresh network. It is a small renovation sub-project — seven activities — and it is **pre-drawn**: every node already shows its activity ID and duration, and the arrows already show precedence, so you do not need to draw anything. Your job is the forward and backward pass: type ES, EF, LS, and LF straight into each box (float fills itself in once a box has both ES and LS). Click **Check my numbers** as often as you like — correct

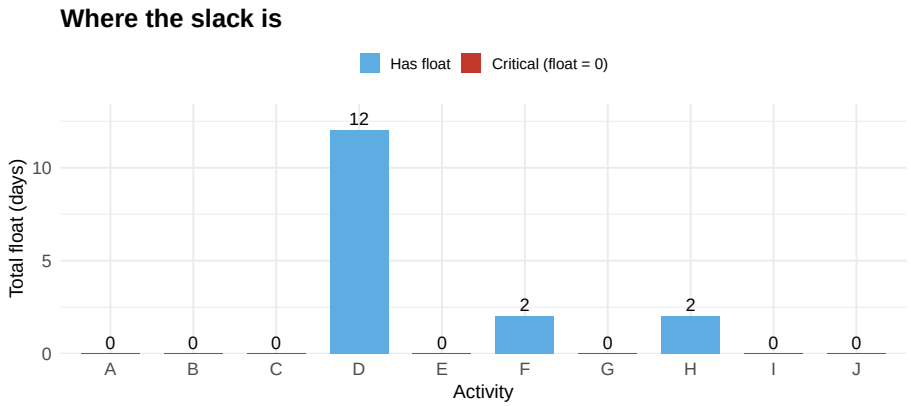


Figure 6.2: Total float per activity. The zero-float bars (A, B, C, E, G, I, J) are the critical path; D has 12 days of slack, F and H have 2.

cells turn green, wrong ones turn red. Once you are confident, click every node you think belongs on the **critical path** and then click **Check critical path**.

Activity	Description	Duration (days)	Predecessors
A	Site prep	4	
B	Foundation pour	6	A
C	Utility rough-in	3	A
D	Framing	5	B
E	Inspection sign-off	4	C
F	Drywall & finish	2	D,E
G	Final walkthrough	3	F

A

4

B

6

C

3

D

5

E

4

F

2

G

3

Check my numbers

Check critical path

Reset

Show the fully solved network

Activity	Dur	ES	EF	LS	LF	Float	Critical
A	4	0	4	0	4	0	<- critical
B	6	4	10	4	10	0	<- critical
C	3	4	7	8	11	4	
D	5	10	15	10	15	0	<- critical
E	4	7	11	11	15	4	
F	2	15	17	15	17	0	<- critical
G	3	17	20	17	20	0	<- critical

Project duration: 20 days. Critical path: A-B-D-F-G. C and E each carry 4 days of float — they can slip into that cushion without moving the finish date, which is exactly why they are not on the critical path even though they sit "in the middle" of the network.

6.6 Three-point (PERT) duration estimates

PERT stands for **Program Evaluation and Review Technique**. The U.S. Navy developed it in the late 1950s for the Polaris submarine-missile program, where almost nothing had been built before and no activity duration was really "known." PERT pairs with CPM (the forward/backward pass you just did,

which assumes each duration is a single fixed number): instead of one number per activity, PERT asks for three and blends them into an expected duration.

When a duration is uncertain, estimate three values — **optimistic** o, **most likely** m, **pessimistic** p — and combine them:

$$t_e = \frac{o + 4m + p}{6}$$

6.6.1 Try it yourself: PERT durations

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: pert-te-06
tasks <- data.frame(
  id = c("A", "B", "C", "D", "E"),
  o  = c(12, 7, 15, 9, 24),
  m  = c(15, 10, 19, 13, 26),
  p  = c(17, 13, 23, 16, 30)
)
tasks$te <- round((tasks$o + 4 * tasks$m + tasks$p) / 6, 2)
tasks$sd <- round((tasks$p - tasks$o) / 6, 2)   # activity standard deviation

cat(sprintf("Sum of expected durations: %.2f days\n", sum(tasks$te)))
for (i in seq_len(nrow(tasks)))
  cat(sprintf("%s  te %.2f days, sd %.2f\n", tasks$id[i], tasks$te[i], tasks$sd[i]))
```

Why the weight of 4 on the most-likely value?

PERT assumes each activity duration follows a Beta distribution skewed toward the most-likely value, with the optimistic and pessimistic values as the tails. Weighting m by 4 and o, p by 1 each (dividing by 6) is the standard approximation of that distribution's mean. The activity standard deviation $(p-o)/6$ becomes important in Chapter 8, where you combine them to find the probability of finishing on time.

Now do it by hand on a fresh set of estimates. Calculate each activity's expected duration and standard deviation, then total the expected durations along the chain.

PERT by hand: expected duration and standard deviation

Give each answer to two decimal places.

Activity	o	m	p	t_e	SD
A	4	6	14		
B	8	10	18		
C	3	5	7		
D	10	14	24		
E	2	3	8		

- Sum of expected durations, A to E: _____ days *(to 2 decimal places)*

Show the worked solution

Activity	o	m	p	t_e	SD
A	4	6	14	7.00	1.67
B	8	10	18	11.00	1.67
C	3	5	7	5.00	0.67
D	10	14	24	15.00	2.33
E	2	3	8	3.67	1.00

Activity D shows the skew: with o = 10, m = 14 and p = 24, the long tail pulls t_e up to 15 — a day above the most-likely value — and gives the largest standard deviation (2.33), so it is the activity most worth watching.

6.7 Shortening the critical path

Only work on the **critical path** shortens the project. Options, roughly in order of preference:

1. **Eliminate** critical tasks (is it truly needed?).
2. **Re-plan series paths to run in parallel.**
3. **Overlap** sequential tasks (start the successor before the predecessor fully finishes — *fast-tracking*).
4. **Shorten durations on critical activities** — prefer earlier-starting tasks, then the longest, then the easiest, then the ones **cheapest to speed up**.

6.7.1 Try it yourself: recompute the critical path after a change

Try it — crash B as far as you like: both long chains share it, so the critical path stays put. Then drag E's crashed duration down and watch the critical path flip to the other chain.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: path-duration-06
b_crashed_days <- 5    # B crashed from its normal 8 days
e_crashed_days <- 5    # E crashed from its normal 5 days (5 = not crashed)

# All start-to-finish paths through the sub-project network, with activity durations.
dur <- c(A = 5, B = 8, C = 3, D = 4, E = 5, F = 7, G = 6, H = 8, I = 6, J = 5)

paths <- list(
  "A-B-C-E-G-I-J" = c("A","B","C","E","G","I","J"),
  "A-B-C-F-H-J"   = c("A","B","C","F","H","J"),
  "A-D-G-I-J"     = c("A","D","G","I","J")
)

path_len <- sapply(paths, function(p) sum(dur[p]))

# Now crash B and E and re-check:
dur["B"] <- b_crashed_days
dur["E"] <- e_crashed_days
path_len2 <- sapply(paths, function(p) sum(dur[p]))

crit_before <- names(which.max(path_len))
crit_after  <- names(which.max(path_len2))

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(rbind(path_len, path_len2), beside = TRUE, names.arg = paste("Path", 1:3)
              ylim = c(0, max(path_len, path_len2) * 1.25), main = "Path length: before

cat(sprintf("New critical path: %s (%g days)
", crit_after, max(path_len2)))
cat(sprintf("Critical path shifted?  %s
", if (crit_before != crit_after) "Yes, to a different chain" else "No, same chain"))
for (i in seq_along(paths)) {
  cat(sprintf("Path %d: %s  %g d -> %g d
", i, names(paths)[i], path_len[i], path_len2[i]))
}
```

6.7.2 Try it yourself: which activities actually threaten the finish date?

Float answers this precisely: bump every activity by the same 2 days, one at a time, and see which ones move the finish date and which get absorbed. This is the Week 6 Meridian network from the case study below. Live cell — drag the

delay, or lengthen C or E, and watch which activities threaten the finish date.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: float-sensitivity-06
bump_days <- 2    # delay added to one activity at a time
c_days    <- 20
e_days    <- 25

dur  <- c(A = 10, B = 8, C = c_days, D = 15, E = e_days, F = 6, G = 5, H = 12, I = 4, J = 2)
pred <- list(A = c(), B = "A", C = "B", D = "B", E = "B",
             F = c("C", "D", "E"), G = "F", H = "F", I = c("G", "H"), J = "I")

project_duration <- function(dur, pred) {
  id <- names(dur)
  EF <- setNames(rep(NA_real_, length(id)), id)
  for (a in id) {
    p  <- pred[[a]]
    ES <- if (length(p) == 0) 0 else max(EF[p])
    EF[a] <- ES + dur[a]
  }
  max(EF)
}

baseline <- project_duration(dur, pred)

# Add the delay to one activity at a time and see what happens to the finish date.
impact <- setNames(numeric(length(dur)), names(dur))
for (a in names(dur)) {
  bumped  <- dur; bumped[a] <- bumped[a] + bump_days
  impact[a] <- project_duration(bumped, pred) - baseline
}

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(impact, col = ifelse(impact > 0, "#e74c3c", "#bdc3c7"), xlab = "Activity delayed", ylab =
        main = paste("Effect of a", bump_days, "day delay on the finish"))

cat(sprintf("Baseline project duration: %g days\n", baseline))
cat(sprintf("Critical activities  %s\n", paste(names(impact)[impact > 0], collapse = ", ")))
for (a in names(dur)) {
  cat(sprintf("+%g days to %s  finish moves %g day(s)%s\n", bump_days, a, impact[a],
              if (impact[a] == 0) " (float absorbs it)" else " (CRITICAL)"))
}
```

6.8 From network to Gantt chart

The AON network shows **logic** — what depends on what — but it is not drawn against a calendar, so it is easy to misjudge how much real time separates two activities, or which ones are actually running at the same time. Take the same **ES** and **EF** you just calculated in the forward pass and plot each activity as a bar running from its early start to its early finish on a day axis, and you get a **Gantt chart**: the identical schedule, now readable at a glance.

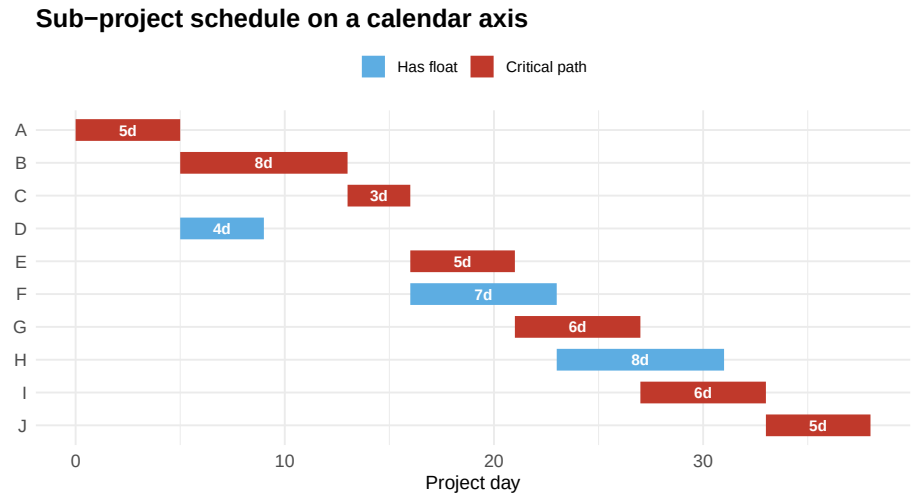


Figure 6.3: The ten-activity sub-project network, redrawn as a Gantt chart from the same ES/EF/float numbers as the float chart above. Dark bars (A, B, C, E, G, I, J) are the critical path; light bars (D, F, H) carry float.

Nothing here is new information — it is the same ten rows as the solved network table and the float chart above, just laid out so you can see **when** each activity runs and **which ones overlap**, not only how much slack each one carries. The merges (D and E both feeding G) and bursts are now visible as bars that start or end together, instead of arrows you have to trace.

The same trick works on the small renovation network from the node-fill exercise:

Notice the gap between C's bar ending and F's bar starting — that gap *is* C's float, drawn as empty space instead of a number in a box. A Gantt chart is the natural format for tracking progress against the plan once the project is under way, for assigning resources to specific calendar days, and for showing a sponsor the schedule without making them read a network diagram. Chapter 8 builds on this: precedence relationships beyond simple finish-to-start, leads and lags, and turning the single-number duration into a probability of finishing on time.

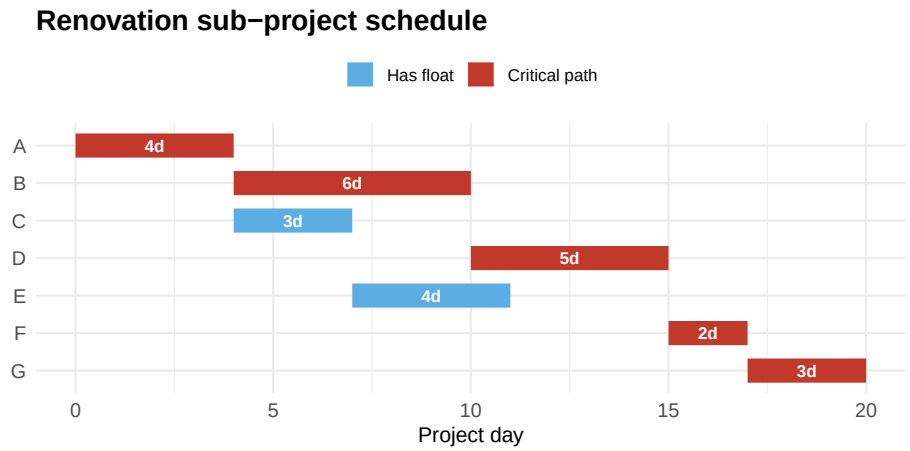


Figure 6.4: The seven-activity renovation network as a Gantt chart. Critical path A-B-D-F-G is dark; C and E (light) carry the float visible as the gap between where they finish and where F can actually start.

6.9 The downside of network diagrams

- Often **not drawn to scale** — time is just a number, which can create a false sense of how much time is available.
- Can be **too large and complex** to be useful.
- **Faulty construction logic** can oversimplify or misrepresent the work.
- Sometimes used for work they do not suit.
- **Positive bias** creeps into PERT estimates (people are optimistic).

6.10 Key Takeaways

- Build the **AON network** from a precedence table (durations + immediate predecessors); it flows left to right with no loops.
- **Forward pass**: $EF = ES + DUR$; at a **merge**, ES = the largest incoming EF. Gives the **project duration**.
- **Backward pass**: $LS = LF - DUR$; at a **burst**, LF = the smallest outgoing LS.
- **Float** $= LS - ES = LF - EF$. **Zero-float** activities form the **critical path** — the longest dependent chain, and the only one worth shortening.
- **PERT**: $t_e = (o + 4m + p)/6$, activity SD $= (p - o)/6$.
- Shorten the critical path by **eliminating, parallelizing, overlapping, or crashing** critical tasks — and re-check, because the critical path can shift.
- A **Gantt chart** plots the same ES/EF numbers on a calendar axis — same schedule, now readable at a glance and ready to track progress against.

6.11 Case Study: Meridian R/C — Week 6

CASE STUDY FILE — Week 6: The network comes together

Daniel Cho has been asking the same question in every steering call since the charter was signed: when does the Trailblazer prototype actually ship to Kestrel? This week you have a real answer, because you and Priya Nandakumar finally turned the "Prototype Build & Qualification" branch of the WBS into an activity-on-node network instead of a list of jobs on a whiteboard.

Ten activities, in the sequence the tooling, procurement, and testing work actually has to happen: concept CAD and detailed design release feed three parallel branches — Priya's body and frame tooling builds, and Marcus Webb's motor-and-controller procurement from MCS — which all have to finish before Sofia Reyes's team can assemble a prototype to test and certify. You run the forward pass first, activity by activity, and the merge at Prototype Assembly (F) is the one that matters: it cannot start until the *slowest* of tooling and procurement finishes, not the fastest. Motor and controller procurement (E) turns out to be the long pole at 25 days, arriving at the merge two full weeks after Marcus's frame tooling and five days after Priya's body tooling — which is exactly why the MCS relationship is worth protecting rather than swapping out on price alone. The backward pass confirms it: activities A, B, E, F, H, I, and J all land with zero float. That chain — concept CAD through detailed design, motor procurement, assembly, toy-safety certification, packaging, and your sign-off report to Renata Ford — is the critical path, and it sets the whole prototype phase at 67 working days, just under 13.5 weeks. Body tooling (C) has 5 days of cushion, frame tooling (D) has 10, and the durability test (G) has 7 — comfortable float today, but float that Priya and Sofia will need to manage carefully once resource assignments get made in a few weeks.

You flag one thing for the team before the meeting ends: activity E is both the longest single activity on the critical path and completely outside Meridian's control once the purchase order is placed. Anything that slows MCS down slows the whole project down. That single line in the network diagram is the reason procurement risk is going to dominate next week's risk-identification session.

Table 6.5: The Week 6 Meridian Trailblazer CPM network. Project duration: 67 working days.

Activity	Description	Duration (days)	Predecessors	ES	EF	LS	LF	Float
A	Concept CAD	10	—	0	10	0	10	0
B	Detailed design release	8	A	10	18	10	18	0
C	Body tooling build	20	B	18	38	23	43	5
D	Frame tooling build	15	B	18	33	28	43	10
E	Motor & controls procurement (MCS)	25	B	18	43	18	43	0
F	Prototype assembly	6	C, D, E	43	49	43	49	0
G	Drop/durability test	5	F	49	54	56	61	7
H	Toy-safety certification	12	F	49	61	49	61	0

I	Packaging & client sample ship	4	G, H	61	65	61	65	0
J	PM sign-off / report to Kestrel	2	I	65	67	65	67	0

6.12 Practice Problems

Problem 1: Solve this network

Activity	Predecessors	Duration
A	none	3
B	A	4
C	A	2
D	B	5
E	C	1
F	E	2
G	D, E	4
H	F, G	3
I	H	6

Find the project duration and the critical path.

Solution

```
p1 <- data.frame(
  id = LETTERS[1:9],
  dur = c(3, 4, 2, 5, 1, 2, 4, 3, 6),
  pred = c("", "A", "A", "B", "C", "E", "D,E", "F,G", "H"),
  stringsAsFactors = FALSE
)
r1 <- cpm(p1)          # reuse the solver defined earlier in this chapter
print(r1, row.names = FALSE)
cat(sprintf("\nProject duration: %d days\n", max(r1$EF)))
cat("Critical path:", paste(r1$Activity[r1$Float == 0], collapse = " - "), "\n")
```

```
 Activity Dur ES EF LS LF Float      Critical
        A   3  0  3  0  3     0   <- critical
        B   4  3  7  3  7     0   <- critical
        C   2  3  5  9 11     6
        D   5  7 12  7 12     0   <- critical
        E   1  5  6 11 12     6
        F   2  6  8 14 16     8
        G   4 12 16 12 16     0   <- critical
        H   3 16 19 16 19     0   <- critical
        I   6 19 25 19 25     0   <- critical

Project duration: 25 days
Critical path: A - B - D - G - H - I
```

Duration 25 days; critical path A–B–D–G–H–I. (The chain A–B–D–G is what pushes E, F, and C off the critical path — D's 5 days arriving at the G merge beat E's early finish by 6 days.)

Problem 2: PERT and the optimistic tail

An activity is estimated at $o = 6$, $m = 9$, $p = 24$ days.

(a) What is t_e? (b) Why is t_e noticeably larger than m? (c) What is the activity standard deviation?

Solution

(a) $t_e = (6 + 36 + 24)/6 = 11$ days. (b) The pessimistic tail is far from the most-likely value (24 vs 9) while the optimistic tail is close (6 vs 9), so the distribution is **right-skewed** and its mean sits above m.
(b) SD $= (24 - 6)/6 = 3$ days — a wide, risky activity.

Problem 3 (discussion): which task do you crash?

On the sub-project network, the critical path is A–B–C–E–G–I–J (38 days) and the sponsor needs 34. Crash data: B costs \$500/day to shorten (max 3 days), G costs \$1 200/day (max 2 days), E costs \$300/day (max 2 days). Which combination gets you to 34 days at least cost, and what do you check afterward?

Discussion points

You need 4 days. Cheapest first: E for 2 days (\$600), then B for 2 days (\$1 000) — total \$1 600 for 4 days. Avoid G at \$1 200/day. **Then re-run the network**: after removing 4 days from this chain, another path (e.g. A–B–C–F–H–J) may now be critical or co-critical, and crashing further would have to shorten *both*.

6.13 Further Resources

PERT / Three-Point Estimating Explained (Cornelius Fichtner): https://www.youtube.com/watch?v=AQ_Wr1Me___8

Three-Point Estimates and PERT (PMC Lounge): https://www.youtube.com/watch?v=u-9o-qa3HDg

Chapter 7

Risk Management

7.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Distinguish** risk from uncertainty, and threats from opportunities.
2. **Classify** a risk and write a well-formed risk statement.
3. **Run** a qualitative analysis with a probability–impact matrix.
4. **Compute** expected monetary value (EMV) and solve a simple decision tree.
5. **Select** a response strategy: accept, minimize, share, transfer, or reserve.
6. **Size** a contingency reserve and distinguish task contingency from managerial contingency.
7. **Explain** how FMEA supports project risk management, complete a simplified FMEA form, and compute a Risk Priority Number (RPN) from standardized Severity/Occurrence/Detection scales.

Before we begin: name one thing that could sink the RC-vehicle project

Now: how likely is it, and how bad would it be? You just did qualitative risk analysis. The rest of this chapter makes it systematic.

7.2 Introduction: the known unknowns

The RC-vehicle brief is a risk register waiting to be written. One candidate motor supplier is **unproven** and its staff are **negotiating a first labour contract**. The carbon-fibre supplier is **mid-restructuring** after a foreign acquisition. Choosing a **new assembly line** brings a **learning curve**. None of these is a certainty — but each is *identifiable*, and that is exactly what makes it a **risk** rather than an uncertainty.

> **Risk** = the probability or threat of loss, injury, or any negative occurrence caused by internal or external vulnerabilities, that **can be reduced by preemptive action**.
>
> An **uncertainty** is when *neither* the probability *nor* the mode of occurrence is known. If you do not know about it, you cannot act on it.

The core equation:

$$\text{Risk} = \text{Probability of event} \times \text{Impact of event}$$

Probability is usually a percentage; impact is dollars or schedule delay. **Risk and opportunity are mirror images** — an opportunity is an unforeseen event with a *positive* outcome, handled with the same tools for the opposite end.

> **Note**
>
> **Industry snapshot.** Aircraft makers such as Bombardier **minimize** part risk by forcing every significant supplier into direct contact with their quality department and auditing their processes. The European Space Agency and Airbus **share** the risk of a new airframe across nations because no one country can carry it. A fixed-price defense contract **transfers** budget-overrun risk to the vendor.

Expected Monetary Value (EMV) Explained Simply (Online PM Courses): https://www.youtube.com/watch?v=dD-Vygp_MiM

7.3 The four-stage process

1. **Risk identification** — determine the specific risk factors that can reasonably be expected to affect the project.
2. **Analysis of probability and consequences** — how likely, and how bad.
3. **Risk mitigation strategies** — steps to reduce the impact of the risks judged threatening enough to act on.
4. **Control and documentation** — track the risks and build a **knowledge base** for future projects from the lessons learned.

7.4 Risk identification and classification

Sort each identified risk into a class. A common (not exhaustive) list:

- **Financial** — material price moves, currency, budget cuts

- **Technology** — unproven design or process
- **Resources** — key people, equipment availability
- **Schedule** — dependency slips, long lead times
- **Weather**, **Political**, **Environmental**

Techniques: structured **brainstorming**, **interviewing** to get optimistic / most-likely / pessimistic views of an event, **probability distributions** to represent uncertain outcomes, and **expert judgment** to validate.

7.5 Qualitative analysis: the probability–impact matrix

Plot each risk by **likelihood** (x-axis) and **impact** (y-axis). The top-right is **red** (mitigation planned in advance), the middle band **yellow** (team decides whether to plan), the bottom-left **green** (just stay aware).

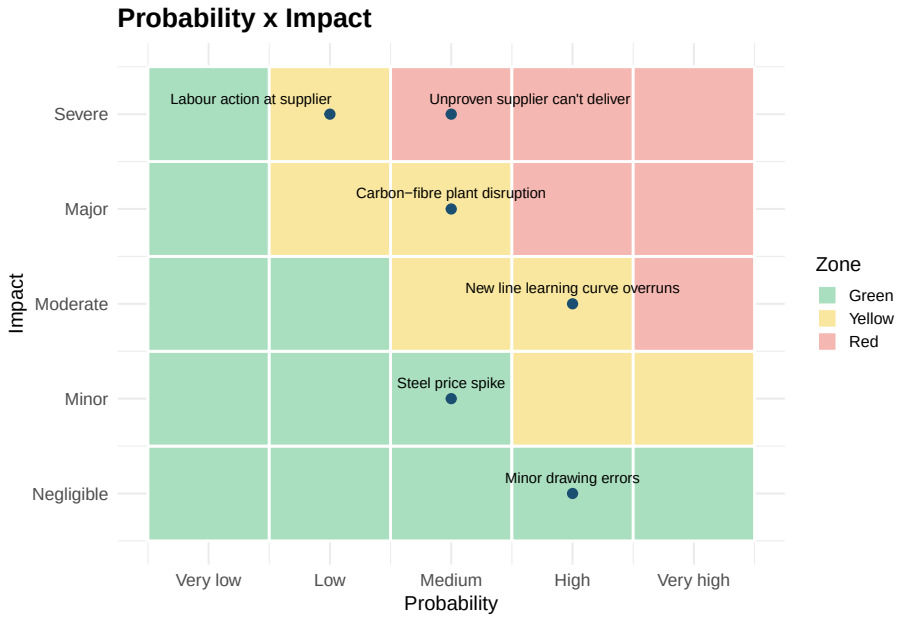


Figure 7.1: Probability-impact matrix for the RC-vehicle project. Red-zone risks (unproven supplier failure, carbon-fibre disruption) get a written mitigation plan before anything goes wrong.

7.5.1 Try it yourself: colour the risks

Zones follow probability x impact (each scored 1 to 5): up to 6 is **green**, 7 to 12 is **yellow**, above 12 is **red**.

Which zone is each risk in?

Multiply, then use the cut-offs 6 and 12.

Categories: Green / Yellow / Red

1. Likelihood 4, impact 4 _____
2. Likelihood 2, impact 3 _____
3. Likelihood 3, impact 4 _____
4. Likelihood 5, impact 1 _____
5. Likelihood 3, impact 5 _____
6. Likelihood 2, impact 5 _____
7. Likelihood 1, impact 5 _____
8. Likelihood 4, impact 2 _____

Answers: 1 Red; 2 Green; 3 Yellow; 4 Green; 5 Red; 6 Yellow; 7 Green; 8 Yellow.

7.6 Quantitative analysis

Tools: **sensitivity analysis** (which risks move the outcome most), **expected monetary value** (the probability-weighted average outcome), **decision trees** (lay out choices and chance events), and **Monte Carlo** simulation (run the network thousands of times with sampled durations/costs).

7.6.1 Worked Example 1: EMV of a single risk

The "new assembly line learning curve" risk: a **40 %** chance it adds **$120 000** of rework and overtime; otherwise $0.

Live cell — change the probability or the impact of the risk.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: emv-example-07
p_occurs <- 0.40
impact   <- 120000

emv <- p_occurs * impact + (1 - p_occurs) * 0

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

probs <- seq(0, 1, length.out = 100)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(probs * 100, probs * impact / 1000, type = "l", lwd = 4, col = "#2c3e50", xlab =
```

```
     main = "EMV grows with probability")
abline(v = p_occurs * 100, lty = 2, col = "#bdc3c7")
points(p_occurs * 100, emv / 1000, pch = 19, cex = 2.2, col = "#e74c3c")

cat(sprintf("EMV of this risk: %s\n", money(emv)))
cat(sprintf("Calculation  %.2f x %s\n", p_occurs, money(impact)))
```

7.6.2 Try it yourself: decision tree — buy the copper now or later?

You need **750 lb of copper** for the RC-vehicle wiring harness tooling. Buy it **now** and pay **$100** storage, or buy it **later** at the summer price — which is **$5.58/lb with 90 % probability**, or **$4.50/lb with 10 % probability**.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: decision-tree-07
lb <- 750
storage <- 100
price_now <- 5.58

# Branch: buy now (price locked, pay storage)
buy_now <- price_now * lb + storage

# Branch: buy later (two chance outcomes)
p_high     <- 0.90
price_high <- 5.58
price_low  <- 4.50
p_low      <- 1 - p_high
fmt2 <- function(x) formatC(x, format = "f", digits = 2, big.mark = " ")
buy_later <- p_high * (price_high * lb) + p_low * (price_low * lb)

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
emvs <- c("Buy now" = buy_now, "Buy later" = buy_later)
bp <- barplot(emvs, col = c("#2980b9", "#e67e22"), ylim = c(0, max(emvs) * 1.15),
              ylab = "Expected cost ($)", main = "EMV of each branch")
text(bp, emvs, labels = paste0("$", format(round(emvs), big.mark = " ")), pos = 3, cex = 1.3)

cat(sprintf("Choose: %s (saves $%s on average)\n",
            ifelse(buy_later < buy_now, "BUY LATER", "BUY NOW"), fmt2(abs(buy_now - buy_later))))
cat(sprintf("EMV, buy now  $%s\n", fmt2(buy_now)))
cat(sprintf("EMV, buy later  $%s\n", fmt2(buy_later)))
```

```
cat(sprintf("Price if high (%.0f%%)  $%s\n", 100 * p_high, fmt2(price_high * lb)))
cat(sprintf("Price if low (%.0f%%)  $%s\n", 100 * p_low, fmt2(price_low * lb)))
```

What would flip the decision?

Raise the probability of the low summer price, widen the price gap, or cut the storage cost toward zero and "buy later" wins by more. Push storage cost up (a bonded warehouse, insurance, spoilage) or make the high price nearly certain and "buy now" wins. A decision tree makes the tipping point explicit instead of leaving it to gut feel.

7.6.3 Try it yourself: a risk register by hand

Fill in the EMV for each risk, then total them to size the contingency reserve. Each team picks its own **data set**.

Risk register: expected monetary value

EMV = probability x impact.

Risk	Probability (%)	Impact ()|EMV()
Tool-steel delivery slips	30	40000
Supplier fails qualification	20	90000
Test lab double-booked	50	12000
Key engineer leaves	10	60000
Import exchange rate jumps	40	25000

- Total EMV (contingency reserve): $_____ *(to 0 decimal places)*
- EMV of the highest-ranked risk: $_____ *(to 0 decimal places)*

Show the worked solution

EMV = probability x impact for each row, for example 0.30 x $40 000 = $12 000 for the tool-steel slip. In data set 1 the EMVs are $12 000, $18 000, $6 000, $6 000 and $10 000: the total, **$52 000**, is the reserve, and the supplier-qualification risk ranks first at **$18 000** even though the tool-steel slip is the most likely of the large risks.

7.7 Response strategies

Strategy	What it means	Example on our project
Accept	The risk is minor or unlikely enough to live with	Small drawing errors — caught in review, low cost

Strategy	What it means	Example on our project
Minimize	Reduce probability or impact	Audit the unproven supplier; put their staff in contact with your quality team
Share	Split the risk (and reward) with a partner	Co-develop the new control system with the supplier under a shared-savings deal
Transfer	Shift the risk to another party	Fixed-price contract (vendor eats overruns); **liquidated damages** for non-performance
Contingency reserve	Set aside funds/time for the risk if it hits	10–15 % reserve against the carbon-fibre and learning-curve risks
Other	Anything else fit for the situation	Dual-source the motor; pre-qualify a backup

For **opportunities**, the mirror strategies are **exploit, share, enhance,** and **accept**.

7.7.1 Try it yourself: choose the response

Which response strategy is each action?

Pick the response strategy each action represents.

Categories: Accept / Minimize / Share / Transfer / Contingency reserve

1. Sign a fixed-price contract so the vendor absorbs cost overruns _____
2. Audit the unproven supplier and put your quality engineer on site _____
3. Co-develop the control system with a partner under a shared-savings deal _____
4. A cosmetic blemish on an internal bracket: log it and move on _____
5. Hold 12 % of the budget against the carbon-fibre disruption risk _____
6. Add a liquidated-damages clause for late delivery _____
7. Form a joint venture with another firm to build the new airframe _____
8. Pre-qualify a second motor supplier _____

Answers: 1 Transfer; 2 Minimize; 3 Share; 4 Accept; 5 Contingency reserve; 6 Transfer; 7 Share; 8 Minimize.

7.8 Contingency reserves

Two kinds:

- **Task (project) contingency** — offsets overruns on individual tasks/work packages. **Managed by the project manager.** As work completes, reserve for finished tasks is released and cannot be reused.
- **Managerial contingency** — a project-level buffer for scope changes forced by the customer, "acts of God," and other unforeseeable events. **Managed by senior management, not the PM.**

In construction it is common to set aside **10–15 %** of the price. Other industries resist it — the customer may read a visible reserve as a sign of poor planning.

7.8.1 Try it yourself: does the mitigation pay for itself?

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: mitigation-value-07
# Risk: unproven supplier fails to deliver on time.
p_fail_no_action <- 0.35
impact_if_fail   <- 260000       # expedite alternative source + line downtime

# Mitigation: pre-qualify a backup supplier now.
mitigation_cost  <- 22000
p_fail_with_action <- 0.10       # backup ready -> failure much less damaging
impact_with_action <- 90000      # switch to backup, smaller hit

emv_before <- p_fail_no_action * impact_if_fail
emv_after  <- mitigation_cost + p_fail_with_action * impact_with_action
saving     <- emv_before - emv_after

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

stack <- cbind("Do nothing" = c(0, emv_before),
               "Pre-qualify backup" = c(mitigation_cost, emv_after - mitigation_cost))
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(stack / 1000, col = c("#7f8c8d", "#e67e22"), ylab = "Expected cost ($ thousand)
        main = "Expected cost: spent now (grey) + loss (orange)")
```

```
if (saving >= 0) {
  cat(sprintf("Mitigation pays: expected saving %s\n", money(saving)))
} else {
  cat(sprintf("!!Mitigation does not pay: expected loss of %s!!\n", money(-saving)))
}
cat(sprintf("EMV, do nothing  %s\n", money(emv_before)))
cat(sprintf("EMV, pre-qualify backup  %s (incl. %s spent now)\n", money(emv_after), money(mitigat
```

7.9 FMEA as a risk tool

Failure Mode and Effects Analysis is often the first step of a reliability study: review as many components, assemblies, and subsystems as possible to identify **failure modes** (what can fail and how), their **causes**, and their **effects** (the consequences). It is a structured, bottom-up way to surface technical risks before they reach the customer — and it feeds the same probability–impact thinking used above. Instead of a single probability and a single dollar impact, FMEA scores each failure mode on three separate 1–10 scales — **Severity**, **Occurrence**, **Detection** — and multiplies them into a **Risk Priority Number (RPN)**.

7.9.1 The FMEA form

Every failure mode gets one row on a standard form: what the step is supposed to do, how it can fail, the effect of that failure, a severity rating, the likely cause, an occurrence rating, what currently catches it, a detection rating, the resulting RPN, and the action taken if the RPN is too high.

Table 7.1: FMEA form, one row, worked from the motor-controller risk.

Field	Example
Process Step	Install motor controller
Function	Deliver throttle signal reliably at all speeds
Failure Mode	Controller intermittently drops signal
Effect	Vehicle loses power mid-use; return; safety concern at speed
S	8
Cause	Unqualified solder joints from new (Paragon) supplier
O	6
Current Controls	Final functional test on assembly bench
D	5
RPN	8 x 6 x 5 = 240
Action	Require a qualification sample and burn-in test before contract (see Week 7 case study, R1)

7.9.2 Rating Severity, Occurrence, and Detection

Three people scoring the same failure mode should land on close to the same numbers, so each factor is rated against a **predefined table**, not a gut feel.

- **Severity (S)** — how bad is the *effect* if the failure happens? Rated on the effect, not the cause — so severity can really only be brought down by changing the design, not the process.
- **Occurrence (O)** — how likely is the *cause* to happen?
- **Detection (D)** — how likely are *current controls* to catch the failure before it reaches the customer? A 10 means no control exists at all; a 1 means the process is error-proofed so the defect physically cannot get out.

Table 7.2: Severity rating scale (S).

Rating	Criteria
10	Hazardous, no warning – safety/regulatory failure with no warning
9	Hazardous, with warning – safety/regulatory failure, warning given
8	Very high – product/service inoperable, loss of primary function
7	High – operable, but at reduced performance
6	Moderate – operable, degradation noticeable to most customers
5	Low – operable, noticed by the average customer
4	Very low – operable, noticed only by a discriminating customer
3	Minor – nonconforming fit/finish, noticed by a discriminating customer
2	Very minor – noticed only by a very observant customer
1	None – no discernible effect

Table 7.3: Occurrence rating scale (O).

Rating	Criteria
10	Very high – failure almost inevitable
9	Very high – failures occur very frequently
8	High – failures occur frequently
7	High – failures occur often, on a regular basis
6	Moderate – occasional failures
5	Moderate – a moderate number of failures
4	Low – relatively few failures
3	Low – few failures
2	Remote – failure unlikely
1	Remote – failure nearly impossible

Table 7.4: Detection rating scale (D).

Rating	Criteria
10	Absolute uncertainty – no current control exists
9	Very remote – control will almost certainly not detect
8	Remote – control has a poor chance of detection
7	Very low – control has a low chance of detection
6	Low – control may detect
5	Moderate – control has a moderate chance of detection
4	Moderately high – control has a reasonably good chance of detection
3	High – control has a good chance of detection
2	Very high – control will almost certainly detect
1	Almost certain – error-proofed; the defect cannot reach the customer

7.9.3 Calculating the Risk Priority Number

$$\text{RPN} = \text{Severity (S)} \times \text{Occurrence (O)} \times \text{Detection (D)}$$

RPN ranges from 1 (S=O=D=1) to 1000 (S=O=D=10). Rank failure modes by RPN and work down the list — but don't switch off judgment: a severity of 9 or 10 (a safety or regulatory hazard) usually earns action regardless of where the RPN lands, because two very different S–O–D combinations can multiply out to the same number.

Live cell — change S, O, or D for any failure mode and watch the ranking move.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: rpn-fmea-07
# Three failure modes from the RC-vehicle build

# 1. Motor controller drops signal (unproven Paragon supplier)
S1 <- 8
O1 <- 6
D1 <- 5
RPN1 <- S1 * O1 * D1

# 2. Carbon-fibre panel delaminates
S2 <- 9
O2 <- 3
D2 <- 6
RPN2 <- S2 * O2 * D2
```

```
# 3. Minor paint overspray on trim
S3 <- 2
O3 <- 5
D3 <- 3
RPN3 <- S3 * O3 * D3

modes <- c("Controller signal drop", "Panel delamination", "Paint overspray")
short <- c("Controller", "Panel", "Overspray")
sev   <- c(S1, S2, S3)
rpns  <- c(RPN1, RPN2, RPN3)
ord   <- order(rpns, decreasing = TRUE)

# RPN is a sort, not a verdict: flag any Severity >= 8 for action regardless of rank.
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(rpns[ord], names.arg = short[ord], col = ifelse(sev[ord] >= 8, "#e74c3c", "#298
        ylab = "RPN (S x O x D)", main = "Priority order by RPN")

cat(sprintf("Top priority: %s (RPN = %d)\n", modes[ord[1]], rpns[ord[1]]))
for (i in ord) {
  flag <- if (sev[i] >= 8) " - !!S >= 8: flag for action!!" else ""
  cat(sprintf("%s  RPN %d (S=%d, O=%d, D=%d)%s\n", modes[i], rpns[i], sev[i],
              c(O1, O2, O3)[i], c(D1, D2, D3)[i], flag))
}
```

Why doesn't the highest severity automatically win?

The panel delamination (S=9) has a *lower* RPN than the controller signal drop (S=8) here, because the controller's occurrence and detection are both worse. RPN is a useful sort, not a verdict: most teams still flag any S >= 8 for action even if its RPN is not the top of the list, the same way the probability–impact matrix earlier in this chapter still red-flags a low-probability event if its impact is severe enough.

7.9.4 Try it yourself: rank the failure modes

Calculate the RPN for each failure mode, rank them (1 = highest RPN), and flag the ones that earn action regardless of rank because Severity is 8 or more. Each team picks its own **data set**.

FMEA: RPN, rank and severity flag

A high severity earns action even when the RPN is not near the top.

Failure mode	S	O	D	RPN	Rank	Flag S >= 8 (Yes / No)
Hose clamp loosens	7	4	5			

Failure mode	S	O	D	RPN	Rank	Flag S >= 8 (Yes / No)
Sensor connector corrodes	9	3	6			
Label misprinted	3	6	2			
Seal extrudes from groove	8	5	4			
Fastener under-torqued	6	7	5			

Show the worked solution

For data set 1 the RPNs are 140, 162, 36, 160 and 210, so the order is fastener (1), connector (2), seal (3), clamp (4), label (5). The connector (S = 9) and the seal (S = 8) are flagged for action because severity is 8 or more; the seal would be flagged even if its RPN were lower. RPN is a sort, not a verdict.

7.10 Key Takeaways

- A **risk** is identifiable and can be acted on preemptively; an **uncertainty** cannot. **Risk = Probability × Impact.** Opportunities are the positive mirror.
- Four stages: **identify → analyze → mitigate → control & document** (build the knowledge base).
- **Qualitative**: plot risks on a **probability–impact matrix**; red-zone risks get a written plan *before* the event.
- **Quantitative**: **EMV** = probability × impact; **decision trees** compare choices with chance events; Monte Carlo for whole-network risk.
- Responses: **accept, minimize, share, transfer, reserve.** Pick per risk; ring-fence reserve against the biggest ones.
- **Task contingency** is the PM's; **managerial contingency** is management's, for the truly unforeseeable.
- **FMEA** systematically surfaces technical failure modes, causes, and effects, scoring each on **Severity, Occurrence, Detection** (predefined 1–10 tables) and multiplying them into an **RPN** to prioritize action.

7.11 Case Study: Meridian R/C — Week 7

CASE STUDY FILE — Week 7: The Motor-Supplier Risk Register

You call a formal risk workshop this week, before a single dollar is committed to a motor-controller contract. Priya Nandakumar, Marcus Webb, and Sofia Reyes join you around the whiteboard; Daniel Cho, mindful of the $800,000 charter ceiling he authorized back in Week 3, has asked for a dollar-costed register, not a list of worries. Kestrel Retail Group's 100,000-unit holiday order is still contingent on the Trailblazer prototype clearing durability testing and toy-safety certification, so anything that could derail either milestone earns a line on the register.

The motor-and-controller sourcing question is still wide open — that decision does not land until Week 10 — and it dominates the room. MCS, the trusted incumbent in Surrey, BC, is mid-move to a Mexican production line. Paragon, in Kingston, ON, is cheaper but unproven, and its staff are still negotiating their first labour contract. Marcus argues the team should keep an open mind toward Paragon's lower sticker price; Sofia counters that an unqualified controller is exactly the kind of risk that shows up in a durability-test failure report. Rather than settle the argument today, you convert it into a risk. Alongside it you register the MCS relocation, the carbon-fibre trim supplier's ongoing restructuring, and the ever-present chance that certification itself turns something up.

ID	Risk	Probability	Impact	EMV
R1	Paragon's unproven controller tech fails qualification	30%	$80,000	$24,000
R2	MCS's move to Mexico delays the first shipment	25%	$45,000	$11,250
R3	Fibro Source re-structuring delays the carbon-fibre trim package	20%	$30,000	$6,000

R4	Toy-safety certification finds a defect requiring redesign	15%	$100,000	$15,000

R1 posts the highest EMV of the four — not because it carries the biggest single impact (R4's $100,000 redesign is worse on paper) but because its probability is twice R4's. That is the number Sofia holds up in the meeting: whichever way the Week 10 sourcing decision goes, Paragon cannot be treated as a drop-in substitute without a qualification sample first.
Response strategies, agreed before anyone leaves the room:

- **R1** — mitigate (require a qualification sample before signing any contract) or avoid entirely (stay with MCS).
- **R2** — accept, backed by a schedule contingency buffer.
- **R3** — transfer (a penalty clause in the Fibro Source contract) or mitigate (line up a backup carbon-fibre source).
- **R4** — mitigate, by running an internal pre-test ahead of the formal certification submission.

Nothing here forces the MCS/Paragon call yet — that negotiation is still three weeks away — but the register now puts a number on waiting too long to decide.

7.12 Practice Problems

Problem 1: The road-trip risk register (from the course exercise)

Six friends borrow an older car and drive to an unfamiliar beach. The weather forecast is mixed with a chance of rain and high winds; one friend is a very picky eater; the owner plans to replace the car soon due to problems; there is a fireworks show and all-day music at a crowded venue. List the risks, classify each, and give a mitigation strategy.

Model answer (partial)

Risk	Class	Strategy
Get lost / arrive late	Schedule	*Minimize* — offline map, leave early
Car breaks down	Technology / Resources	*Minimize* (check oil, tires) + *transfer* (roadside assistance)

Risk	Class	Strategy
Rain / high winds spoil the day	Weather	*Accept* + *minimize* (pack a tarp, check the radar)
Picky eater has nothing to eat	Resources	*Minimize* — bring a known-safe option
No parking at a crowded venue	Resources	*Minimize* — arrive before the crowd
Injury in the crowd / fireworks	Environmental / Safety	*Minimize* — agree a meeting point, stay together

Problem 2: EMV of competing suppliers

Supplier X: \$140 000, 15 % chance of a 3-week slip costing \$45 000. Supplier Y: \$128 000, 35 % chance of a 3-week slip costing \$45 000. On expected cost alone, which is cheaper?

Solution

```
X <- 140000 + 0.15 * 45000
Y <- 128000 + 0.35 * 45000
cat(sprintf("Supplier X expected cost: $%s\n", format(X, big.mark = " ")))
cat(sprintf("Supplier Y expected cost: $%s\n", format(Y, big.mark = " ")))
cat(sprintf("Cheaper on EMV: Supplier %s\n", ifelse(X < Y, "X", "Y")))
```

```
Supplier X expected cost: $146 750
Supplier Y expected cost: $143 750
Cheaper on EMV: Supplier Y
```

X: \$146 750. Y: \$143 750. Y is cheaper on expected cost — but note X's risk is *less likely*, which may matter more than the \$3 000 if a slip would blow the holiday launch window. EMV informs the decision; it does not make it.

Problem 3 (discussion): red-zone risk with no cheap mitigation

The "carbon-fibre plant disruption" risk sits in the red zone, but the supplier is the only qualified source and re-qualifying another would take four months you do not have. What are your options?

Discussion points

You cannot *minimize* the probability much, so work the **impact**: build a larger **contingency reserve** (time and money) against it; **transfer** part of it via contract terms (penalty clauses, guaranteed allocation); **share** it by co-investing in the supplier's modernization for priority supply; carry **safety stock** of the critical carbon-fibre parts; and design a **fallback** (an aluminium body variant) that could be substituted if the supply fails, accepting a heavier product rather than a missed launch.

7.13 Further Resources

Decision Tree Analysis for Calculating EMV (PMC Lounge): https://www.youtube.com/watch?v=vmkdHT9VmZg

Decision Tree and Expected Monetary Value (Cornelius Fichtner): https://www.youtube.com/watch?v=uC2XBGpIcAs

Chapter 8

Project Scheduling

8.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Read and build** a Gantt chart and explain what it adds over a network diagram.
2. **Distinguish** the four precedence relationships (FS, FF, SS, SF) and use leads and lags.
3. **Combine** PERT activity estimates into a project mean and standard deviation.
4. **Calculate** the probability of finishing by a target date.
5. **Compute** an activity's crash cost slope and build a **least-cost crashing** sequence.
6. **Compare** crashing and fast-tracking, and state the risks of each.

Before we begin: a schedule is a promise — how confident are you in it?

"The project finishes in 83 days" sounds precise. Is that a date you would bet your bonus on, or a coin flip? This chapter turns a single-number schedule into a probability you can actually stand behind.

8.2 Introduction: from a network to a calendar

The AON network from Chapter 6 tells you the project takes N days. The **schedule** puts those days on a calendar, assigns resources, and gives you something to track progress against. The classic tool is the **Gantt chart**, developed by Henry Gantt between 1910 and 1915 and used at scale for the first time on the **Hoover Dam** (1931–1936).

> **Note**
>
> **Industry snapshot.** An automotive program's master schedule is a Gantt with thousands of rows rolled up to a one-page milestone view for the steering committee. A food-plant install schedule shows the trades stacked so the project manager can see where two crews need the same space on the same day. A defense schedule is baselined and every slip against it is reported.

3-Point Estimation for Project Schedule and Cost: https://www.youtube.com/watch?v=_YiAhMkBGbQ

8.3 Gantt charts

A Gantt chart is a **time-phased network**: it links each activity to a calendar. Linked to a schedule, it also **tracks progress against the plan**. Benefits: easy to read; shows the network *and* the schedule together; supports updating and control; useful for assigning resources; and easy to create.

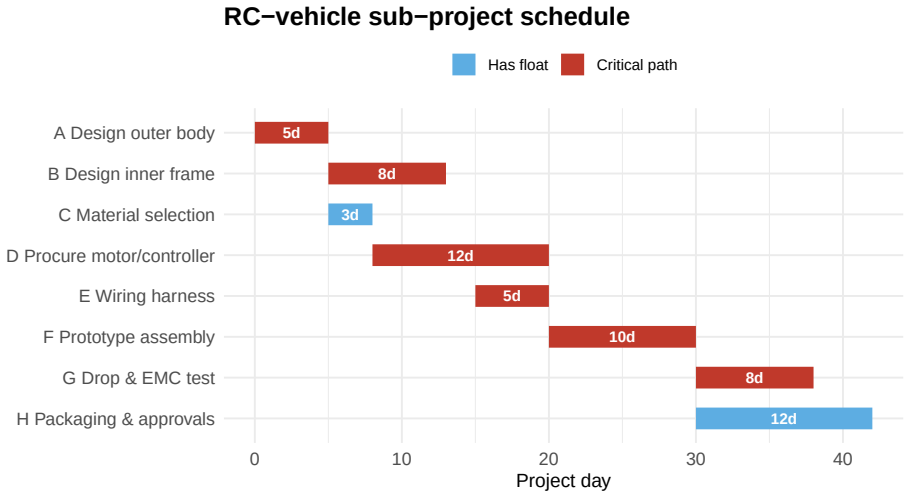


Figure 8.1: A Gantt chart for the RC-vehicle sub-project. Bars are activities; the dark segments show the critical path. Arrows (implied by stacking) show that assembly cannot start until design finishes.

8.4 Precedence relationships

Most dependencies are **finish-to-start**, but there are four:

Relationship	Meaning	Example
Finish-to-Start (FS)	B can't start until A finishes	Build the wall before you paint it
Finish-to-Finish (FF)	B can't finish until A finishes	Can't finish editing until writing is finished
Start-to-Start (SS)	B can't start until A starts (often simultaneous)	Start trimming edges when you start mowing
Start-to-Finish (SF)	A starting triggers B finishing (rare)	The new shift starting ends the old shift

A **lead** lets a successor start early (overlap); a **lag** forces a wait (e.g. "wait 2 days for the paint to cure" is FS + 2-day lag).

8.4.1 Try it yourself: which relationship is it?

Which precedence relationship does each statement describe?

Pick the relationship each statement describes.

Categories: Finish-to-Start (FS) / Finish-to-Finish (FF) / Start-to-Start (SS) / Start-to-Finish (SF)

1. The concrete must cure before the wall is painted. _____
2. Editing cannot finish until the writing has finished. _____
3. Trimming the edges starts when the mowing starts. _____
4. The old system is switched off only once the new system has started up. _____
5. Final testing cannot be completed until the last build is completed. _____
6. Cleaning of each room can begin once painting in that room has begun. _____
7. Assembly begins only after the long-lead part has arrived. _____
8. The outgoing shift stops only when the incoming shift has started. _____

Answers: 1 Finish-to-Start (FS); 2 Finish-to-Finish (FF); 3 Start-to-Start (SS); 4 Start-to-Finish (SF); 5 Finish-to-Finish (FF); 6 Start-to-Start (SS); 7 Finish-to-Start (FS); 8 Start-to-Finish (SF).

8.5 Probabilistic scheduling with PERT

A single duration hides uncertainty. Using three-point estimates (Chapter 6), for each activity:

$$t_e = \frac{o + 4m + p}{6}, \qquad \sigma = \frac{p - o}{6}, \qquad \sigma^2 = \left(\frac{p - o}{6}\right)^2$$

For the **project**, sum along the **critical path**:

$$\mu_{\text{project}} = \sum t_e, \qquad \sigma^2_{\text{project}} = \sum \sigma^2, \qquad \sigma_{\text{project}} = \sqrt{\sigma^2_{\text{project}}}$$

8.5.1 Worked Example 1: project duration and its spread

The critical path is A–B–C–D–E with these three-point estimates:

Live cell — change any activity's optimistic, most-likely, or pessimistic estimate.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: pert-project-08
cp <- data.frame(
  id = c("A", "B", "C", "D", "E"),
  o  = c(12, 7, 15, 9, 24),
  m  = c(15, 10, 19, 13, 26),
  p  = c(17, 13, 23, 16, 30)
)
cp$te  <- (cp$o + 4 * cp$m + cp$p) / 6
cp$var <- ((cp$p - cp$o) / 6)^2

mu    <- sum(cp$te)
sigma <- sqrt(sum(cp$var))

cat(sprintf("Project mean duration: %.1f days\n", mu))
cat(sprintf("Project variance  %.2f\n", sum(cp$var)))
cat(sprintf("Project std deviation  %.2f days\n", sigma))
cat(sprintf("Rough 95%% range  %.0f to %.0f days\n", mu - 2 * sigma, mu + 2 * sigma))
for (i in seq_len(nrow(cp)))
  cat(sprintf("%s  te %.2f days, variance %.2f\n", cp$id[i], cp$te[i], cp$var[i]))
```

8.5.2 Try it yourself: probability of hitting a target date

Standardize the target: $z = \dfrac{\text{target} - \mu}{\sigma}$, then read the normal cumulative probability.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: prob-on-time-08
mu    <- 83.0     # from the worked example
sigma <- 2.42
confidence_pct <- 90    # confidence you want in the target
target_day     <- 86    # the day you are promising

z    <- (target_day - mu) / sigma
prob <- pnorm(z)
conf_day <- mu + qnorm(confidence_pct / 100) * sigma

x <- seq(mu - 4 * sigma, mu + 4 * sigma, length.out = 200)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(x, dnorm(x, mu, sigma), type = "n", xlab = "Project finish (day)", ylab = "Density",
     main = "Probability of finishing on time")
shade <- x <= target_day
if (any(shade)) polygon(c(x[shade][1], x[shade], target_day), c(0, dnorm(x[shade], mu, sigma), 0)
                        col = "#aed6f1", border = NA)
lines(x, dnorm(x, mu, sigma), lwd = 4, col = "#2c3e50")
abline(v = target_day, lty = 2, lwd = 2, col = "#e74c3c")
abline(v = conf_day, lty = 3, lwd = 2, col = "#16a085")

if (100 * prob >= confidence_pct) {
  cat(sprintf("P(finish by day %g) = %.1f%%: meets the %g%% goal\n", target_day, 100 * prob, conf
} else {
  cat(sprintf("!!P(finish by day %g) = %.1f%%: below the %g%% goal!!\n", target_day, 100 * prob,
}
cat(sprintf("z-score  %+.2f\n", z))
cat(sprintf("Day for %g%% confidence  day %.1f (mu + %.2f x sigma)\n",
            confidence_pct, conf_day, qnorm(confidence_pct / 100)))
```

Why quote “day 86 at 90 % confidence” instead of “day 83”?

Day 83 is the *mean* — by definition you have only about a 50 % chance of meeting it. Committing to the mean is committing to a coin flip. Adding a buffer to reach 85–90 % confidence is how you turn a schedule into a promise you can keep, and it is the honest version of the “padding” problem from Chapter 9 — one visible buffer at the project level, not hidden padding in every task.

8.5.3 Try it yourself: how likely is the target date?

Take the three-point estimates for the activities on the critical path and work through to the probability of finishing by the target. Each team picks its own **data set** (and its own target day).

Probability of finishing on time

Variance is the square of the standard deviation. Two decimal places.

Activity	o	m	p	t_e	Variance
A	4	6	10		
B	8	10	16		
C	5	7	9		
D	6	9	18		
E	3	4	5		

- Project mean duration: _____ days *(to 2 decimal places)*
- Project standard deviation: _____ days *(to 2 decimal places)*
- z for a target of day 41: _____ *(to 2 decimal places)*
- Probability of finishing by day 41: _____ % *(to 1 decimal place)*
- Day to promise for 90 % confidence: _____ day *(to 1 decimal place)*

Show the worked solution

For data set 1: t_e = 6.33, 10.67, 7.00, 10.00, 4.00, which sum to a mean of **38.0 days**; the variances 1.00, 1.78, 0.44, 4.00 and 0.11 sum to 7.33, so sigma = **2.71 days**. For a target of day 41, z = (41 - 38) / 2.71 = 1.11 and the normal CDF gives about **87 %**. For 90 % confidence use 38.0 + 1.2816 x 2.71 = **day 41.5**. The mean (38) alone would be a 50 % date.

8.5.4 Try it yourself: simulate the schedule

The formula above assumes the project total is normally distributed. Check it by brute force: each run draws every activity's duration from a PERT (beta-shaped) distribution and adds them up. Press **Run again** for a new set of 2 000 runs; in class, each team enters its own run number.

Show the live R code behind this simulation

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: schedule-sim-08
seed       <- 1
target_day <- 80

# The chapter's critical path A to E (days): optimistic, most likely, pessimistic.
o <- c(12, 7, 15, 9, 24)
m <- c(15, 10, 19, 13, 26)
p <- c(17, 13, 23, 16, 30)
runs <- 2000
```

```
set.seed(seed)
# PERT-beta: mean (o + 4m + p) / 6, spread set by the same three numbers.
a <- 1 + 4 * (m - o) / (p - o)
b <- 1 + 4 * (p - m) / (p - o)
total <- numeric(runs)
for (i in seq_along(o)) total <- total + o[i] + (p[i] - o[i]) * rbeta(runs, a[i], b[i])

pct_on_time <- 100 * mean(total <= target_day)
mu    <- sum((o + 4 * m + p) / 6)
sigma <- sqrt(sum(((p - o) / 6)^2))
formula_pct <- 100 * pnorm((target_day - mu) / sigma)

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
hist(total, breaks = 30, col = "#aed6f1", border = "white",
     xlab = "Project finish (day)", ylab = "Runs", main = "Simulated project durations")
abline(v = target_day, lty = 2, lwd = 3, col = "#e74c3c")

if (pct_on_time >= 90) {
  cat(sprintf("%.1f%% of runs finish by day %g: meets a 90%% promise\n", pct_on_time, target_day)
} else {
  cat(sprintf("%.1f%% of runs finish by day %g: !!below a 90%% promise!!\n", pct_on_time, target_
}
cat(sprintf("Normal-curve formula  %.1f%%\n", formula_pct))
cat(sprintf("Mean simulated finish  day %.1f\n", mean(total)))
cat(sprintf("Middle 90%% of runs  day %.1f to %.1f\n", quantile(total, 0.05), quantile(total, 0.9
```

8.6 Crashing the schedule

When you crash: the initial schedule was too optimistic; the market wants the product sooner; the project has slipped badly; or the contract rewards finishing early / punishes finishing late.

How you accelerate an activity: improve the productivity of existing resources (focus the team, review best practice); change the working method (swap drawing review for CAD model review); or add resources (overtime, extra shifts, expedited freight).

> ⚠ Warning
>
> **Caution on overtime.** Research on engineering work found optimal marginal output at about **4 hours of overtime per week**; by **10 hours** of overtime the *marginal* extra output had fallen to **zero**. More hours is not more work.

Every activity has a **crash cost slope** — the cost to buy back one time unit:

$$\text{slope} = \frac{\text{crash cost} - \text{normal cost}}{\text{normal time} - \text{crash time}}$$

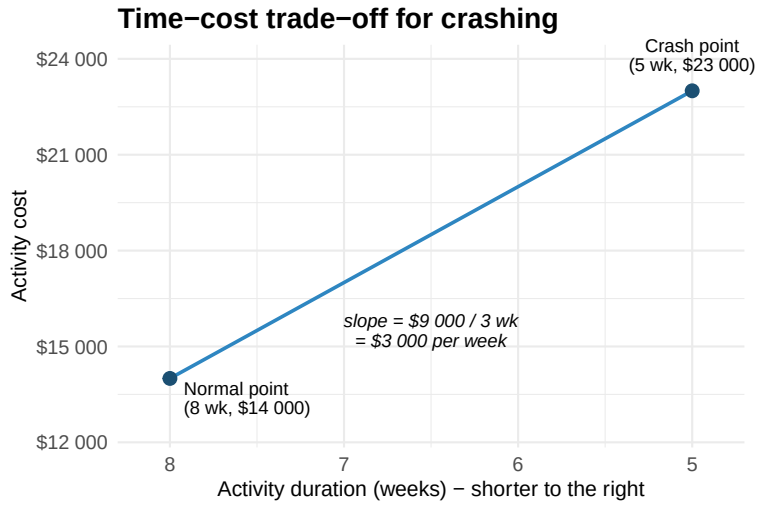


Figure 8.2: Time-cost trade-off for one activity. Between the normal and crash points, buying time gets steadily more expensive per day; past the crash point you cannot buy any more.

8.6.1 Try it yourself: least-cost crashing

To shorten the project, crash **critical-path** activities in **cheapest-slope- first** order, one time unit at a time, until you hit the target or run out of crashable critical tasks.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: crash-sequence-08
target_reduction <- 4          # days to remove from the project

# Critical-path activities: normal/crash time and cost.
act <- data.frame(
  id         = c("A", "F", "H"),
  normal_t   = c(9, 6, 8),
  crash_t    = c(7, 5, 4),
  normal_c   = c(5500, 4000, 3500),
```

```
  crash_c    = c(7550, 9000, 7550)
)
act$slope     <- (act$crash_c - act$normal_c) / (act$normal_t - act$crash_t)
act$max_crash <- act$normal_t - act$crash_t

order_by_cost <- act[order(act$slope), ]
order_by_cost$days_used <- 0

remaining <- target_reduction
total_cost <- 0
steps <- character(0)
for (i in seq_len(nrow(order_by_cost))) {
  take <- min(remaining, order_by_cost$max_crash[i])
  if (take <= 0) break
  cost <- take * order_by_cost$slope[i]
  total_cost <- total_cost + cost
  remaining <- remaining - take
  order_by_cost$days_used[i] <- take
  steps <- c(steps, sprintf("Crash %s by %g day(s) at $%s/day  $%s",
                            order_by_cost$id[i], take, format(round(order_by_cost$slope[i]), big.
                            format(round(cost), big.mark = " ")))
}

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(order_by_cost$slope, names.arg = order_by_cost$id, col = ifelse(order_by_cost$days_
              ylim = c(0, max(order_by_cost$slope) * 1.2),
              xlab = "Activity (cheapest first)", ylab = "Crash cost ($ per day)",
              main = "Crash cost per day (blue = crashed)")
text(bp, order_by_cost$slope, labels = ifelse(order_by_cost$days_used > 0, paste(order_by_cost$da
     pos = 3, cex = 1.3)

if (remaining > 0) {
  cat(sprintf("!!Only %g of %g days can be crashed: $%s!!\n", target_reduction - remaining,
              target_reduction, format(round(total_cost), big.mark = " ")))
} else {
  cat(sprintf("Cheapest way to cut %g days: $%s\n", target_reduction, format(round(total_cost), b
}
for (s in steps) cat(s, "\n", sep = "")
```

8.6.2 Try it yourself: crash slopes and the cheapest cut

All four activities are on the critical path. Work out each slope and each activity's maximum crash, then the least cost of the cut asked for. Each team picks its own **data set**.

Crash cost slopes

Crash the cheapest slope first, up to each activity's maximum.

Activity	Normal (wk)	Crash (wk)	Normal cost ()|$Crashcost()$	Slope ($/wk)	Max crash (wk)
A	10	7	20000	29000	
B	8	6	15000	19000	
C	6	5	9000	12500	
D	9	6	18000	23400	

- Least cost to cut 5 weeks: $_____ *(to 0 decimal places)*

Show the worked solution

Slope = (crash cost - normal cost) / (normal time - crash time). For data set 1 the slopes are A $3 000, B $2 000, C $3 500 and D $1 800 per week. Cheapest first: D for 3 weeks ($5 400), then B for 2 weeks ($4 000), so the 5-week cut costs **$9 400**. A and C are never touched. Re-check the network afterwards: the critical path can shift.

8.7 Fast-tracking versus crashing

	Crashing	**Fast-tracking**
Method	Add resources (overtime, staff, expediting)	Overlap tasks that were sequential
Cost	Always adds cost	Little or no added cost
Risk	Diminishing returns; overtime burnout	Rework if the predecessor changes after the successor started

Both only help on the **critical path**, and after either one you must **re-check the network** — the critical path can move.

8.8 Key Takeaways

- A **Gantt chart** puts the network on a calendar and lets you track progress; it shows activities, dependencies, and resources at once.
- Four precedence types: **FS** (default), **FF**, **SS**, **SF** — with **leads** (overlap) and **lags** (forced wait).

- PERT project: $\mu = \sum t_e$ and $\sigma = \sqrt{\sum \sigma^2}$ along the **critical path**. The mean is a ~50 % date.
- Probability of on-time: $z = (\text{target} - \mu)/\sigma$, then the normal CDF. Commit to an ~85–90 % date, not the mean.
- **Crash cost slope** = (crash cost − normal cost) / (normal time − crash time). Crash **critical tasks, cheapest slope first**.
- **Crashing** buys time with money; **fast-tracking** buys it with rework risk. Re-check the critical path after either.

8.9 Case Study: Meridian R/C — Week 8

CASE STUDY FILE — Week 8: Can the Schedule Be Pulled In?

This week you pull up the Gantt chart derived from the fixed CPM network for the Trailblazer prototype — 10 activities, 67 working days, critical path A–B–E–F–H–I–J — and the picture is uncomfortably tight. Every activity on that spine has zero float: concept CAD, detailed design release, motor & controls procurement (E, 25 days), prototype assembly (F), toy-safety certification (H, 12 days), packaging & client sample ship, and your sign-off report to Renata Ford. The only slack anywhere in the network sits in body tooling (C, 5 days float), frame tooling (D, 10 days float), and the drop/durability test (G, 7 days float) — none of which help you finish sooner.

Daniel Cho, sitting in on the Friday steering call, asks the question you knew was coming: with Kestrel's 100,000-unit holiday-season order riding on this prototype, can the 67-day timeline be pulled in? You walk the team through the two realistic levers, both of which only work because E and H — and nothing else in the network — are actually critical.

Crashing E (motor & controls procurement) buys up to 5 days, at **$1,200 per day** for air freight and expedited customs — a maximum of $6,000 to shave the full 5 days off the 25-day activity.

Crashing H (toy-safety certification) buys up to 4 days, at **$900 per day** for priority lab review — a maximum of $3,600 to shave the full 4 days off the 12-day activity.

There is also a **fast-tracking** option on H: instead of waiting for F (assembly) to finish before booking the certification lab, Sofia Reyes could pre-book the lab slot while assembly is still underway. That saves roughly 3 days at little direct cost — but if a late change during assembly affects the sample, the pre-booked test may have to be repeated, eating into the time it was meant to save.

Laying the numbers side by side for Daniel, one thing jumps out: dollar for dollar, **H is actually the cheaper activity to crash** ($900/day versus E's $1,200/day), even though E offers more total days of compression (5 versus 4). Applying the chapter's own cheapest-slope-first rule, H is the

activity to crash first if only a few days are needed; E is the fallback if more days are required than H alone can supply. Whichever lever gets pulled, the team agrees to re-check the critical path afterward — crashing or fast-tracking either activity could shift where the true bottleneck sits.

8.10 Practice Problems

Problem 1: Least-cost crash to a deadline

Project critical path A–F–H, currently 23 weeks; the contract needs 20. Crash data: A \$513/wk (max 4 wk), F \$1 500/wk (max 1 wk), H \$950/wk (max 2 wk). Find the least-cost way to remove 3 weeks.

Solution

```
crash <- data.frame(
  id = c("A", "F", "H"),
  slope = c(513, 1500, 950),
  max = c(4, 1, 2)
)
need <- 3
crash <- crash[order(crash$slope), ]
left <- need; cost <- 0
for (i in seq_len(nrow(crash))) {
  take <- min(left, crash$max[i]); if (take <= 0) break
  cost <- cost + take * crash$slope[i]; left <- left - take
  cat(sprintf("Crash %s by %d wk @ $%d = $%s\n", crash$id[i], take, crash$slope[i],
              format(take * crash$slope[i], big.mark = " ")))
}
cat(sprintf("Total: $%s for %d weeks\n", format(cost, big.mark = " "), need - left))
```

```
Crash A by 3 wk @ $513 = $1 539
Total: $1 539 for 3 weeks
```

Crash A by 3 weeks for \$1 539 — far cheaper than touching F or H.

Problem 2: Confidence in a date

A project's critical path has $\mu = 60$ days and $\sigma = 4$ days. The customer wants delivery in 64 days.

(a) What is the probability of meeting day 64? (b) What date should you commit to for 95 % confidence?

Solution

```
mu <- 60; sigma <- 4
cat(sprintf("P(finish by day 64) = %.1f%%\n", 100 * pnorm((64 - mu) / sigma)))
cat(sprintf("95%% confident date  = day %.1f\n", mu + qnorm(0.95) * sigma))
```

```
P(finish by day 64) = 84.1%
95% confident date  = day 66.6
```

(a) ~84 %. (b) Day 66.6 — commit to day 67.

Problem 3 (discussion): crash or fast-track the RC-vehicle launch?

The retailer moves the launch three weeks earlier. Design is on the critical path and is followed by prototype assembly (also critical). You could add a second designer and pay overtime (crash), or start building the frame jig from the released frame drawings while the body design is still being finalized (fast-track). Which, and why?

Discussion points

Fast-tracking the jig against the *frame* drawings is low-cost and reasonably safe *if* the frame design is stable — but if the body design forces a late frame change, the jig is rework. Crashing design with a second modeller adds cost and has co-ordination overhead (two people on one model), but no rework risk. A common answer: fast-track the clearly stable parts, crash the rest, and hold a contingency for the rework risk you accepted.

8.11 Further Resources

PERT / Three-Point Estimation Technique: https://www.youtube.com/watch?v=6B7Qtc3rrQ4

Forward and Backward Pass -> Project Duration and Critical Path: https://www.youtube.com/watch?v=4oDLMs11Exs

Chapter 9

Cost Estimation and Budgeting

9.1 Learning Objectives

By the end of this chapter you will be able to:

1. **List** the common sources of project cost and **classify** any cost against the four cost dichotomies (direct/indirect, fixed/variable, normal/expedited, recurring/non-recurring).
2. **Calculate** direct labour cost from a rate and hours, and **apply** an indirect-cost percentage.
3. **Select** an estimating method (ballpark, parametric, feasibility, definitive) appropriate to the project stage, and **state** its expected accuracy.
4. **Explain** how planning horizon, duration, people, structure, padding, and organizational culture affect estimate quality.
5. **Compare** top-down and bottom-up budgeting and **build** a bottom-up budget from a work breakdown structure using **activity-based costing**.
6. **Construct** a time-phased budget and **plot** the cumulative cost baseline (the S-curve).
7. **Size** a contingency reserve and **distinguish** it from a management reserve.

Before we begin: you have been asked for "a number" before you have any details

A stakeholder corners you: *"Ballpark it — what will this project cost?"* You have a one-page concept and nothing else. What do you say, and how do you say it so the number does not follow you around for the next six months as a commitment? Hold that thought.

9.2 Introduction: the number that decides whether the project happens

Back to the running project. Your company must present a budget to **Senior Management before the Scope and Charter are approved**. The project brief hands you rough ranges to work from:

Table 9.1: Selected budget ranges from the RC-vehicle project brief

Item	Range
Project Manager (salary)	\$60 000 – \$80 000
Team members (salary, each)	\$30 000 – \$45 000
Sundry / overhead (per hour)	\$8.00 – \$15.00
Motor / controller	\$100 000 – \$175 000
Carbon fibre	\$120 000 – \$200 000
Aluminium	\$50 000 – \$115 000
Tooling	\$100 000 – \$180 000
Testing	\$25 000 – \$100 000
Government approval (CSA, CE, MTO)	\$2 000 – \$10 000
Packaging / shipping	\$5 000 – \$25 000

Cost estimation is the first step in building a budget, and the budget determines whether the project is feasible at all. Two principles run through everything below:

1. The more clearly you define each cost element up front, the fewer budgeting errors you make.
2. The more accurate your initial estimate, the more your budget reflects reality — and the less likely a painful overrun later.

> **Note**
>
> **Industry snapshot.** A defense contractor bidding a **firm-fixed-price** contract must submit a *definitive* estimate (±5 %): if the real cost comes in higher, the contractor eats the difference. A food company scoping a new retort line at the concept stage lives with a *ballpark* (±30 %) and revises as quotes arrive. Same tools, very different tolerance for error — driven by where in the project you are standing.

The ABCs of Project Cost Management (Adriana Girdler): https://www.youtube.com/watch?v=iEndSB5IgQM

9.3 Where project cost comes from

Cost management starts in the **planning phase**, when you identify the project's human and material resources and lay out a **time-phased budget** that reflects when each will be used. The common sources of cost:

- **Labour** — the project team and anyone charging time to the project.
- **Materials** — everything consumed to build the deliverable.
- **Subcontractors** — external vendors, *and* internal support departments (test lab, drafting) that cross-charge.
- **Equipment and facilities** — purchased, leased, or allocated.
- **Travel.**
- **Shipping** — inbound and outbound, normal and expedited.

9.4 The eight cost types

Every cost gets sorted into **one or more** of eight categories, arranged as four either/or pairs. A single cost usually carries **two or three** labels at once.

Table 9.2: The four cost dichotomies (eight types).

Dichotomy	Type	Meaning
Direct vs. Indirect	**Direct**	Clearly generated by the project (project labour, project materia
	Indirect	Overhead: rent, property tax, benefits, utilities, depreciation, ad
Fixed vs. Variable	**Fixed**	Does not change with usage (a 6-month equipment lease at a fla
	Variable	Proportional to usage (machine-hour charges, fuel, mileage).
Normal vs. Expedited	**Normal**	Incurred during normal operation and working hours; includes o
	Expedited	Unplanned cost to go faster: rush freight, extra hires, premium t
Recurring vs. Non-recurring	**Recurring**	Happens again through the project life: utilities, logistics, ongoi
	Non-recurring	One-time, usually at the start or end: training, prototype toolin

9.4.1 Direct labour cost

The core calculation is simple:

$$\text{Total Direct Labour Cost} = \text{Direct Labour Rate} \times \text{Hours Spent}$$

Indirect cost is harder to pin to a task, so many organizations apply it as a **percentage of direct cost**:

$$\text{Indirect Cost} = \text{Indirect Rate} \times \text{Direct Cost}, \qquad \text{Indirect Rate} \approx 20\%\text{–}50\%$$

9.4.2 Worked Example 1: fully-burdened cost of a work package

The "Design body & frame" work package needs **1 mechanical designer for 220 h** at \$46/h and **1 junior engineer for 120 h** at \$32/h. Direct materials for prototypes are \$4 500. The company applies indirect cost at **35 %** of direct cost.

Live cell — change the hours, rates, materials, or the indirect rate.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: cost-wp-example-09
designer_hours <- 220
designer_rate  <- 46
junior_hours   <- 120
junior_rate    <- 32
direct_materials <- 4500
indirect_rate <- 0.35

direct_labour <- designer_hours * designer_rate + junior_hours * junior_rate
direct_cost   <- direct_labour + direct_materials
indirect_cost <- indirect_rate * direct_cost
total_cost    <- direct_cost + indirect_cost

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

parts <- c("Direct labour" = direct_labour, "Direct materials" = direct_materials, "Ind
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(parts / 1000, col = c("#2980b9", "#16a085", "#8e44ad"), ylim = c(0, max(p
              main = "What makes up the work-package cost")
text(bp, parts / 1000, labels = paste0("$", round(parts / 1000, 1), "k"), pos = 3, cex

cat(sprintf("Fully-burdened work-package cost: %s\n", money(total_cost)))
cat(sprintf("Direct labour  %s\n", money(direct_labour)))
cat(sprintf("Direct materials  %s\n", money(direct_materials)))
cat(sprintf("Direct cost  %s\n", money(direct_cost)))
cat(sprintf("Indirect (%g%%)  %s\n", 100 * indirect_rate, money(indirect_cost)))
```

9.4.3 Try it yourself: cost and variance by hand

Cost a test campaign from its drivers, apply the indirect rate, and compare with what was actually spent. Each team picks its own **data set**.

Fully burdened cost and variance

Planned cost = direct cost x (1 + indirect rate).

- Technicians x weeks: 3 x 4
- Hours per week each: 37.5
- Labour rate: $36 / h
- Consumables: $5 400
- Indirect rate: 30 % of direct cost
- Actual cost: $24 600
- Direct labour: $_____ *(to 0 decimal places)*
- Direct cost (labour + consumables): $_____ *(to 0 decimal places)*
- Planned, fully burdened: $_____ *(to 0 decimal places)*
- Variance (planned - actual; negative = over budget): $_____ *(to 0 decimal places)*
- Variance as % of planned: _____ % *(to 1 decimal place)*

Show the worked solution

Data set 1: labour = 3 x 4 x 37.5 x $36 = $16 200; plus $5 400 of consumables gives a direct cost of $21 600; at 30 % indirect the planned cost is $28 080. Against an actual of $24 600 the variance is +$3 480, or 12.4 %: under budget (favourable).

Practice: classify the cost (from the MGMT-3105 budgeting case study)

For each item, give **up to three** labels from: A Direct, B Indirect, C Fixed, D Variable, E Recurring, F Non-recurring, G Normal, H Expedited.

#	Cost item
1	Technician's hourly wages for project assembly
2	Office rent for company headquarters
3	One-time purchase of specialized testing software
4	Electricity used by production equipment
6	Hiring extra workers to meet a tight deadline
9	Rush shipment of components to meet schedule
12	Cost of the project manager's salary
17	Routine monthly maintenance of equipment
23	Equipment lease for six months (flat fee)
40	One-time cost for prototype tooling

Answers

#	Answer
1	A Direct, D Variable, G Normal
2	B Indirect, C Fixed, E Recurring
3	A Direct, F Non-recurring
4	D Variable, E Recurring, B Indirect
6	H Expedited, A Direct
9	H Expedited, A Direct
12	C Fixed, E Recurring, A Direct
17	*(trick)* E Recurring, B Indirect
23	B Indirect, C Fixed, E Recurring
40	A Direct, F Non-recurring, C Fixed

The pattern: **anything charged straight to the project is Direct; overhead is Indirect; anything you pay to go faster is Expedited; anything paid once is Non-recurring.** Most items are genuinely two or three of these at once.

9.4.4 Try it yourself: label each cost three ways

Every cost carries a label from each of the three pairs below. Pick one option in each column.

Direct or indirect, fixed or variable, normal or expedited

Direct = charged straight to the project; expedited = paid to go faster than planned.

Cost	Direct or indirect (Direct / Indirect)	Fixed or variable (Fixed / Variable)	Normal or expedited (Normal / Expedited)
Assembler wages on the pilot build			
Plant insurance premium			
Overnight air freight for a late circuit board			
Flat-fee lease of a CMM taken for this project only			
Electricity for the whole plant's compressed air			

Cost	Direct or indirect (Direct / Indirect)	Fixed or variable (Fixed / Variable)	Normal or expedited (Normal / Expedited)
Weekend overtime to recover an unplanned slip			
Prototype material bought for this project			
Administrative staff salaries			

9.4.5 The dichotomies on the running project

- **Fixed vs. variable.** The 6-month lease on the CMM is **fixed** — the fee is the same whether you measure 50 parts or 5 000. The machine-hour charge on the rented moulding press is **variable**. Leasing an expensive machine is a real PM judgement call: will the usage justify the flat fee?
- **Normal vs. expedited.** Air-freighting circuit boards from the supplier to recover a slipped date is **expedited** and **direct**. The overtime you *planned* into the schedule from day one is still **normal**.
- **Recurring vs. non-recurring.** Prototype tooling and the initial market analysis are **non-recurring**. Utilities, logistics, and the team's salaries are **recurring** — and it matters which is which the moment you build a time-phased budget.

9.5 Estimating methods and their accuracy

The method you can use depends on how much you know, which depends on how far into the project you are.

Table 9.3: Estimating methods by project stage and typical accuracy.

Method	When	Ac
Ballpark / order-of-magnitude	Very early; scarce time/resources; deciding whether to even bid.	+/-
Comparative / parametric	A genuinely similar past project exists, with reliable archived actuals.	+/-
Feasibility	Scope is clear and first supplier quotes are back (common in construction).	+/-
Definitive	Most design is complete; major POs can be written; detail is known.	+/

This is the **cone of uncertainty**: the possible range around an estimate is widest at the concept stage and narrows as the project is defined.

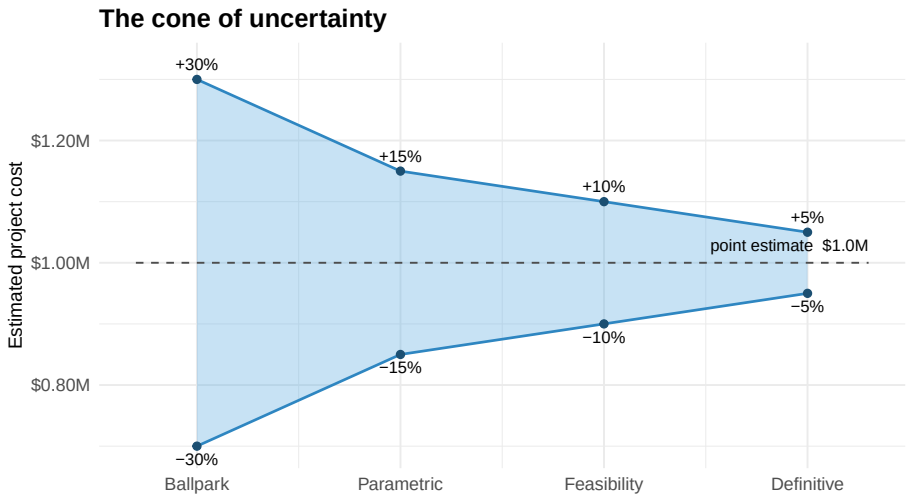


Figure 9.1: The cone of uncertainty. Early estimates can be off by +/- 30 %; the range narrows to +/- 5 % only once most design decisions are locked. The same $1.0M point estimate means very different things at each stage.

9.5.1 Try it yourself: parametric estimate and the accuracy band

A **parametric** estimate uses a cost-estimating relationship (CER) from history — for example, "past RC-vehicle projects cost about $14.50 per assembled part plus $85 000 of fixed project cost." The live cell builds the estimate and shows the band each method would put around it.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: parametric-estimate-09
units             <- 120000
cost_per_unit_CER <- 14.50      # $/unit, from archived projects
fixed_project_cost<- 85000      # $, setup / PM / approvals

point_estimate <- units * cost_per_unit_CER + fixed_project_cost

bands <- data.frame(
  method   = c("Ballpark", "Parametric", "Feasibility", "Definitive"),
  accuracy = c(0.30,        0.15,         0.10,          0.05)
)
bands$low  <- round(point_estimate * (1 - bands$accuracy))
```

```
bands$high <- round(point_estimate * (1 + bands$accuracy))

ticks <- pretty(c(bands$low, bands$high), n = 3)
par(mar = c(5, 13, 3, 3), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(NA, xlim = range(bands$low, bands$high), ylim = c(0.5, 4.5), yaxt = "n", xaxt = "n",
     xlab = "Estimate ($)", ylab = "", main = "Estimate accuracy band by method")
segments(bands$low, 4:1, bands$high, 4:1, lwd = 10, lend = 1, col = "#2980b9")
axis(1, at = ticks, labels = format(ticks, big.mark = " ", scientific = FALSE, trim = TRUE))
axis(2, at = 4:1, labels = bands$method, las = 1, tick = FALSE)
abline(v = point_estimate, lty = 2, lwd = 2, col = "#e74c3c")

cat(sprintf("Parametric point estimate: $%s\n", format(round(point_estimate), big.mark = " ")))
for (i in seq_len(nrow(bands))) {
  cat(sprintf("%s (+/- %.0f%%)  $%s to $%s\n", bands$method[i], bands$accuracy[i] * 100,
              format(bands$low[i], big.mark = " "), format(bands$high[i], big.mark = " ")))
}
```

Why quote a range at all — why not just give "the number"?

Because a single number reads as a **commitment**. If you say "$1.8M" at the concept stage and the definitive estimate later lands at $2.0M, you have "overrun by $200k" — even though $2.0M was always inside the ±30 % ballpark band. Quoting "$1.3M–$2.3M, ballpark" sets the expectation honestly and buys you room to refine. A clear distinction between an **estimate** and a **commitment** is one of the most important things a PM can establish early.

9.5.2 Try it yourself: which estimating method?

Which estimating method, and so which accuracy band, fits each situation?

The method follows how much you know, which depends on how far into the project you are.

Categories: Ballpark (+/- 30 %) / Parametric (+/- 15 %) / Feasibility (+/- 10 %) / Definitive (+/- 5 %)

1. A hallway question about a new product line; all you have is a one-page concept. _____
2. You have costed three near-identical launches and kept clean actuals. _____
3. Scope is clear and the first supplier quotes are back. _____
4. Major purchase orders are drafted and most of the design is released, for a fixed-price bid. _____
5. You are deciding whether to bid at all, with little time and few people. _____
6. A reliable historical cost-per-unit relationship exists for this product family. _____

7. A construction project with defined scope and preliminary subcontractor quotes. _____
8. Contract signature is next week and the detailed drawings are complete. _____

Answers: 1 Ballpark (+/- 30 %); 2 Parametric (+/- 15 %); 3 Feasibility (+/- 10 %); 4 Definitive (+/- 5 %); 5 Ballpark (+/- 30 %); 6 Parametric (+/- 15 %); 7 Feasibility (+/- 10 %); 8 Definitive (+/- 5 %).

9.6 What makes estimates good or bad

Factor	Effect on estimate quality
Planning horizon	Estimates of near-term work are close to 100 % accurate; accuracy falls the further out the task sits.
Project duration	Long projects and new technology add uncertainty, especially if the scope statement is weak.
People	The closer the estimator's experience matches the work, the better. Teams that have worked together before form faster. Staff turnover hurts.
Structure	Dedicated teams deliver faster but tie up people full-time. Matrix organizations are more cost-efficient but slower.
Padding	Estimators add a safety margin. If *everyone* pads a little, the project total is badly overstated and stops reflecting reality.
Organizational culture	Some cultures reward padding; others prize accuracy. The consequences an estimator faces for a miss shape the number they give.
Other	Equipment downtime, sickness, vacation, holidays, legal limits, and project priority all leak into the estimate.

9.6.1 The cost of padding

Padding feels prudent per task. Watch what it does across a whole project.

The fix is not "ban all safety margin" — it is to **remove padding from the**

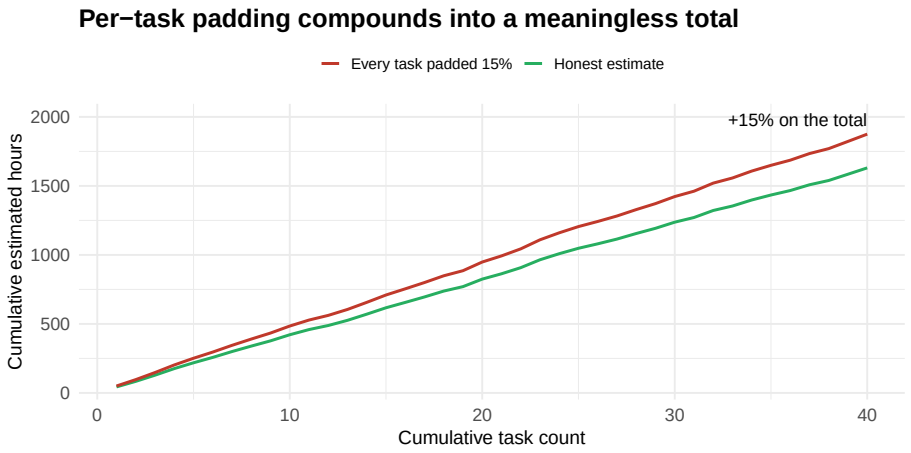


Figure 9.2: A 15 % 'just to be safe' pad added to every one of 40 tasks does not add 15 % to the project — it adds 15 % to the mean, but the padded total sits far above any realistic outcome because the pads do not all get used.

work packages and hold a single, visible **contingency reserve** at the project level (covered below).

9.6.2 Guidelines for estimating (cost and time)

- **Responsibility:** the person who will do the task makes the estimate (except for highly specialised work). It uses their experience and builds their buy-in.
- **Use several estimators:** independent estimates from knowledgeable people, reconciled, remove the extremes.
- **Assume normal conditions:** normal hours, normal resource levels — and write the assumptions down.
- **Define the time units** and the estimating base.
- **Estimate each task independently;** be careful with chains of dependent tasks, and keep one estimator per task.
- **Keep contingencies out of the work packages.** Contingency is a separate, managed fund.

9.7 Budgeting: top-down and bottom-up

The budget and the schedule are built **in tandem** — the budget decides whether the schedule's milestones are reachable.

Aspect	Top-down
How it starts	Senior management sets the overall budget and major work-package figures from ex
Advantages	Management aggregate estimates are often quite accurate; imposes budget disciplin
Disadvantages	Creates friction: lower levels must fit management's fixed total. Turns into a zero-s

9.8 Activity-based costing (ABC)

ABC treats a project as a series of **activities**, each of which **consumes resources**. Cost the activities and you have costed the project. Four steps:

1. Identify the activities that consume resources and assign costs to them.
2. Identify the **cost drivers** for each activity (personnel, materials, machine time…).
3. Compute a **cost rate per driver unit** (e.g. labour $/hour).
4. **Assign cost** = rate × units the project uses. *(Example: an engineer at $60/h working 80 h = $4 800.)*

9.8.1 A preliminary ABC budget

Splitting each activity into its **direct** and **overhead** components:

Table 9.5: Preliminary ABC budget: direct vs. overhead by activity.

Activity	Direct	Overhead	Total
Survey	3 500	500	4 000
Design	7 000	1 000	8 000
Clear site	3 500	500	4 000
Foundation	6 750	750	7 500
Framing	8 000	2 000	10 000
Plumb & wire	3 750	1 250	5 000
TOTAL	**32 500**	**6 000**	**38 500**

9.8.2 Try it yourself: cost an activity, then a variance

The live cell costs a single activity from its drivers, then compares planned vs. actual to get the **variance**. Change the rates, hours, or actuals.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: abc-activity-variance-09
# --- Step 3 & 4: cost the "Assemble & test prototype" activity from drivers ---
labour_rate   <- 38
labour_hours  <- 160
machine_rate  <- 22
machine_hours <- 45
materials     <- 2600
overhead_rate <- 0.30            # 30% of direct

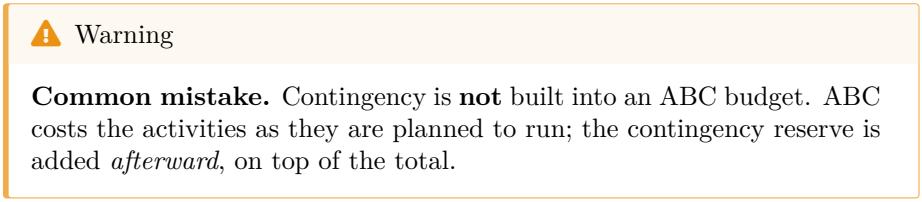
direct   <- labour_rate * labour_hours + machine_rate * machine_hours + materials
overhead <- overhead_rate * direct
planned  <- direct + overhead

# --- Variance: what actually happened ---
actual <- 11800
variance <- planned - actual          # positive = under budget (favourable)
pct <- 100 * variance / planned

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", sc

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(c(Planned = planned, Actual = actual) / 1000, col = c("#2980b9", if (variance >= 0)
              ylim = c(0, max(planned, actual) / 1000 * 1.2), ylab = "$ thousand",
              main = "Planned vs. actual activity cost")
text(bp, c(planned, actual) / 1000, labels = paste0("$", round(c(planned, actual) / 1000, 1), "k'

if (variance >= 0) {
  cat(sprintf("Variance %s (%+.1f%%): UNDER budget (favourable)\n", money(variance), pct))
} else {
  cat(sprintf("Variance %s (%+.1f%%): !!OVER budget (unfavourable)!!\n", money(variance), pct))
}
cat(sprintf("Direct  %s\n", money(direct)))
cat(sprintf("Overhead  %s\n", money(overhead)))
cat(sprintf("Planned activity cost  %s\n", money(planned)))
cat(sprintf("Actual  %s\n", money(actual)))
```



> **Warning**
>
> **Common mistake.** Contingency is **not** built into an ABC budget. ABC costs the activities as they are planned to run; the contingency reserve is added *afterward*, on top of the total.

9.9 The time-phased budget and the S-curve

A **time-phased budget** spreads each activity's cost across the calendar periods when it will actually be spent. Summing down the columns gives **monthly planned spend**; the running total is the **cumulative cost baseline** — the S-curve you will track the project against in Chapter 12.

```
Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0
i Please use the `linewidth` argument instead.
```

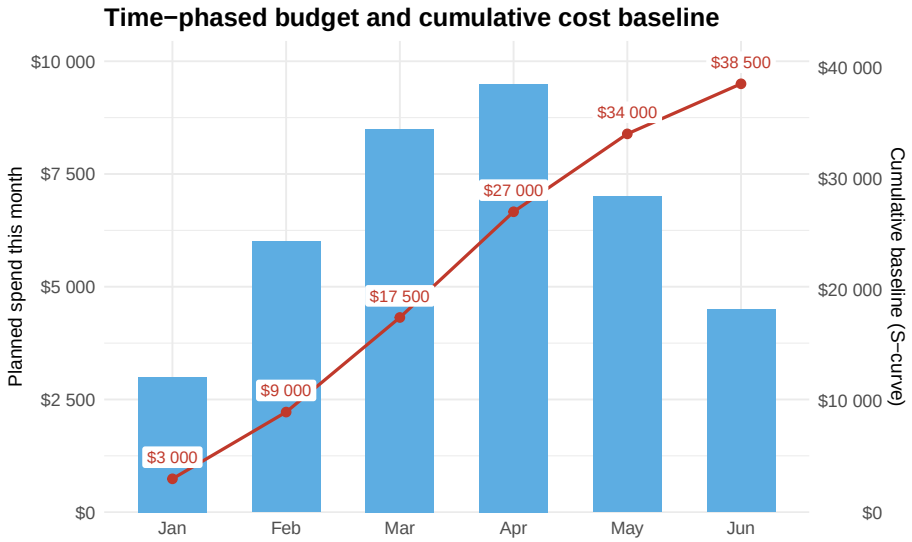


Figure 9.3: Time-phased budget for the six-activity project of the ABC table (total $38,500). Bars are planned spend per month; the line is the cumulative baseline (the S-curve). Spend is slow at first, steepest mid-project, and tapers at the end.

Matching the cost baseline to the schedule baseline is what lets you set **milestones for both schedule and budget** and later tell whether a project that has spent 60 % of its money is 60 %, 40 %, or 80 % done.

9.9.1 Try it yourself: build the S-curve

Time-phase a small budget: add each month's spend across the activities, then the running total is the cumulative baseline. Each team picks its own **data set**. Amounts in dollars.

Time-phased budget and cumulative baseline

Warehouse fit-out: planned spend by activity and month.

Month	Demolition	Wiring	Racking	Painting	Signage	Month total	Cumulative
Jan	4000	0	0	0	0		
Feb	2000	3000	0	0	0		
Mar	0	5000	6000	0	0		
Apr	0	2000	8000	2000	0		
May	0	0	3000	3000	1500		
Jun	0	0	0	1000	2500		

- Peak monthly spend: $_____ *(to 0 decimal places)*
- Cumulative spend at the end of March: $_____ *(to 0 decimal places)*

Show the worked solution

Data set 1: monthly totals are $4 000, $5 000, $11 000, $12 000, $7 500 and $3 500; the cumulative baseline is $4 000, $9 000, $20 000, $32 000, $39 500 and $43 000. Peak spend is $12 000 in April and the baseline stands at $20 000 at the end of March. Plotted, the cumulative column is the S-curve you will track earned value against in Chapter 12.

9.10 Contingency reserves

A **budget contingency** is extra money set aside to cover the uncertainty that every estimate carries — because even the best budget is only an estimate and something unforeseen always happens. Key points:

- Added **after all costs are identified**, as a cushion on top of the calculated total (commonly **10–15 %**; the MGMT-3105 Costs Chart uses **15 %**).
- **Not** included in an ABC budget — it sits on top.
- The **PM controls a project contingency fund**; separately, **top management holds a management reserve** for genuinely unknown-unknown events.

Why stakeholders push back: the percentage can look arbitrary, or feel like being asked to pay for the vendor's poor cost control. Decide early — and write into the scope statement — whether contingency applies across **all** tasks or is held for the tasks **critical** to project success.

Table 9.6: Contingency reserve vs. management reserv

Feature	Contingency reserve
Covers	Identified risks / "known-unknowns" (small scope adjustments, iteration, Murphy'
Owned by	Project manager.

Part of the cost baseline?	Yes — inside the baseline.
Typical size	10–15 % of estimated cost (project-dependent).

9.10.1 Try it yourself: build the RC-vehicle project budget

The live cell builds a rough **bottom-up** budget from the project brief's ranges (using a pick inside each range), rolls up labour and materials, and adds a contingency. Change the picks or the contingency rate and watch the total — then check it against what Senior Management is likely to approve.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: rc-project-budget-09
# --- Labour (annualised salaries prorated to a 7-month project = 7/12) ---
months <- 7
pm_salary        <- 72000
n_team           <- 4
team_salary_each <- 38000
sundry_per_hour  <- 11
sundry_hours     <- 1800          # total support hours booked to the project

labour <- (pm_salary + n_team * team_salary_each) * months / 12 +
          sundry_per_hour * sundry_hours

# --- Materials (one pick inside each brief range) ---
materials <- c(
  chassis_casting   = 55000,
  fasteners         = 9000,
  motor_controller  = 140000,
  carbon_fibre      = 0,          # not selected this time
  aluminium         = 85000,
  welding           = 25000,
  circuit_boards    = 32000,
  gov_approval      = 6000,
  testing           = 60000,
  tooling           = 150000,
  safety_equipment  = 50000,
  packaging_shipping= 15000
)
material_total <- sum(materials)

subtotal      <- labour + material_total
contingency_r <- 0.15
```

```
contingency   <- contingency_r * subtotal
grand_total   <- subtotal + contingency

charter_ceiling <- 800000         # what Senior Management has authorised
headroom <- charter_ceiling - grand_total

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", sc

stack <- matrix(c(labour, material_total, contingency) / 1000, ncol = 1)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(stack, col = c("#2980b9", "#16a085", "#e67e22"), xlim = c(0, 2.6),
        ylim = c(0, max(grand_total, charter_ceiling) / 1000 * 1.1), names.arg = "Budget",
        ylab = "$ thousand", main = "Budget vs. ceiling")
mids <- cumsum(stack) - stack / 2
text(0.7, mids, c("Labour", "Materials", "Contingency"), col = "white", cex = 1.2)
abline(h = charter_ceiling / 1000, lty = 2, lwd = 3, col = "#e74c3c")
text(2.2, charter_ceiling / 1000, "Ceiling", pos = 3, col = "#e74c3c", cex = 1.2)

cat(sprintf("GRAND TOTAL %s\n", money(grand_total)))
if (headroom >= 0) {
  cat(sprintf("Versus the %s ceiling  %s headroom\n", money(charter_ceiling), money(headroom)))
} else {
  cat(sprintf("Versus the %s ceiling  !!%s over!!\n", money(charter_ceiling), money(-headroom)))
}
cat(sprintf("Labour subtotal  %s\n", money(labour)))
cat(sprintf("Material subtotal  %s\n", money(material_total)))
cat(sprintf("Subtotal  %s\n", money(subtotal)))
cat(sprintf("Contingency (%g%%)  %s\n", contingency_r * 100, money(contingency)))
```

How would top-down vs. bottom-up change this number?

The cell above is **bottom-up**: it sums the pieces and lets the total fall out. A **top-down** pass would start with a figure Senior Management is willing to fund — say "$650k, no more" — and force each line to fit. In practice you do both: build bottom-up for defensibility, compare to the top-down ceiling, and negotiate the gap before the Charter is signed. If bottom-up comes in well over the ceiling, that is a **feasibility** finding, not a rounding problem.

9.11 Key Takeaways

- Cost management starts in **planning**. Sources: labour, materials, sub-contractors, equipment/facilities, travel, shipping.
- Every cost carries **two or three** of the eight labels: direct/indirect, fixed/variable, normal/expedited, recurring/non-recurring. **Direct labour cost = rate × hours**; indirect is often **20–50 % of direct**.

- Estimating method follows project stage: **ballpark ±30 %, parametric ±15 %, feasibility ±10 %, definitive ±5 %** — the **cone of uncertainty**. Always quote a **range**, and keep "estimate" separate from "commitment."
- Estimate quality is driven by planning horizon, duration, people, structure, culture, and **padding** — which compounds into a meaningless total. Remove pads from work packages; hold one visible contingency.
- **Top-down** imposes discipline but breeds friction; **bottom-up** (via **activity-based costing** on the WBS) forces real planning. ABC = identify activities → cost drivers → rate per unit → rate × units.
- A **time-phased budget** produces the **cumulative cost baseline (S-curve)** you track against later.
- **Contingency reserve** (10–15 %, PM-owned, inside the baseline, covers known-unknowns) is distinct from the **management reserve** (sponsor-owned, outside the baseline, covers unknown-unknowns). Contingency is added **after** ABC, never inside it.

9.12 Case Study: Meridian R/C — Week 9

CASE STUDY FILE — Week 9: The Bottom-Up Budget Goes to Daniel Cho

Six weeks after Daniel Cho signed off on the $800,000 charter ceiling, you owe him something more defensible than a ballpark: a definitive, bottom-up budget built activity by activity off the WBS you cut with Priya Nandakumar, Marcus Webb, and Sofia Reyes back in Week 5. This week you assembled it, work package by work package, and sat down with Daniel for sign-off.

You started with labour, costing each team member's time against the prototype phase: yourself as PM at $72,000, Priya (tooling) at $42,000, Marcus (procurement) at $38,000, and Sofia (quality and test) at $36,000 — a labour subtotal of **$188,000**. Then you rolled up the direct-cost line items Marcus, Priya, and Sofia had been pricing with suppliers all along: the motor and controller package MCS had quoted at $145,000, the aluminium for the body and frame (the Week 1 material decision) at $95,000, the body and frame tooling dies at $150,000, testing and certification fees at $60,000, government and toy-safety approval fees at $6,000, and packaging and shipping for the client sample run at $18,000 — a direct-cost subtotal of **$474,000**.

Labour plus direct costs gives total direct costs of **$662,000**. Following the course's Costs Chart convention — and remembering that contingency is never buried inside an activity-based budget, only added on top once every line item is identified — you applied a **15% contingency on the full $662,000 direct-cost total**, not just the material-and-vendor subtotal.

That adds **$99,300**, for a **Budget at Completion (BAC) of $761,300**. Daniel did the arithmetic you knew he would: $761,300 against his $800,000 ceiling leaves **$38,700 of headroom** — about 4.8% of the ceiling. Comfortable, but not so much that he waved it through without questions. He pressed hardest on the $150,000 tooling line (largest single item after labour) and on why testing and certification, at $60,000, was priced near the top of the brief's original $25,000–$100,000 range rather than the middle. You walked him through the WBS lineage of each number — 1.2 Tooling & Fabrication, 1.5 Testing & Certification — and he signed off. The bottom-up number now becomes the baseline you will track against once the S-curve is time-phased and, later, once Earned Value tracking begins in Week 12.

9.13 Practice Problems

Problem 1: Fully-burdened labour and an indirect-rate swing

A test campaign needs 2 technicians for 3 weeks (37.5 h/week each) at $34/h, plus $6 200 of consumables. Compute the fully-burdened cost at an indirect rate of 25 %, and again at 45 %. How much does the indirect-rate assumption alone move the number?

Solution

```
techs <- 2; weeks <- 3; hrs_week <- 37.5; rate <- 34
consumables <- 6200

direct_labour <- techs * weeks * hrs_week * rate
direct_cost   <- direct_labour + consumables

for (ir in c(0.25, 0.45)) {
  total <- direct_cost * (1 + ir)
  cat(sprintf("Indirect %2.0f%%:  direct $%s  +  indirect $%s  =  $%s\n",
              ir * 100,
              format(direct_cost, big.mark = " "),
              format(round(direct_cost * ir), big.mark = " "),
              format(round(total), big.mark = " ")))
}
cat(sprintf("\nSwing from the indirect-rate assumption alone: $%s\n",
            format(round(direct_cost * (0.45 - 0.25)), big.mark = " ")))
```

```
Indirect 25%:  direct $13 850  +  indirect $3 462  =  $17 312
Indirect 45%:  direct $13 850  +  indirect $6 232  =  $20 082

Swing from the indirect-rate assumption alone: $2 770
```

The direct cost is fixed at \$21 500, but the burdened total moves by \$4 300 — 20 % of direct — purely on the overhead assumption. Always state the indirect rate you used.

Problem 2: Which estimating method, and what range?

For each situation, name the method and give the ± band around a \$500 000 point estimate.

(a) A retailer asks, in a hallway, roughly what a new product line will cost. You have a concept sketch.
(b) You have run three near-identical RC-vehicle launches in the last four years with clean cost records.
(c) All major POs are drafted and 90 % of the design is released for a firm-fixed-price defense bid.

Solution

- (a) **Ballpark**, ±30 % → \$350 000 – \$650 000.
- (b) **Comparative/parametric**, ±15 % → \$425 000 – \$575 000.
- (c) **Definitive**, ±5 % → \$475 000 – \$525 000. (Anything looser is a bad idea on a fixed-price bid.)

Problem 3: Variance analysis on the ABC budget

Using the preliminary ABC budget above (total planned \$38 500), the project finishes with these actuals: Survey \$4 250, Design \$8 000, Clear site \$3 500, Foundation \$8 500, Framing \$11 250, Plumb & wire \$5 150. Find the variance per activity and overall, and flag the activities that need a post-mortem.

Solution

```
planned <- c(Survey = 4000, Design = 8000, `Clear site` = 4000,
             Foundation = 7500, Framing = 10000, `Plumb & wire` = 5000)
actual  <- c(4250, 8000, 3500, 8500, 11250, 5150)

variance <- planned - actual            # + = under budget
tab <- data.frame(Activity = names(planned),
                  Planned = as.integer(planned),
                  Actual = as.integer(actual),
                  Variance = as.integer(variance),
                  Pct = round(100 * variance / planned, 1))
print(tab, row.names = FALSE)

cat(sprintf("\nTOTAL planned $%s  actual $%s  variance $%s (%.1f%%)\n",
            format(sum(planned), big.mark = " "),
            format(sum(actual),  big.mark = " "),
            format(sum(variance), big.mark = " "),
            100 * sum(variance) / sum(planned)))
```

```
     Activity Planned Actual Variance   Pct
       Survey    4000   4250     -250  -6.2
       Design    8000   8000        0   0.0
   Clear site    4000   3500      500  12.5
   Foundation    7500   8500    -1000 -13.3
      Framing   10000  11250    -1250 -12.5
 Plumb & wire    5000   5150     -150  -3.0

TOTAL planned $38 500  actual $40 650  variance $-2 150 (-5.6%)
```

Design and (roughly) Survey held. **Foundation (-13 %)** and **Framing (-13 %)** blew through and drove the whole project $2 150 (5.6 %) over — those two get the post-mortem.

Problem 4 (discussion): sizing contingency for the RC-vehicle project

The project brief flags several live risks: one candidate motor supplier is unproven, that supplier's staff are negotiating a first labour contract, the carbon-fibre supplier is mid-restructuring after an acquisition, and a new assembly line would carry a learning curve. The subtotal budget is $620 000.

Would you size the contingency at 10 %, 15 %, or 20 %? Would you spread it across all tasks or ring-fence it for specific ones? Defend your answer.

Discussion points

With that many concentrated, plausible risks, **10 % is light**; 15 % is defensible and matches the course's Costs Chart; 20 % is arguable if the unproven-supplier and new-line risks are both realised. A strong answer **ring-fences** most of the reserve against the supplier and assembly-line line items (the tasks critical to success) rather than smearing it evenly, and states that split in the scope statement so stakeholders are not surprised when it is drawn down.

9.14 Further Resources

- **Cost estimating techniques (analogous, parametric, bottom-up):** curated explainer videos below.
- Preview of Chapter 12 — Project Control Systems: the S-curve you built here becomes the baseline for **earned value** analysis.

The Basics of Project Cost Management (ProjectManager): https://www.youtube.com/watch?v=EyPFi0YO32M

Cost Estimates - Key Concepts in Project Management (David McLachlan): https://www.youtube.com/watch?v=hfn9VCAj4Dw

How to Estimate Project Costs Accurately: https://www.youtube.com/watch?v=2IxoWVjsphc

What Is Earned Value? (Ricardo Vargas) - looking ahead to project control: https://www.youtube.com/watch?v=x0EgZnXdunw

Chapter 10

Negotiation and Conflict Management

10.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Identify** what a project manager negotiates and with whom.
2. **Prepare** for a negotiation: interests vs. positions, BATNA, reservation point, ZOPA.
3. **Apply** the principled-negotiation mindset (separate people from problem; focus on interests; win-win or no win).
4. **Determine** whether a zone of possible agreement exists and how wide it is.
5. **Classify** project conflict (goal-oriented, administrative, interpersonal) and its organizational vs. interpersonal causes.
6. **Select** a conflict-resolution method (confronting, compromising, withdrawal, smoothing, forcing) for a situation.
7. **Score** a multi-criteria supplier choice.

Before we begin: recall a negotiation you "won" but regretted

Did you win the point and lose the relationship? Most project negotiations are with people you will work with again next week. That constraint changes everything.

10.2 Introduction: the project manager negotiates every day

As a project manager your day is full of negotiation, inside and outside the organization. You negotiate for or with:

- **Resources** — material, time, equipment, money
- **Contractors** — costs, delivery schedules, quality of material, hours of work
- **Department managers** — manpower, schedules

On the RC-vehicle project the sharp negotiations are obvious: the **motor and control-system supply contract** (trusted MCS, now moving production to Mexico, vs. unproven Paragon four hours away with staff negotiating a first labour contract), and the recurring fight with **functional managers** over which engineers you get and for how long.

> **Note**
>
> **Industry snapshot.** An automotive buyer negotiating a multi-year harness contract is really negotiating the *whole package* — price, tooling ownership, capacity guarantees, engineering support, payment terms. A defense subcontract negotiation is bounded by regulation (allowable cost, audit rights). In a food company, the PM negotiating a sanitation window with operations is negotiating *downtime*, which is money.

Project Management Negotiations: ZOPA, BATNA, and Win-Win (Parallel Project Training): https://www.youtube.com/watch?v=MTnQX11Xey0

10.3 The negotiation mindset

- **Separate the people from the problem.** Everyone has ego, fears, insecurities — do not tap into the other side's, or you lose control of both sides.
- **Stick to business.** Keep personalities out of it; a charm offensive fools no one.
- **Put yourself in their shoes.** Nobody likes a direct attack; it is counter-productive.
- **Win-win or no win.** Talk money **last** if possible; a negotiation is about the whole package, not just the number.

10.4 Getting ready

There are three stages — **before, during, after** — and **before should be the biggest.**

Gather information about the other side: their vital interests, goals, costs, strategies, positions, needs; their product strengths; concession patterns; past negotiating style; their **bottom line**; corporate deadlines and time constraints; their **budget**; geographic limits. Also know your own position, the players, and **who holds the power** — including people behind the scenes.

Develop strategy: aim high (with more than one win-win scenario in mind); be realistic; rank your objectives by priority; know your **bottom line**; decide whether deadlock or termination are tolerable outcomes; separate the **necessary** items from the **expendable**; have a few **red herrings** ready; plan how to help the other side sell the deal internally; find where your interests and theirs intersect.

Write the plan and an agenda: your interests and theirs; your desired outcomes and your estimate of theirs; your authority limits and theirs; any fact sheet either side needs. The agenda names the meeting's goal, is published in advance, and lists invitees and location.

10.4.1 The three questions — and who controls the room

Every negotiator should ask: **How much power do I have? What time pressure is there? Do I trust the other side?**

And: *who controls a conversation?* **Generally the person asking the questions**, because they set the direction. Aim to ask most of the questions — politely — and recognise when it is being done to you.

10.5 BATNA, reservation point, and ZOPA

- **BATNA** — your Best Alternative To a Negotiated Agreement. What you will do if this deal falls through. A strong BATNA is your real source of power.
- **Reservation point** — the worst deal you would still accept (a buyer's *maximum* price, a seller's *minimum* price), set by your BATNA.
- **ZOPA** — the Zone Of Possible Agreement: the overlap between the two reservation points. If there is no overlap, **no deal is possible** at those terms.

10.5.1 Worked Example 1: is there a ZOPA?

You are buying the motor/controller package. Your BATNA (Paragon's quote plus the cost of the risk it carries) puts your **reservation price at $168 000**

— you would not pay MCS more than that. MCS's floor, given their costs and other customers, is **$150 000**.

Live cell — move either side's reservation price and see whether a deal still exists.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: zopa-example-10
buyer_reservation  <- 168000   # most you will pay
seller_reservation <- 150000   # least they will accept

zopa <- seller_reservation <= buyer_reservation

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

lo  <- min(buyer_reservation, seller_reservation)
hi  <- max(buyer_reservation, seller_reservation)
pad <- max(hi - lo, 20000) * 0.5
ticks <- pretty(c(lo - pad, hi + pad), n = 5)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(NA, xlim = c(lo - pad, hi + pad), ylim = c(0, 1), xaxt = "n", yaxt = "n",
     xlab = "Price ($)", ylab = "", main = "Zone of possible agreement")
rect(lo, 0.3, hi, 0.7, col = if (zopa) "#a9dfbf" else "#f5b7b1", border = NA)
axis(1, at = ticks, labels = format(ticks, big.mark = " ", scientific = FALSE, trim = T
points(seller_reservation, 0.5, pch = 19, cex = 3, col = "#2980b9")
points(buyer_reservation, 0.5, pch = 19, cex = 3, col = "#e67e22")
text(seller_reservation, 0.85, "Seller floor", col = "#2980b9", cex = 1.2)
text(buyer_reservation, 0.15, "Buyer ceiling", col = "#e67e22", cex = 1.2)

if (zopa) {
  cat(sprintf("ZOPA exists: %s to %s\n", money(seller_reservation), money(buyer_reserva
  cat(sprintf("ZOPA width  %s\n", money(buyer_reservation - seller_reservation)))
} else {
  cat("!!No ZOPA!!: the seller's floor is above the buyer's ceiling\n")
  cat(sprintf("Gap between the two sides  %s\n", money(seller_reservation - buyer_reser
}
```

10.5.2 Try it yourself: reservation points from a BATNA

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and

press **Run Code** to experiment further.

```
#| label: zopa-calc-10
# Buyer's side: BATNA is Paragon at $150k, but carrying $25k of extra risk cost.
paragon_price   <- 150000
paragon_risk    <- 25000
buyer_reservation <- paragon_price + paragon_risk   # indifference point vs MCS

# Seller's side (MCS): cost to make + minimum acceptable margin.
mcs_cost   <- 132000
mcs_margin <- 15000
seller_reservation <- mcs_cost + mcs_margin

zopa <- seller_reservation <= buyer_reservation

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", sc

lo  <- min(buyer_reservation, seller_reservation)
hi  <- max(buyer_reservation, seller_reservation)
pad <- max(hi - lo, 20000) * 0.5
ticks <- pretty(c(lo - pad, hi + pad), n = 5)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
plot(NA, xlim = c(lo - pad, hi + pad), ylim = c(0, 1), xaxt = "n", yaxt = "n",
     xlab = "Price ($)", ylab = "", main = "Zone of possible agreement")
rect(lo, 0.3, hi, 0.7, col = if (zopa) "#a9dfbf" else "#f5b7b1", border = NA)
axis(1, at = ticks, labels = format(ticks, big.mark = " ", scientific = FALSE, trim = TRUE))
points(seller_reservation, 0.5, pch = 19, cex = 3, col = "#2980b9")
points(buyer_reservation, 0.5, pch = 19, cex = 3, col = "#e67e22")
text(seller_reservation, 0.85, "Seller floor", col = "#2980b9", cex = 1.2)
text(buyer_reservation, 0.15, "Buyer ceiling", col = "#e67e22", cex = 1.2)

if (zopa) {
  cat(sprintf("ZOPA width: %s: a deal is possible\n", money(buyer_reservation - seller_reservatio
} else {
  cat("!!No ZOPA!!: improve your BATNA, expand the package, or walk away\n")
}
cat(sprintf("Buyer reservation (max)  %s\n", money(buyer_reservation)))
cat(sprintf("Seller reservation (min)  %s\n", money(seller_reservation)))
if (zopa) cat(sprintf("Midpoint settlement  about %s\n", money((buyer_reservation + seller_reserv
```

Why does improving your BATNA move the whole negotiation?

Your reservation point is set by your BATNA. If you pre-qualify a credible second supplier (a better BATNA), your reservation price drops — you can credibly walk at a lower number — which shifts the ZOPA and the likely settlement in your favour, *without* saying a word at the table. The best negotiating move often happens before the meeting, in the form of a real alternative.

10.5.3 Try it yourself: reservation points and the ZOPA by hand

The buyer's reservation price comes from their BATNA, the seller's from their costs. Work out both and the width of the zone of possible agreement. Each team picks its own **data set**.

Is there a zone of possible agreement?

Reservation price = BATNA price plus its risk cost (buyer); cost plus minimum margin (seller).

- Buyer's BATNA: alternative supplier's price: $150 000
- Extra risk cost of using that alternative: $25 000
- Seller's cost to make: $132 000
- Seller's minimum margin: $15 000
- Buyer's reservation price (most they will pay): $_____ *(to 0 decimal places)*
- Seller's reservation price (least they will accept): $_____ *(to 0 decimal places)*
- ZOPA width (negative means there is no ZOPA): $_____ *(to 0 decimal places)*

Show the worked solution

Buyer's reservation price = BATNA price + risk cost; seller's = cost to make + margin. Data set 1: buyer $175 000, seller $147 000, so the ZOPA is $147 000 to $175 000, width **$28 000**. In data set 3 the seller's floor ($235 000) is above the buyer's ceiling ($225 000): width **-$10 000**, no ZOPA at these terms; the buyer must improve their BATNA or change the package.

10.6 Language and tells

A few sample moves: *"My colleagues and I cannot settle for that"* (establish power early); *"What are you looking for in these negotiations?"* (make them open first); *"Do you have the authority to commit to that?"* (probe their authority); *"Let's set that aside and come back to it"* (avoid a hard no).

And listen past the words. *"To be brutally honest…"* and *"To tell you the truth…"* often precede a lie. *"Off the record…"* — nothing is off the record; the negotiation is always on. Watch **congruence**: when words and body language disagree, believe the body language.

10.7 Conflict management

Conflict *"begins when you perceive that someone has frustrated, or is about to frustrate, a major concern of yours."* It is **perceptual** (it doesn't matter whether a wrong was actually done, only that it is perceived) and **dynamic** (the original cause may no longer matter, yet the conflict persists).

Three types on projects:

- **Goal-oriented** — disagreement over end results, scope outcomes, specs, priorities. Fuelled by vague goals that let people fill in their own interpretation (and hidden agendas).
- **Administrative** — who has authority or control over functions, tasks, and decisions. The **matrix** structure breeds this — everyone has two bosses.
- **Interpersonal** — personality differences: work ethic, behavioural style, ego, personalities that don't mesh.

10.7.1 Try it yourself: what type of conflict is it?

Goal-oriented, administrative or interpersonal?

Remember: personality is the first diagnosis and usually the wrong one.

Categories: Goal-oriented / Administrative / Interpersonal

1. Marketing and engineering disagree on what the finished prototype must demonstrate. _____
2. Who signs off a design change: the PM or the functional manager? _____
3. Two engineers have clashed on every project since a personal falling-out. _____
4. Vague specifications let each side read the requirement its own way. _____
5. A matrix employee takes instructions from two bosses who contradict each other. _____
6. One teammate's blunt style keeps rubbing another the wrong way. _____
7. The customer wants it lighter and the sponsor wants it cheaper. _____
8. A disagreement over who controls the test-lab schedule. _____

Answers: 1 Goal-oriented; 2 Administrative; 3 Interpersonal; 4 Goal-oriented; 5 Administrative; 6 Interpersonal; 7 Goal-oriented; 8 Administrative.

10.7.2 Three views of conflict

School	View	Response
Traditional	All conflict is bad	Suppress and eliminate it, fast
Behavioral	Conflict is natural and inevitable	Accept it and manage it well
Interactionist	A *controlled* amount of conflict is *beneficial*	Encourage some — it prevents stagnation and drives innovation; too little = inertia, too much = chaos

10.7.3 Sources, ranked by intensity

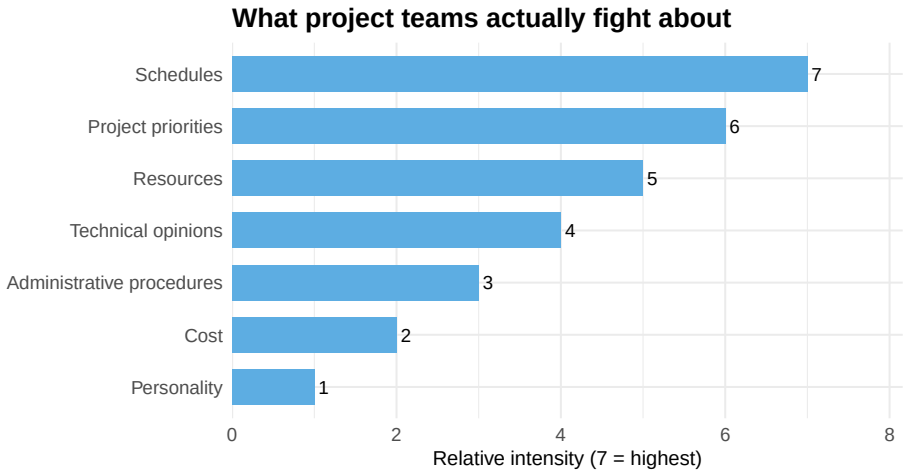


Figure 10.1: Sources of project conflict ranked by typical intensity (from PMBOK data). Schedules, priorities, and resources dominate; personality is last – it is usually the first thing blamed and rarely the real cause.

Organizational causes: reward systems (functional managers hoard their best people), scarce resources, uncertainty (weak matrix — unclear who is in charge), different mindsets among departments. **Interpersonal causes:** misconceptions about others' behaviour, faulty communication (often *style*, not content), personal grudges.

> *"It only becomes personal when the root cause of the conflict is not resolved. Personality is very frequently the first diagnosis"* — and usually the wrong one.

10.7.4 Conflict-resolution methods (PMBOK)

Method	What it is	Best when
Confronting (problem-solving)	Solve the real, underlying problem	There is time and goodwill; the durable fix
Compromising	Each side gives something; partial satisfaction for both	Positions are far apart and a deadline looms
Withdrawal / avoidance	Retreat or postpone	The issue is trivial, or emotions need to cool
Smoothing	Emphasize agreement, play down differences	Preserving the relationship matters more right now
Forcing	Push one view at the other's expense	A true emergency, or a matter of safety/ethics

10.7.5 Try it yourself: choose the resolution method

Which resolution method fits each situation?

Pick the method that fits best.

Categories: Confronting / Compromising / Withdrawal / Smoothing / Forcing

1. A disagreement over a spec detail, with plenty of time and goodwill: solve the real problem. _____
2. The two sides are far apart and the gate review is tomorrow: each gives something. _____
3. A trivial scheduling grumble, with tempers high: postpone it. _____
4. Preserving a key relationship matters more than winning this point today. _____
5. A safety interlock has been bypassed on the line: stop it now. _____
6. Repeated hand-off errors need a joint working session on the root cause. _____
7. Time is short, so split the difference between the two delivery dates. _____
8. Emphasize what the two teams agree on and play down the disagreement for now. _____

Answers: 1 Confronting; 2 Compromising; 3 Withdrawal; 4 Smoothing; 5 Forcing; 6 Confronting; 7 Compromising; 8 Smoothing.

10.7.6 Try it yourself: score the supplier choice

Multi-criteria scoring turns "MCS feels safer" into a defensible number.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: supplier-scoring-10
criteria <- c("Quality record", "Delivery reliability", "Price",
              "Technical capability", "Supply-chain risk")
weight   <- c(0.30, 0.25, 0.15, 0.15, 0.15)   # must sum to 1

# Score each supplier 1 (poor) to 5 (excellent) on each criterion.
mcs     <- c(5, 4, 3, 3, 2)   # trusted, but production moving to Mexico
paragon <- c(3, 3, 4, 5, 3)   # unproven, new tech, plant 4 h away, labour talks

mcs_total     <- sum(weight * mcs)
paragon_total <- sum(weight * paragon)

cat(sprintf("Recommended: %s\n", ifelse(mcs_total >= paragon_total, "MCS", "Paragon")))
cat(sprintf("MCS weighted score  %.2f\n", mcs_total))
cat(sprintf("Paragon weighted score  %.2f\n", paragon_total))
cat(sprintf("Weights sum to  %s\n", ifelse(abs(sum(weight) - 1) < 1e-9, "1.00 (ok)",
                                           sprintf("!!%.2f (should be 1)!!", sum(weight)
cat("Change the weights to your project's priorities and watch the answer move.\n")
```

10.7.7 Try it yourself: weighted supplier scoring by hand

Multiply each score by its weight, then total each supplier's column. Each team picks its own **data set**.

Weighted supplier score

Scores run from 1 (poor) to 5 (excellent); the weights sum to 1.

Criterion	Weight	Alpha score	Beta score	Gamma score	Alpha x weight	Beta x weight	Gamma x weight
Quality record	0.40	4.00	5.00	3.00			
Delivery reliability	0.30	3.00	4.00	5.00			
Price	0.20	5.00	2.00	4.00			
Technical capability	0.10	2.00	3.00	5.00			

- Alpha total: _____ *(to 2 decimal places)*
- Beta total: _____ *(to 2 decimal places)*
- Gamma total: _____ *(to 2 decimal places)*

Show the worked solution

Weighted total = sum of (score x weight) down each supplier's column. In data set 1: Alpha 3.70, Beta 3.90, Gamma 4.00, so Gamma is recommended. The ranking depends on the weights: shift weight from Price to Quality and re-run the sums to see how fragile a narrow win can be.

10.8 Key Takeaways

- PMs negotiate constantly — for resources, with contractors, with functional managers. **Before** is the biggest stage; prepare more than you think you need.
- **Separate people from the problem**, focus on **interests** not positions, aim for **win-win or no win**, and talk money last.
- **BATNA** sets your **reservation point**; the overlap of both reservation points is the **ZOPA**. No ZOPA, no deal — improve your BATNA or change the package.
- The person **asking the questions** usually controls the conversation.
- Project conflict is **perceptual and dynamic**. Types: **goal-oriented, administrative, interpersonal**. The **interactionist** view holds that a controlled amount of conflict is healthy.
- Teams fight most about **schedules, priorities, and resources** — **personality** is last and is usually a misdiagnosis of an unresolved root cause.
- Match the resolution method — **confronting, compromising, withdrawal, smoothing, forcing** — to the situation; confronting is the durable fix.

10.9 Case Study: Meridian R/C — Week 10

CASE STUDY FILE — Week 10: The MCS/Paragon Showdown

By Week 10, the motor and controller package — the single biggest line item in Meridian's budget and the longest-lead, critical-path activity in the schedule — stops being a formality. The Week 9 budget assumed MCS's original quote: **$145,000** for the full package, with a **25-day lead time**. It is the number already baked into every schedule and cost baseline you have built so far.

Then Marcus Webb, your procurement lead, brings you a competing quote. Paragon, the newer Kingston, Ontario supplier, will do the same package for **$118,000** — in just **20 days**. Marcus wants to switch immediately. Sofia Reyes, your quality and test engineer, pushes back hard: Paragon's controller technology is unproven, and the Week 7 risk register already flagged it as **R1** — a 30% chance the technology fails qualification, at an $80,000 impact, for an EMV of **$24,000**. Add that risk cost to Paragon's sticker price and the real, risk-adjusted cost is **$118,000 + $24,000 =**

$142,000 — narrowly cheaper than MCS on paper, but not the bargain it first appears.
You will not let the team split the difference by shouting. Your BATNA going into the negotiation is simple and safe: proceed with MCS at $145,000, the proven supplier, with no qualification risk. But a strong BATNA is leverage, not a decision — so instead of picking a side in the Marcus/Sofia standoff, you take Paragon's quote back to MCS. MCS knows the stakes: losing this contract could cost them the follow-on 100,000-unit production order Kestrel Retail Group is set to place once Meridian's prototype passes testing. Facing a credible alternative, MCS concedes — dropping its price to **$136,000**, a **$9,000 (6.2%)** reduction from its original quote, while holding its 25-day lead time.
The outcome resolves both negotiations at once. Meridian keeps its proven supplier and captures real, verified savings, with no new qualification risk introduced. And the Marcus/Sofia conflict is settled not by forcing one side to lose, but through **compromising**: Marcus gets the lower price he wanted, Sofia keeps the proven supplier she insisted on. Neither gets everything; both get enough.

10.10 Practice Problems

Problem 1: Name the cause of conflict

Which cause does each statement point to?

1. "I can't talk to her after what her son said to my son."
2. "Sales management here are lazy — they always leave by 4:00."
3. "She barely looked at my report; she's already decided to ignore it."
4. "Our computers are slow because their division always gets the best ones."
5. "The worse your project does at year-end, the better mine looks."
6. "Why should I do that for the PM if my functional manager didn't tell me?"

Answers

1. **Personal grudge** (interpersonal).
2. **Different mindsets among departments** (organizational).
3. **Misconceptions about others' behaviour** (interpersonal).
4. **Scarce resources** (organizational).
5. **Reward systems** (organizational).
6. **Uncertainty / two bosses** (organizational, matrix).

Problem 2: ZOPA and a walk-away

Your BATNA for a tooling package is to build it in-house for $98 000, so that is the most you will pay. The vendor's floor is $102 000.

(a) Is there a ZOPA? (b) What are your options?

Solution

```r
buyer_max  <- 98000   # set by your BATNA: the in-house cost
seller_min <- 102000
cat(ifelse(seller_min <= buyer_max, "ZOPA exists", "No ZOPA"), "\n")
cat(sprintf("Gap to close: $%s\n", format(seller_min - buyer_max, big.mark = " ")))
```

```
No ZOPA
Gap to close: $4 000
```

(a) No — the vendor's floor is $4 000 above your ceiling. (b) Exercise your BATNA (build in-house at $98 000), expand the package so the vendor's per-unit economics improve, split tooling ownership, or find a third vendor.

Problem 3 (discussion): the functional manager skips your stand-up

The maintenance supervisor on your project has missed the first two morning meetings. How could this signal *each* of: reward systems, scarce resources, uncertainty, different mindsets, misconceptions, faulty communication, personal grudge?

Discussion points

Reward systems: their boss doesn't value project work, so they deprioritize it. *Scarce resources*: they are triple-booked and the meeting lost. *Uncertainty*: they are not sure the meeting applies to them. *Different mindsets*: maintenance culture sees planning meetings as talk, not work. *Misconceptions*: they think you have already sidelined their input. *Faulty communication*: the invite never reached them, or the time was ambiguous. *Grudge*: a past project soured them on you. The point: **do not jump to the grudge** — confront it by asking, and you will usually find one of the structural causes.

10.11 Further Resources

Negotiation Strategy: ZOPA (Mediator Academy): https://www.youtube.com/watch?v=JEoFHePrp6k

BATNA and ZOPA in Negotiations: https://www.youtube.com/watch?v=kJWF2oYzTq0

Chapter 11

Resource Management

11.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Name** the constraint types on a project and focus on the resource constraints.
2. **Build** a resource histogram and spot over-allocation.
3. **Apply** resource **leveling** and the heuristics that decide which task moves first.
4. **Distinguish** time-constrained from resource-constrained scheduling.
5. **Apply** resource **smoothing** within available float.
6. **Explain** the multi-project resource problem and the priority rules used to resolve it.
7. **Quantify** the cost of multitasking on effective capacity.

Before we begin: your plan assumed the person was free — were they?

Almost every first-draft project schedule is built as if the organization has unlimited trained people standing by. It never does. This chapter is what happens when the plan meets the staffing reality.

11.2 Introduction: the optimism of the first schedule

The RC-vehicle schedule from Chapter 8 quietly assumed every resource was available exactly when needed. Reality: there is **one CMM** for validation, the **millwrights** are shared with production, the design engineers are **borrowed** from a functional department, and the new assembly line has a **learning curve**. *Building plans under the assumption of unlimited resources inevitably leads to*

schedule slippage and extra cost as reality overtakes the optimism.

Projects run inside three kinds of constraint — **technical**, **physical**, and **resource** — and this chapter is about the last one: **people, materials, space, money, equipment.**

> **Note**
>
> **Industry snapshot.** An automotive company with one set of climate chambers watches validation projects stack up behind that single resource — the classic bottleneck. A food plant schedules every project around the *one* weekend sanitation window. A defense program's cleared personnel are a fixed pool, so every new task is a reallocation, not an addition.

Resource Allocation and Leveling: Tutorial and Example (AF Math & Engineering): https://www.youtube.com/watch?v=JlUXah6fsLI

11.3 Resource loading and the histogram

A **resource histogram** plots how much of a resource is required over time. Stack each activity's demand day by day and the over-allocations jump out — provided you remember the chart assumes each person is available **100 % of the time**, which is rarely true.

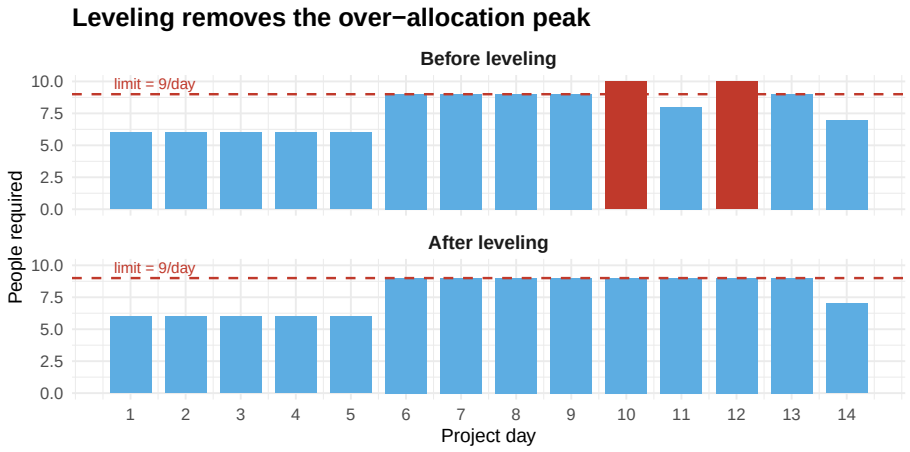


Figure 11.1: Resource histogram before and after leveling, against a limit of 9 people/day. Leveling shifts a task with float so the peak on day 12 no longer breaches the limit – without extending the project.

11.3.1 Try it yourself: build the histogram

Four tasks run on a fixed schedule. For each day, add up the people needed and decide whether the day is over the limit. Each team picks its own **data set** (and its own limit).

Resource histogram: people needed per day

The columns for Tasks A to D show the people each task needs on that day (0 if it is not running).

Day	Task A	Task B	Task C	Task D	Limit	Total	Over the limit? (Yes / No)
1	3	0	0	0	8		
2	3	0	0	0	8		
3	3	4	0	0	8		
4	3	4	0	3	8		
5	3	4	0	3	8		
6	0	4	2	3	8		
7	0	4	2	3	8		
8	0	4	2	0	8		
9	0	0	2	0	8		
10	0	0	2	0	8		

- Peak demand: _____ people *(to 0 decimal places)*
- Number of days over the limit: _____ days *(to 0 decimal places)*

Show the worked solution

Add the four task columns for each day. In data set 1 the totals are 3, 3, 7, 10, 10, 9, 9, 6, 2, 2 against a limit of 8, so days 4 to 7 are over the limit: four days, with a peak of 10. Those are the days leveling has to fix, by moving tasks that have float.

11.4 Resource leveling

> **PMBOK:** *a technique in which start and finish dates are adjusted based on resource constraints, with the goal of balancing demand for resources against the available supply.*

Two objectives: (1) make sure resources are available at the right time, and (2) let each activity run with the **smoothest possible transitions** across resource-usage levels.

When you must choose which activity to move, work through them in this order:

1. Activities with the **smallest float**
2. Activities with the **smallest duration**

3. Activities with the **lowest ID** (earliest in the WBS)
4. Activities with the **most successors**
5. Activities requiring the **most resources**

11.4.1 Worked Example 1: which task do you move?

Day 12 needs 10 people against a limit of 9. Four activities are running that day:

Live cell — change the float, duration, successors, or people for each task.

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: leveling-example-11
running_day12 <- data.frame(
  task = c("D", "E", "H", "J"),
  float = c(12, 0, 0, 5),
  duration = c(6, 6, 7, 3),
  successors = c(1, 2, 1, 0),
  people = c(3, 3, 3, 1)
)

# Only tasks with float can move without extending the project.
movable <- running_day12[running_day12$float > 0, ]
# Among movable tasks, prefer smallest float, then smallest duration.
pick <- movable[order(movable$float, movable$duration), ][1, ]

if (nrow(movable) == 0) {
  cat("!!No task has float: leveling would extend the project!!\n")
} else {
  cat(sprintf("Move first: task %s (float %g, duration %g)\n", pick$task, pick$float, p
  cat(sprintf("Movable tasks  %s\n", paste(movable$task, collapse = ", ")))
}
crit <- running_day12$task[running_day12$float == 0]
if (length(crit) > 0)
  cat(sprintf("Critical (float 0)  !!%s!!\n", paste(crit, collapse = ", ")))
cat("Critical tasks cannot move without extending the project.\n")
```

11.4.2 Try it yourself: which task do you move first?

Order the tasks by the leveling rule

Number these in the correct order:

- _____ P: float 2 days, duration 6 days, ID 3

- _____ Q: float 2 days, duration 4 days, ID 8
- _____ R: float 5 days, duration 3 days, ID 2
- _____ S: float 2 days, duration 4 days, ID 5
- _____ T: float 9 days, duration 7 days, ID 1

Answers, in the order listed: 3, 2, 4, 1, 5.

11.5 Time-constrained vs. resource-constrained

- **Time-constrained**: the end date is fixed; you add or reassign resources to protect it (this is closer to *smoothing*).
- **Resource-constrained**: the resource limit is fixed; if leveling within float is not enough, **the project duration extends** (undesirable, but sometimes unavoidable).

11.6 Resource smoothing

Smoothing is done **after** leveling and **never extends the project duration**. It uses available **float** to even out resource usage — filling the valleys from the peaks.

11.6.1 Try it yourself: smooth a resource within its float

Show the live R code behind this card

The card above re-runs this exact webR code every time you change a value in its table, substituting your values for the data at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: smoothing-11
# Weekly demand for one resource, and the resource's weekly limit.
weekly_demand <- c(52, 30, 61, 24, 45, 20, 58)
limit <- 45

peak <- max(weekly_demand)
total_work <- sum(weekly_demand)
weeks <- length(weekly_demand)
ideal_flat <- total_work / weeks

if (ideal_flat <= limit) {
  cat("Smoothing can solve this: the work fits the limit if spread evenly\n")
} else {
  cat("!!Smoothing is not enough: needs leveling or more resource!!\n")
}
cat(sprintf("Peak week demand  %g (limit %g, %g over)\n", peak, limit, max(0, peak - limit)))
cat(sprintf("Total work  %g resource-weeks over %d weeks\n", total_work, weeks))
```

```
cat(sprintf("Perfectly smooth  %.1f per week\n", ideal_flat))

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bar_col <- ifelse(weekly_demand > limit, "#c0392b", "#2980b9")
barplot(weekly_demand, names.arg = seq_len(weeks), col = bar_col,
        xlab = "Week", ylab = "Resource demand",
        main = "Red weeks exceed the limit (dashed line)")
abline(h = limit, lty = 2, lwd = 2)
abline(h = ideal_flat, lty = 3, lwd = 2, col = "#16a085")
```

11.7 Splitting a task

Splitting means interrupting continuous work on an activity, moving that resource to another activity, then returning. It helps *only* when there are no large **start-up or shutdown costs**, and **only some tasks can be split** at all (you cannot pause a concrete pour).

11.8 The multi-project resource problem

Organizations rarely buy new resources for each project — they **share** them across projects. The core challenge is **interdependency**: a resource decision in one project ripples into others. Common problems:

- Inefficient use of resources
- Resource **bottlenecks**
- **Ripple effects**
- Heightened pressure on people to **multitask**

Any multi-project scheduling system tries to minimize three things:

1. **Schedule slippage** — often carrying real financial penalty clauses per day late.
2. **Resource utilization** losses — the unused-capacity gap; the goal is to fill it with other projects' work.
3. **In-process inventory (WIP)** — work waiting on an unavailable resource, stacked behind a bottleneck (e.g. one shared climate chamber).

11.8.1 Priority rules for allocating shared resources

Rule	Logic
First in line (first come, first served)	Simplest — follow the master calendar; ignores priority and commercial value

Rule	Logic
Greatest resource demand	Schedule the most resource-hungry project first so bottlenecks surface early
Greatest resource utilization (most resource *types*)	Schedule the project that uses the widest mix of resources first, keeping all of them busy
Minimum late finish (earliest finish first)	Prioritize the project with the earliest late-finish date
Mathematical programming	Optimize (minimize total time / utilization loss / lateness); powerful but complex and slow for large problems

11.8.2 Try it yourself: which priority rule?

Which priority rule is each decision using?

Pick the rule each decision follows.

Categories: First in line / Greatest resource demand / Greatest resource utilization / Minimum late finish / Mathematical programming

1. Whichever project was approved first gets the CMM first. _____
2. Put the project with the biggest peak demand in first so bottlenecks surface early. _____
3. Favour the project using the widest mix of resource types, to keep them all busy. _____
4. The project with the earliest late-finish date goes first. _____
5. An optimization model minimizes total lateness across 40 projects. _____
6. Follow the master calendar and ignore commercial value. _____
7. One launch date is immovable and tighter than all the others, so that project goes first. _____

Answers: 1 First in line; 2 Greatest resource demand; 3 Greatest resource utilization; 4 Minimum late finish; 5 Mathematical programming; 6 First in line; 7 Minimum late finish.

11.8.3 Try it yourself: "40 hours is not one week's work"

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

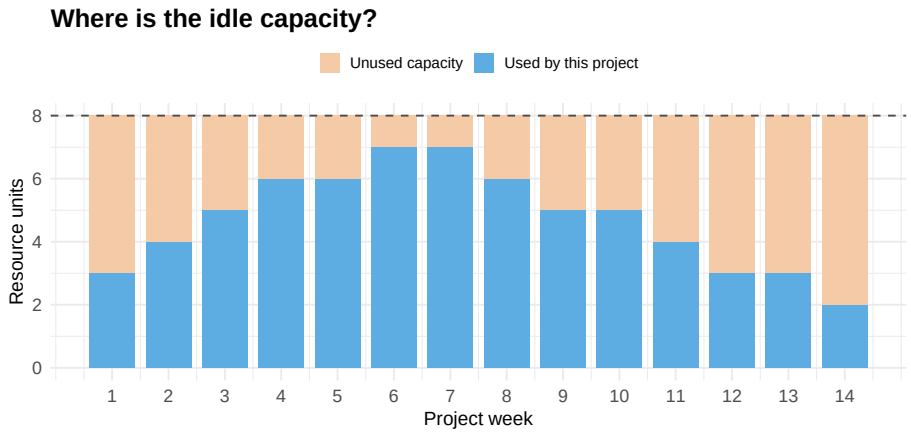


Figure 11.2: Resource utilization across a project. The shaded area is capacity the project does not use – in a multi-project shop it should be filled with concurrent projects' work rather than left idle.

```
#| label: multitask-11
nominal_hours <- 40
# Each additional concurrent project imposes a context-switching loss.
switch_loss_per_extra_project <- 0.15   # 15% of capacity lost per extra project
projects_carried <- 3

effective_hours <- function(projects) {
  extras <- projects - 1
  max(nominal_hours * (1 - switch_loss_per_extra_project * extras), 0)
}

counts <- 1:6
eff <- sapply(counts, effective_hours)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(eff, names.arg = counts, ylim = c(0, nominal_hours * 1.1),
              col = ifelse(counts == projects_carried, "#e74c3c", "#2980b9"),
              xlab = "Projects carried at once", ylab = "Effective hours per week",
              main = "Nominal hours are not effective hours")
abline(h = nominal_hours, lty = 2, lwd = 2, col = "#7f8c8d")

now <- effective_hours(projects_carried)
cat(sprintf("On %g project(s): %.1f effective hours of %g (%.0f%% productive)\n",
            projects_carried, now, nominal_hours, 100 * now / nominal_hours))
for (projects in counts) {
  cat(sprintf("On %d project(s)  %.1f effective hours (%.0f%% productive)\n",
              projects, eff[projects], 100 * eff[projects] / nominal_hours))
```

```
}
```

11.8.4 Try it yourself: effective hours by hand

Each extra project you carry costs a share of your capacity. Work out what is left. Each team picks its own **data set**.

Effective capacity when multitasking

Effective hours = nominal x (1 - loss x (projects - 1)).

- Nominal weekly hours: 40
- Capacity lost per extra project: 15 %
- Projects carried: 3
- Effective hours per week: _____ h *(to 2 decimal places)*
- Productive share of nominal hours: _____ % *(to 1 decimal place)*
- Hours lost per week: _____ h *(to 2 decimal places)*
- Effective hours with one project fewer: _____ h *(to 2 decimal places)*

Show the worked solution

With n projects there are n - 1 “extra” ones. Data set 1: 40 x (1 - 0.15 x 2) = 28 effective hours, 70 % productive, 12 h lost; dropping to 2 projects gives 40 x 0.85 = 34 h. Cutting one project recovers 6 h: the nominal 40 hours were never real.

11.9 Sustainability and resources

Resource management is not only about people-hours. The **circular-economy** thinking from Chapter 1 applies directly: minimize raw material use, prefer recyclable and local inputs, design the project so equipment and tooling can be reused on the next program, and track material and energy as managed resources with their own KPIs — not just labour and money.

11.10 Key Takeaways

- Projects run inside **technical, physical, and resource** constraints; resources are people, materials, space, money, equipment.
- A **resource histogram** exposes **over-allocation** (against a 100 %-availability assumption).
- **Leveling** shifts tasks within their **float** to respect a resource limit; move tasks in order of **smallest float → smallest duration → lowest ID**

→ **most successors** → **most resources**. Critical-path tasks have no float, so moving them extends the project.

- **Time-constrained** protects the date with resources; **resource-constrained** may extend the date.
- **Smoothing** happens after leveling, uses float, and **never** extends duration.
- **Splitting** helps only when start-up/shutdown costs are low and the task can be split.
- Multi-project sharing creates **bottlenecks, ripple effects, and multitasking pressure**; minimize **slippage, utilization loss, and WIP**, and pick a **priority rule** deliberately.
- **Multitasking is not free** — "40 hours" of nominal capacity is far less once someone carries several projects.

11.11 Case Study: Meridian R/C — Week 11

CASE STUDY FILE — Week 11: Priya's 160% Week

By Week 11 the tooling phase is in full swing, and two activities are running side by side: C (body tooling build, 20 days) and D (frame tooling build, 15 days), both released from B (detailed design release) on day 18. Priya Nandakumar, Meridian's Manufacturing Engineer, is the only person qualified to supervise both dies at once — the matrix organization never gave her a dedicated tooling team, just herself and two die-shop technicians who report through her for the duration of the project.

You pull the resource histogram for the tooling window and the number jumps out immediately: Priya is booked at 100% on C and 60% on D at the same time — 160% of one person's capacity, a 60-percentage-point overallocation that no amount of overtime coffee is going to fix on its own.

Before moving anything, you go back to the CPM table. C has 5 days of float (LS day 23, LF day 43). D has 10 days of float (LS day 28, LF day 43). Neither is on the critical path A–B–E–F–H–I–J, so neither move threatens the 67-day finish — but D gives you twice the room to work with. You shift D's start later, inside its own float, so Priya's two commitments no longer collide the way they did on paper. C keeps running on its original clock; D slides toward its late-start date of day 28 and still lands exactly on its late-finish date of day 43, precisely when F (prototype assembly) needs both dies ready. The overallocation is resolved, and the 67-day project finish never moves.

While you're in the histogram, you also pull the underlying weekly team-hours demand for the whole tooling phase against the department's 45-hour/week capacity ceiling: 30, 48, 44, 52, and 38 hours across five consecutive weeks. Weeks 2 and 4 breach the limit, by 3 and 7 hours respec-

tively — week 4 is the single worst overallocation — while weeks 1, 3, and 5 sit under capacity. Averaged out, the phase demands 42.4 hours per week, comfortably under the 45-hour limit. That average is the tell: there is enough slack in the valley weeks to absorb the peak weeks' extra hours using float alone, which makes this a smoothing problem rather than a leveling one — no added headcount, no extended schedule, just a better-shaped curve.

Weekly team-hours demand, tooling phase (capacity limit: 45 hrs/week)

Week	Demand (hrs)	vs. 45-hr limit
1	30	15 under
2	48	3 over
3	44	1 under
4	52	7 over (peak)
5	38	7 under

Activity float, for reference

Activity	Float	Critical?
B (detailed design release)	0 days	Yes
C (body tooling build)	5 days	No
D (frame tooling build)	10 days	No

11.12 Practice Problems

Problem 1: Level to the limit

Day 10 requires 10 people; the limit is 9. Running that day: task C (float 4, duration 5, 4 people), task D (float 0, duration 6, 3 people), task F (float 0, duration 6, 3 people). Which task moves, and what is the risk if none can?

Solution

```
d10 <- data.frame(task = c("C", "D", "F"),
                  float = c(4, 0, 0), dur = c(5, 6, 6), ppl = c(4, 3, 3))
mov <- d10[d10$float > 0, ]
cat("Only task with float:", mov$task, "\n")
cat("Move C into its 4 days of slack -> day-10 demand drops to 6, under the limit.\n")
cat("If C could not move (no split, hard dependency), the project duration extends.\n")
```

```
Only task with float: C
Move C into its 4 days of slack -> day-10 demand drops to 6, under the limit.
If C could not move (no split, hard dependency), the project duration extends.
```

Problem 2: Does this need leveling or smoothing?

A resource has a 40 h/week limit. Weekly demand over 6 weeks is 55, 20, 48, 15, 50, 22 h. Total 210 h.

Solution

```
demand <- c(55, 20, 48, 15, 50, 22); limit <- 40
flat <- sum(demand) / length(demand)
cat(sprintf("Even spread: %.1f h/week vs limit %d\n", flat, limit))
cat(ifelse(flat <= limit,
           "Fits if spread evenly -> smoothing (shift within float) can solve it.\n",
           "Still over even when flat -> leveling (extend) or add resource.\n"))
```

```
Even spread: 35.0 h/week vs limit 40
Fits if spread evenly -> smoothing (shift within float) can solve it.
```

35 h/week average is under the 40 h limit, so **smoothing** within float can solve it without extending the project.

Problem 3 (discussion): the shared CMM bottleneck

Three projects, including yours, all need the single CMM for final validation in the same two-week window. Using the priority rules, how would you decide the order, and what could reduce the bottleneck?

Discussion points

Apply a rule and be explicit about it: **minimum late finish** (whichever project has the tightest deadline goes first) is defensible if one launch date is immovable; **greatest resource demand** surfaces the bottleneck earliest. Reduce it by: staggering the validation tasks into each project's float, renting a second CMM for the peak, outsourcing some measurement to a metrology lab, or moving non-CMM checks off the critical window so the CMM time is used only for what truly needs it.

11.13 Further Resources

What Is Resource Leveling in Project Management?: https://www.youtube.com/watch?v=iSGWSGfG65U

Chapter 12

Project Control Systems

12.1 Learning Objectives

By the end of this chapter you will be able to:

1. **Describe** the monitoring-and-control loop and the performance measurement baseline.
2. **Read** the three earned-value curves: planned value (PV), earned value (EV), and actual cost (AC).
3. **Compute** schedule and cost variance and the SPI / CPI performance indices.
4. **Forecast** with EAC, ETC, VAC, and TCPI, and interpret them.
5. **Run** integrated change control so the baseline stays meaningful.
6. **Report** status honestly and recognize the "watermelon" trap.

Before we begin: the project has spent 60 % of its money — is it 60 % done?

Almost certainly not. "Percent of budget spent" and "percent of work finished" are different numbers, and the gap between them is the whole subject of this chapter.

12.2 Introduction: from a plan to a control system

You have planned the RC-vehicle project — scope, WBS, schedule, budget. It is now **week 8**. The sponsor asks the only question that matters at this stage: **are we on track, and if not, how far off, and what will it cost to finish?**

Answering that needs a **control system**: a closed loop that **measures** progress, **compares** it to the baseline, **analyzes** the variance, and drives a corrective **action** — then measures again.

```
Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0
i Please use the `linewidth` argument instead.
```

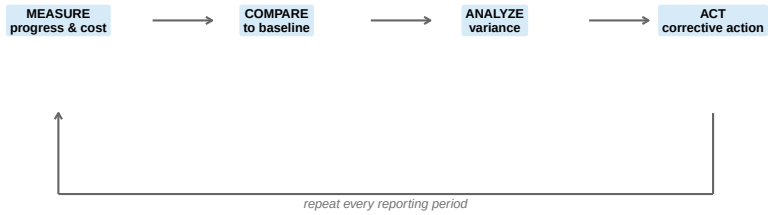


Figure 12.1: The monitoring-and-control loop. Without the 'compare to baseline' step, status reporting is just activity; with it, you get variance you can act on.

The reference for "compare" is the **performance measurement baseline** — the approved **scope + schedule + cost** locked in at the end of planning (Chapter 9's S-curve is its cost dimension).

> **Note**
>
> **Industry snapshot.** Earned value management is **contractually mandated** on large U.S. defense programs (EIA-748). Automotive programs run a lighter version at the milestone level. A food-plant capital project tracks committed vs. spent vs. installed. The math below is the same at every scale.

What Is Earned Value? (Ricardo Vargas): https://www.youtube.com/watch?v=x0EgZnXdunw

12.3 The three curves

Earned value management tracks three cumulative quantities against time:

Term	Also called	Meaning
PV — Planned Value	BCWS	The budgeted cost of the work **scheduled** to be done by now
EV — Earned Value	BCWP	The budgeted cost of the work **actually done** by now
AC — Actual Cost	ACWP	What has actually been **spent** to do that work

BAC (Budget At Completion) is the total baseline budget — where the PV curve ends.

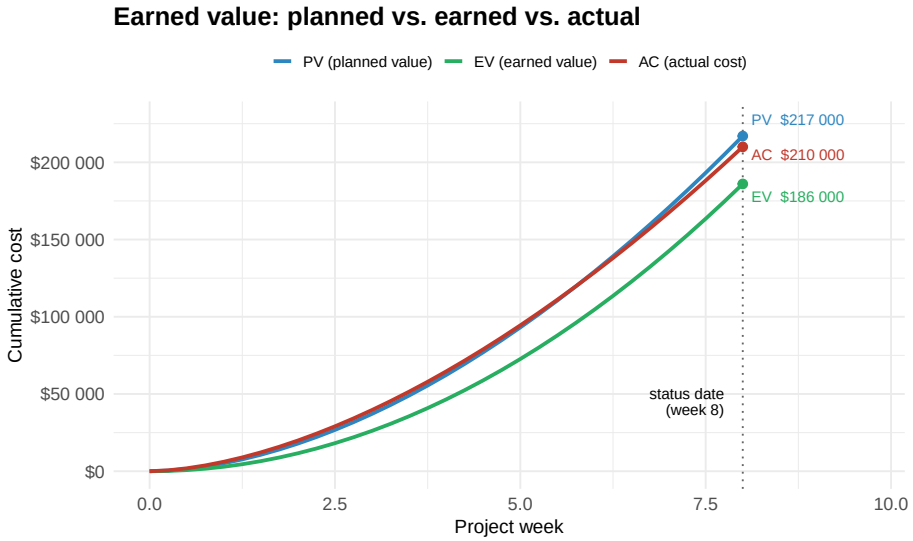


Figure 12.2: The three curves through week 8 of the RC-vehicle project (BAC = \$620 000, off the top of this view). By the status date EV sits below PV – behind schedule – and AC sits above EV – over cost. The vertical gaps at week 8 are the schedule and cost variances.

12.4 The six core metrics

$$\begin{aligned} \text{Schedule Variance} \quad & SV = EV - PV \\ \text{Cost Variance} \quad & CV = EV - AC \\ \text{Schedule Performance Index} \quad & SPI = EV/PV \\ \text{Cost Performance Index} \quad & CPI = EV/AC \end{aligned}$$

Reading	Schedule	Cost
Ahead / under	$SV > 0$, $SPI > 1$	$CV > 0$, $CPI > 1$
On plan	$SV = 0$, $SPI = 1$	$CV = 0$, $CPI = 1$
Behind / over	$SV < 0$, $SPI < 1$	$CV < 0$, $CPI < 1$

12.4.1 Worked Example 1: the RC-vehicle project at week 8

Baseline budget **BAC = $620 000**. By week 8 the plan said **35 %** of the work should be done; **30 %** is actually done; **$210 000** has been spent.

Live cell — change the planned %, the actual %, or the actual cost.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: evm-example-12
BAC <- 620000
planned_pct <- 0.35
actual_pct  <- 0.30
AC <- 210000

PV <- planned_pct * BAC
EV <- actual_pct  * BAC

SV  <- EV - PV
CV  <- EV - AC
SPI <- EV / PV
CPI <- EV / AC

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(c(PV, EV, AC) / 1000, names.arg = c("PV", "EV", "AC"), col = c("#bdc3c7"
              ylim = c(0, max(PV, EV, AC) / 1000 * 1.2), ylab = "$ thousand",
              main = "Planned vs. earned vs. actual")

sched <- if (SV < 0) "!!behind schedule!!" else "on or ahead of schedule"
cost  <- if (CV < 0) "!!over budget!!" else "on or under budget"
cat(sprintf("Project is %s and %s\n", sched, cost))
cat(sprintf("Planned value (PV)  %s\n", money(PV)))
cat(sprintf("Earned value (EV)  %s\n", money(EV)))
cat(sprintf("Actual cost (AC)  %s\n", money(AC)))
```

```
cat(sprintf("Schedule variance (SV)  %s\n", money(SV)))
cat(sprintf("Cost variance (CV)  %s\n", money(CV)))
cat(sprintf("SPI  %.3f (%.0f cents of schedule per planned dollar)\n", SPI, SPI * 100))
cat(sprintf("CPI  %.3f (%.0f cents of work per dollar spent)\n", CPI, CPI * 100))
```

The project is **behind schedule and over cost** — and this is visible in week 8, while there is still time to act, instead of as a surprise at the end.

12.4.2 Try it yourself: earned value by work package

Each work package has its budget, how much of it was planned to be done by now, how much is actually done, and what it has cost. Work out PV, EV, CV and SV for each package, then the project indices and forecast. Each team picks its own **data set**.

Earned value by work package

SPI and CPI use the project totals. Three decimals for the indices.

Work package	BAC	PV ()$Planned()$	CV ()$EV()$	()$SV()$
Design	80000	100	100	84000
Tooling	150000	80	60	100000
Assembly	60000	50	40	28000
Testing	40000	25	10	5000

- Project SPI: _____ *(to 3 decimal places)*
- Project CPI: _____ *(to 3 decimal places)*
- EAC = BAC / CPI: $_____ *(to 0 decimal places)*

Show the worked solution

PV = BAC x planned %, EV = BAC x actual %, CV = EV - AC, SV = EV - PV. For data set 1 the totals are BAC $330 000, PV $240 000, EV $198 000 and AC $217 000, so SPI = 198 / 240 = **0.825**, CPI = 198 / 217 = **0.912** and EAC = 330 000 / 0.912 = **about $361 700**. Tooling is the package doing the damage: it is both behind (SV = -$30 000) and over (CV = -$10 000).

12.5 Forecasting

Forecast	Formula	Meaning
EAC (Estimate At Completion)	BAC/CPI	Total cost if current cost efficiency holds

Forecast	Formula	Meaning
EAC (schedule too)	$AC + \frac{BAC - EV}{CPI \times SPI}$	If both cost *and* schedule performance persist
EAC (one-off overrun)	$AC + (BAC - EV)$	If the overrun so far was a one-time event
ETC (Estimate To Complete)	$EAC - AC$	Money still needed from here
VAC (Variance At Completion)	$BAC - EAC$	Projected total over/under budget
TCPI (To-Complete Performance Index)	$\frac{BAC - EV}{BAC - AC}$	The cost efficiency the *remaining* work must hit to still land on BAC

12.5.1 Try it yourself: EAC scenarios

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: eac-scenarios-12
BAC <- 620000
EV  <- 186000
AC  <- 210000
PV  <- 217000
CPI <- EV / AC
SPI <- EV / PV

eac_cpi       <- BAC / CPI
eac_oneoff    <- AC + (BAC - EV)
eac_cost_sched<- AC + (BAC - EV) / (CPI * SPI)

etc  <- eac_cpi - AC
vac  <- BAC - eac_cpi
tcpi <- (BAC - EV) / (BAC - AC)

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark

eacs <- c("BAC\n(plan)" = BAC, "CPI\ncontinues" = eac_cpi, "One-off\noverrun" = eac_one
          "Cost and\nschedule" = eac_cost_sched)
par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(eacs / 1000, col = c("#bdc3c7", "#2980b9", "#16a085", "#8e44ad"),
```

```
        ylim = c(0, max(eacs) / 1000 * 1.1), ylab = "$ thousand", main = "Estimate at completion
abline(h = BAC / 1000, lty = 2, lwd = 2, col = "#e74c3c")

msg <- if (tcpi > 1.10) {
  "!!a big ask; the BAC is likely unrealistic now!!"
} else if (tcpi > 1.0) {
  "recoverable, but the team must run leaner than planned"
} else {
  "achievable"
}
cat(sprintf("TCPI %.3f: %s\n", tcpi, msg))
cat(sprintf("CPI  %.3f\n", CPI))
cat(sprintf("SPI  %.3f\n", SPI))
cat(sprintf("EAC, cost efficiency continues  %s\n", money(eac_cpi)))
cat(sprintf("EAC, overrun was a one-off  %s\n", money(eac_oneoff)))
cat(sprintf("EAC, cost AND schedule persist  %s\n", money(eac_cost_sched)))
cat(sprintf("ETC (money still needed)  %s\n", money(etc)))
cat(sprintf("VAC (projected overrun)  %s\n", money(vac)))
```

Which EAC formula should you actually report?

It depends on *why* you are over. If a one-time event caused it (a failed test you have since fixed), $AC + (BAC - EV)$ is fair. If the overrun reflects how the team is fundamentally working (rates, productivity), BAC/CPI is the honest number. If you are *also* behind schedule and recovery will cost overtime, the $CPI \times SPI$ version is the most conservative. Report the one your evidence supports — and say which assumption you made.

12.5.2 Try it yourself: which forecast do you report?

Which EAC formula does each situation call for?

Report the formula your evidence supports.

Categories: BAC / CPI (efficiency continues) / AC + (BAC - EV) (one-off overrun) / AC + (BAC - EV) / (CPI x SPI) (cost and schedule persist)

1. A one-off equipment failure caused the overrun and it has been repaired. _____
2. The overrun reflects how this team works: its rates and productivity will not change. _____
3. The project is over cost and behind schedule, and catching up will need overtime. _____
4. A single failed test cost $30 000 and will not happen again. _____
5. The cost efficiency so far is typical of this team. _____
6. The slip must be recovered with premium-rate crews, so both indices will persist. _____

Answers: 1 AC + (BAC - EV) (one-off overrun); 2 BAC / CPI (efficiency continues); 3 AC + (BAC - EV) / (CPI x SPI) (cost and schedule persist); 4 AC + (BAC - EV) (one-off overrun); 5 BAC / CPI (efficiency continues); 6 AC + (BAC - EV) / (CPI x SPI) (cost and schedule persist).

12.6 Reading the S-curve honestly

- **Spend is not progress.** A project can be on budget (AC PV) and badly behind (EV PV) — it just has not done the work yet.
- **The "watermelon" report** — green on the outside, red on the inside. High-level status shows green because spend matches plan, while the earned value underneath is red.
- **Leading vs. lagging indicators.** SPI and CPI are somewhat lagging. Leading signals — rising defect find-rate, slipping sub-milestones, key staff leaving — warn you earlier. Watch both.

12.6.1 Try it yourself: read the status

What is each project's real condition?

An index above 1 is good. Work out SPI = EV / PV and CPI = EV / AC where needed.

Categories: Ahead and under budget / Behind but under budget / Ahead but over budget / Behind and over budget

1. SPI 1.10, CPI 1.05 _____
2. SPI 0.85, CPI 1.10 _____
3. SPI 1.12, CPI 0.90 _____
4. SPI 0.80, CPI 0.92 _____
5. PV $300k, EV $270k, AC $250k _____
6. PV $200k, EV $230k, AC $260k _____
7. PV $400k, EV $380k, AC $410k _____
8. PV $150k, EV $165k, AC $150k _____

Answers: 1 Ahead and under budget; 2 Behind but under budget; 3 Ahead but over budget; 4 Behind and over budget; 5 Behind but under budget; 6 Ahead but over budget; 7 Behind and over budget; 8 Ahead and under budget.

12.7 Integrated change control

Control only works against a **meaningful baseline**. Every change request goes through the same gate as in Chapter 3: assess the impact on scope, schedule, and cost; approve or reject; and if approved, **re-baseline** so future variance is measured against the new plan — not the old one you can no longer hit. **Configuration management** keeps the product definition (drawings, specs, software versions) in step with the approved baseline.

12.8 Status reporting

The course's **Project Manager Report** and **Team Member Report** templates are the reporting instruments: they roll up progress, cost, schedule, risks, and issues on a fixed cadence. Good reporting is **honest** (report the red while it is still fixable), **consistent** (same metrics every period so trends are visible), and **actionable** (every variance has an owner and a next step). A dashboard that only ever shows green is not a control system — it is a comfort blanket.

12.9 Key Takeaways

- **Control** is a closed loop: **measure → compare to baseline → analyze variance → act**, every reporting period.
- Track **PV** (work scheduled), **EV** (work done), **AC** (money spent), all in budget dollars, against **BAC**.
- $SV = EV - PV$, $CV = EV - AC$, $SPI = EV/PV$, $CPI = EV/AC$. Below 1 / negative = behind / over.
- Forecast with **EAC** (BAC/CPI is the common honest form), **ETC** = $EAC - AC$, **VAC** = $BAC - EAC$, **TCPI** = $(BAC - EV)/(BAC - AC)$.
- **Spend is not progress.** Beware the **watermelon** report; watch **leading** indicators, not just SPI/CPI.
- Keep the **baseline meaningful** through integrated change control and configuration management.
- Report **honestly, consistently, and actionably** — every variance gets an owner.

12.10 Case Study: Meridian R/C — Week 12

CASE STUDY FILE — Week 12: One Good Number Doesn't Make a Good Week

Day 50, and it's time to run the loop this chapter just walked you through — measure, compare, analyze, act — against real Trailblazer numbers instead of a textbook example. Sofia Reyes's weekly report lands with a summary line that looks encouraging at a glance: spend is roughly tracking the plan. You know better than to stop at a glance.

Metric	Value
BAC (Budget At Completion)	$761,300
PV (Planned Value, day 50)	$560,000
EV (Earned Value, day 50)	$520,000
AC (Actual Cost, day 50)	$545,000

You run the four core numbers yourself before the steering call. CV = EV − AC = \$520,000 − \$545,000 = **−\$25,000** — over budget. SV = EV − PV = \$520,000 − \$560,000 = **−\$40,000** — behind schedule. CPI = EV/AC = 520/545 **0.95**. SPI = EV/PV = 520/560 **0.93**. Running BAC/CPI forward gives an EAC of roughly **\$798,000** — a forecast, not a certainty, but not a comfortable one either.

The schedule slip traces to one place: activity H, toy-safety certification, which has sat on the critical path with zero float since the CPM network was first built back in Week 6. The certification lab pushed back its review date, and because H has no float, every day it slips is a day the whole 67-day project finish slips with it.

The cost story is the sharper lesson. Two weeks ago, Marcus Webb talked MCS down from \$145,000 to **\$136,000** for the motor-and-controller package — a genuine \$9,000 win, hard-negotiated, real money, and the whole point of using the competing Paragon quote as leverage. So why is AC still running \$25,000 ahead of EV? Because elsewhere on the project, packaging samples had to move by expedited freight to protect the schedule, and that bill quietly ate the MCS saving and then some. A favourable variance on one activity, read on its own, told the team nothing reliable about the project's overall health — exactly the trap this chapter's "watermelon" warning describes: green on the outside, red once you look underneath.

There is a sliver of good news buried in the bad. At roughly \$798,000, the EAC still sits just under Daniel Cho's \$800,000 charter ceiling from Week 3 — a thin margin, but a margin, assuming nothing else slips before Week 13's closeout.

12.11 Practice Problems

Problem 1: Full EVM snapshot

BAC = \$400 000. At the status date: PV = \$180 000, EV = \$150 000, AC = \$165 000. Compute SV, CV, SPI, CPI, EAC (BAC/CPI), ETC, VAC, TCPI, and state the project's condition in one sentence.

Solution

```
BAC <- 400000; PV <- 180000; EV <- 150000; AC <- 165000
SV <- EV - PV; CV <- EV - AC; SPI <- EV / PV; CPI <- EV / AC
EAC <- BAC / CPI; ETC <- EAC - AC; VAC <- BAC - EAC
TCPI <- (BAC - EV) / (BAC - AC)
cat(sprintf("SV=$%s  CV=$%s  SPI=%.3f  CPI=%.3f\n",
            format(SV, big.mark=" "), format(CV, big.mark=" "), SPI, CPI))
cat(sprintf("EAC=$%s  ETC=$%s  VAC=$%s  TCPI=%.3f\n",
            format(round(EAC), big.mark=" "), format(round(ETC), big.mark=" "),
            format(round(VAC), big.mark=" "), TCPI))
```

```
SV=$-30 000  CV=$-15 000  SPI=0.833  CPI=0.909
EAC=$440 000  ETC=$275 000  VAC=$-40 000  TCPI=1.064
```

Behind schedule (SPI 0.83) and over cost (CPI 0.91); projected to finish about $40 000 over budget, and the remaining work must run at TCPI 1.06 to still hit BAC.

Problem 2: On budget but in trouble

A project reports PV = $500 000, EV = $400 000, AC = $400 000. The sponsor sees "spend exactly matches earned value — CPI = 1.0 — we're fine." Are they?

Answer

Cost efficiency is fine (CPI = 1.0), but **SPI = EV/PV = 0.80** — the project has completed only 80 % of the work it should have by now. It is **20 % behind schedule** and will either finish late or need to spend more (overtime, added staff) to catch up, which will then push CPI below 1. This is the classic watermelon: the cost view is green, the schedule view is red.

Problem 3 (discussion): re-baseline or not?

At week 8 the RC-vehicle project is forecast (EAC) at $700 000 against a $620 000 BAC, mostly because the retailer added a second body colourway (an approved scope change) and the new line's learning curve ran longer than planned. Should you re-baseline?

Discussion points

Split the two causes. The **approved scope change** is a legitimate reason to re-baseline — the extra colourway was authorised, so the budget and schedule should be formally updated and future variance measured against the new plan. The **learning-curve overrun** is *performance* against the plan, not a change to the plan — that stays as variance you must recover or explain. Re-baselining to bury performance problems ("we'll just move the goalposts") destroys the value of the control system; re-baselining for authorised changes keeps it honest.

12.12 Further Resources

Earned Value Management Explained: PV, EV, AC, CPI, SPI: https://www.youtube.com/watch?v=EEIu8-87nAY

Earned Value Analysis Calculations Tutorial (CPI, SPI, CV, SV, EAC, ETC, TCPI): https://www.youtube.com/watch?v=7DLyZjsrG14

Chapter 13

Project Closeout and Termination

13.1 Learning Objectives

By the end of this chapter you will be able to:

1. **State** what project termination provides and distinguish its types — extinction, addition, integration, starvation.
2. **Work through** the seven elements of closeout.
3. **Explain** what commonly prevents effective closeout.
4. **Recognize** the early warning signs of project failure.
5. **Apply** the cancel/continue decision using sunk cost, cost-to-complete, and remaining benefit.
6. **Reconcile** a completed project: planned vs. actual schedule and cost.
7. **Draw** the common lessons from famous project failures.

Before we begin: which is harder — starting a project or finishing one?

Starting has energy and a blank page. Finishing has a tired team, a client eager to move on, and a next project already calling. Closeout is *"the most difficult phase to complete effectively and has the most impact on future projects."*

13.2 Introduction: planning for extinction

A defining feature of a project is a **finite life**. When you plan a project, you are also *"planning for its extinction."* The RC-vehicle prototype project will end — and if it succeeds, its team likely rolls straight into supporting the 100 000-unit production run. That is a real, named way for a project to end, not an afterthought.

Project termination provides for:

- **Acceptance** of the project by the sponsor
- **Completion** of the project records
- **Final revision and issue** of documentation to its as-built condition
- **Retention** of the essential documentation — above all, the **lessons learned**

Unlock the Secrets to Easy Project Closure (Association for Project Management): https://www.youtube.com/watch?v=0HVtpstAT7M

13.3 Types of termination

```
Warning: The `label.size` argument of `geom_label()` is deprecated as of ggplot2 3.5.0
i Please use the `linewidth` argument instead.
```

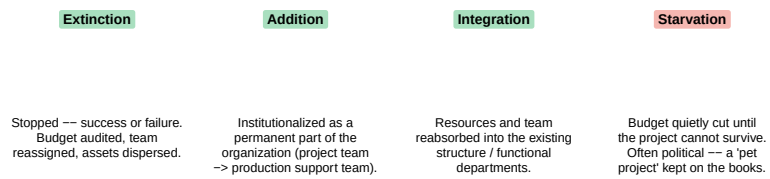


Figure 13.1: Four ways a project ends. Extinction, addition, and integration are normal; starvation is a slow, deliberate neglect.

- **Extinction** — the project is stopped, successfully or not. The final budget is audited, team members get new assignments, material assets are dispersed or transferred per company/contract terms.
- **Addition** — a successful project is made a **formal part of the organization**. An automaker that has launched a new vehicle keeps the project team on as a **production support team**.
- **Integration** — the team and resources are **reabsorbed** into the existing structure. Common but complicated — and organizations often **lose key people** here, who leave for fresh project challenges rather than return to the old department.
- **Starvation** — not really a termination act but *"a willful form of neglect"*: the budget is decreased slowly until the project cannot remain viable. Kept "on the books" for political reasons, a powerful sponsor's pet project, or so it can be reactivated when budgets recover.

13.3.1 Try it yourself: which way did the project end?

Which type of termination is each scenario?

Pick the termination type for each scenario.

Categories: Extinction / Addition / Integration / Starvation

1. A new vehicle's launch team is kept on permanently as the production-support team. _____
2. A failed pilot is shut down, its budget audited and its test rig sold. _____
3. A cancelled R&D team's engineers return to the design department. _____
4. A sponsor's favourite project keeps a token budget so it stays 'active' on paper. _____
5. A successful product line becomes a permanent business unit. _____
6. The team and equipment are reabsorbed into functional departments, and some key people leave. _____
7. The budget is trimmed a little every quarter until the project can no longer deliver. _____
8. The contract ends, assets are transferred per its terms, and the team is reassigned. _____

Answers: 1 Addition; 2 Extinction; 3 Integration; 4 Starvation; 5 Addition; 6 Integration; 7 Starvation; 8 Extinction.

13.4 The seven elements of closeout

Tabl

#	Element	What
1	**Verify scope**	Compare to the original scope statement; resolve discrepancies; confi
2	**Closeout contracts**	Resolve all open issues with the customer; return or destroy propriet
3	**Closeout administration**	Return all loaned personnel; complete performance evaluations.
4	**Conduct a lessons-learned review**	Survey everyone who was on the project; hold the meeting; be critica
5	**Build a project history file**	Document planned vs. actual schedule and cost (with reasons), appr
6	**Create the final project report**	Document overall success, project organization, recommended change
7	**Celebrate success**	Involve everyone who played a role; hold it away from the workplace

13.4.1 What prevents effective closeout

- Getting the project **signed off** discourages the other closeout activities.

- The **assumed urgency** of every project pressures the team to cut corners on the back end.
- Closeout is given **low priority** and easily ignored.
- **Lessons learned** is dismissed as a bookkeeping chore.
- People assume that because every project is unique, **carryover is minimal**.
- People just want to **move on** to the next challenge.

13.4.2 Try it yourself: the seven elements in order

Put the seven closeout elements in order

Number these in the correct order:

- _____ Build a project history file
- _____ Celebrate success
- _____ Closeout administration
- _____ Closeout contracts
- _____ Conduct a lessons-learned review
- _____ Create the final project report
- _____ Verify scope

Answers, in the order listed: 5, 7, 3, 2, 4, 6, 1.

13.5 Project failure

Failure = the project does not deliver what was required, judged against the **acceptance criteria** of the sponsor and customer, and can be triggered by changes in **scope, cost, or time**.

Early warning signs:

- Lack of viable **commercial objectives**
- Lack of sufficient **decision-making authority**
- **Market volatility**
- **Low priority** assigned to the project

When to consider cancelling:

- Costs **exceed the business benefits**
- The project no longer meets **strategic-fit** criteria
- **Deadlines** keep being missed
- **Technology evolves** beyond the project's scope

13.5.1 Worked Example 1: continue or cancel?

A project has already spent **$480 000** (sunk). Finishing it needs another **$260 000**. The product, if delivered, is now worth an estimated **$300 000** in benefit — the market moved while you were building.

Live cell — change the cost to finish or the remaining benefit.

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: cancel-decision-13
sunk            <- 480000     # already spent -- IRRELEVANT to the decision
cost_to_finish  <- 260000
remaining_benefit <- 300000

net_if_continue <- remaining_benefit - cost_to_finish

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", sc

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(c("Sunk\n(ignore)" = sunk, "Cost to\nfinish" = cost_to_finish, "Remaining\nbenefit" = rem
        col = c("#bdc3c7", "#e67e22", ifelse(net_if_continue > 0, "#16a085", "#e74c3c")),
        ylab = "$ thousand", main = "Only the last two bars matter")

if (net_if_continue > 0) {
  cat("CONTINUE: the remaining benefit still beats the remaining cost\n")
} else {
  cat(sprintf("!!CANCEL!!: finishing loses money from here, regardless of the %s already gone\n",
}
cat(sprintf("Sunk cost (ignore it)  %s\n", money(sunk)))
cat(sprintf("Cost to finish  %s\n", money(cost_to_finish)))
cat(sprintf("Benefit if finished  %s\n", money(remaining_benefit)))
cat(sprintf("Net from *here*  %s\n", money(net_if_continue)))
```

The $480 000 is a **sunk cost** — it is spent whether you continue or not, so it must not enter the decision. From here, spending $260 000 to get $300 000 is still worth $40 000. (The sunk-cost fallacy — "we've come too far to stop" — is what keeps doomed projects alive.)

13.5.2 Try it yourself: what counts in the decision?

Which items belong in the continue-or-cancel decision?

Only money that changes depending on what you decide now belongs in the decision.

Categories: Counts in the decision / Ignore: it is sunk

1. $480 000 already spent on tooling _____
2. $260 000 still needed to finish _____
3. The $300 000 benefit expected if the product is delivered _____

4. The software licence fee already paid, which the project cannot resell _____
5. A $40 000 penalty payable only if the contract is cancelled _____
6. $25 000 salvage value of the test rig if sold now _____
7. Salaries paid last quarter _____
8. The non-refundable deposit already paid to the supplier _____

Answers: 1 Ignore: it is sunk; 2 Counts in the decision; 3 Counts in the decision; 4 Ignore: it is sunk; 5 Counts in the decision; 6 Counts in the decision; 7 Ignore: it is sunk; 8 Ignore: it is sunk.

13.5.3 Try it yourself: the cancel decision under uncertainty

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: cancel-decision-13b
sunk           <- 480000
cost_to_finish <- 260000

# The remaining benefit is uncertain -- try a range of market outcomes.
benefit_weak   <- 150000
benefit_below  <- 250000
benefit_plan   <- 300000
benefit_strong <- 400000
benefits <- c(benefit_weak, benefit_below, benefit_plan, benefit_strong)

net <- benefits - cost_to_finish

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark
smoney <- function(x) paste0(ifelse(x < 0, "-", "+"), "$", format(round(abs(x)), big.ma

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
barplot(net / 1000, names.arg = c("Weak", "Below plan", "On plan", "Strong"), col = ife
        main = "Net value of finishing, by outcome")
abline(h = 0, lwd = 2)

cat(sprintf("Break-even benefit: %s\n", money(cost_to_finish)))
cat(sprintf("Sunk cost (ignored)  %s\n", money(sunk)))
for (i in seq_along(benefits)) {
  verdict <- if (net[i] > 0) "CONTINUE" else "!!CANCEL!!"
  cat(sprintf("If benefit = %s  net %s: %s\n", money(benefits[i]), smoney(net[i]), verd
}
```

13.5.4 Try it yourself: continue or cancel, with an uncertain benefit

The sunk cost is given on purpose: it should not change your answers. Each team picks its own **data set**.

Continue or cancel?

Use only future costs and benefits. Expected benefit = probability-weighted average.

- Already spent (sunk): $640 000
- Cost to finish: $180 000
- Benefit if the market is strong: $260 000 (probability 60 %)
- Benefit if the market is weak: $100 000
- Expected remaining benefit: $_____ *(to 0 decimal places)*
- Expected net value of finishing, from here (negative = lose money): $_____ *(to 0 decimal places)*
- Benefit needed for finishing to break even: $_____ *(to 0 decimal places)*

Show the worked solution

The sunk cost never enters. Data set 1: expected benefit = 0.6 x $260 000 + 0.4 x $100 000 = $196 000; less the $180 000 still to spend, finishing is worth **+$16 000** from here. The break-even benefit is simply the cost to finish, $180 000. In data set 2 the expected benefit ($116 000) falls short of the $150 000 needed, so cancel, however much has been spent.

13.6 Lessons from spectacular failures

- **Bank of America "MasterNet"** (1984) — trust-department automation. Over budget by $35M, rising toward $80M; a 1988 failure meant loss of control over billions in trust accounts, all eventually recovered, but the **entire 255-person department was let go** as depositors fled. *Cause: scope and complexity understood far too late.*
- **Future Imagery Architecture** (US spy satellites, 1997) — bid won by a contractor that had never built an imaging satellite; internal estimate already predicted a $4B overrun. **Testing was cut to save money**; a subcontractor's undisclosed manufacturing change caused gyroscope defects undiscovered for three years; costs reached ~$18B by 2005. *Cause: unqualified bidder, cut testing, uncontrolled change.*
- **Canadian Firearms Registry** (1995) — estimated at $119M to implement; a 2002 audit projected ~**$1B**. *Causes: scope not understood, no plan*

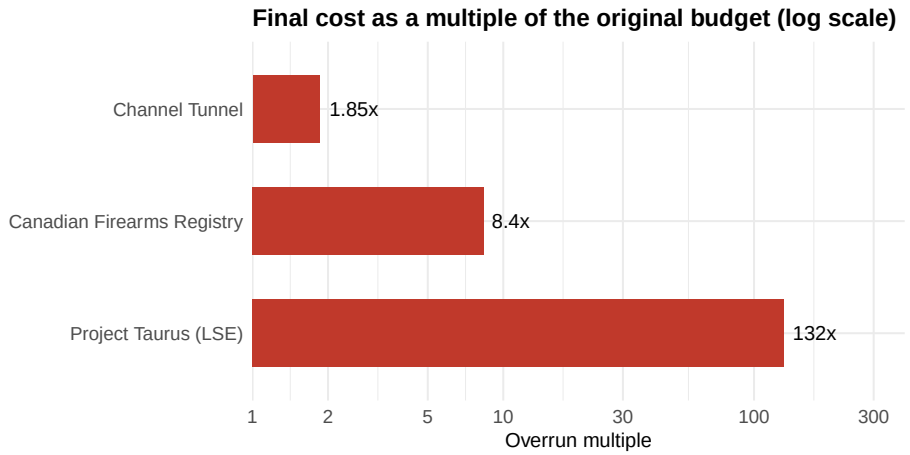


Figure 13.2: Cost overrun as a multiple of the original budget for three well-documented failures. Project Taurus (London Stock Exchange) ran to roughly 132 times its first budget before being abandoned after 11 years.

for inter-agency cooperation, inconsistent data criteria, cost cover-up, no measurement of whether the registry actually worked.

- **Channel Tunnel** (budgeted 1979, opened 1994, one year late) — final cost roughly **double** the original budget.
- **Project Taurus** (London Stock Exchange paperless settlement, 1983) — abandoned after **11 years** at ~**132 times** its original budget after switching to an incompatible US system that clashed with British finance law.

The recurring threads: scope not truly understood; optimism and cost concealment; cutting testing to save money; no plan for the interfaces between organizations; and pressing on past every off-ramp because of sunk cost.

13.6.1 Try it yourself: the final project reconciliation

Show the live R code behind this calculator

The card above re-runs this exact webR code every time you change an input, substituting your values for the variables at the top. Edit the code directly and press **Run Code** to experiment further.

```
#| label: reconciliation-13
# Planned vs actual for the RC-vehicle prototype project at closeout.
planned_weeks <- 30
actual_weeks  <- 34
planned_cost  <- 620000
actual_cost   <- 703000
```

```
planned_scope_items <- 12
delivered_items     <- 12    # all deliverables met

sched_var_pct <- 100 * (actual_weeks - planned_weeks) / planned_weeks
cost_var_pct  <- 100 * (actual_cost - planned_cost) / planned_cost

money <- function(x) paste0(ifelse(x < 0, "-", ""), "$", format(round(abs(x)), big.mark = " ", sc

par(mar = c(5, 5, 3, 1), cex.main = 1.3, cex.lab = 1.2, cex.axis = 1.1)
bp <- barplot(c(Schedule = sched_var_pct, Cost = cost_var_pct), col = ifelse(c(sched_var_pct, cos
              ylim = c(min(0, sched_var_pct, cost_var_pct) * 1.2, max(25, sched_var_pct, cost_var
              ylab = "Variance vs. plan (%)", main = "Variance vs. the 20% threshold")
abline(h = 0, lwd = 2)
abline(h = 20, lty = 2, lwd = 2, col = "#e74c3c")

success <- delivered_items == planned_scope_items && cost_var_pct < 20 && sched_var_pct < 20
if (success) {
  cat(sprintf("%s\n", if (sched_var_pct > 0 || cost_var_pct > 0) "SUCCESS with overruns" else "SU
} else {
  cat("!!TROUBLED!!\n")
}
cat(sprintf("Schedule  %g -> %g wk (%+.1f%%)\n", planned_weeks, actual_weeks, sched_var_pct))
cat(sprintf("Cost  %s -> %s (%+.1f%%)\n", money(planned_cost), money(actual_cost), cost_var_pct))
cat(sprintf("Scope  %g of %g deliverables accepted\n", delivered_items, planned_scope_items))
cat(sprintf("Next step  %s\n", if (success) "document the causes in the history file and lessons
                               else "the lessons-learned review is the priority deliverable"))
```

13.7 Closing out the RC-vehicle project

- **Verify scope** against the signed scope statement: all body options delivered, drop and EMC tests passed, compliance approvals in hand, retailer satisfaction confirmed.
- **Closeout contracts**: settle final invoices with MCS/Paragon and the carbon-fibre supplier; return the customer's proprietary artwork.
- **Administration**: return the borrowed design engineers and millwrights to their functional managers with completed performance reviews.
- **Lessons learned**: the new-line learning curve and the unproven-supplier risk are the two entries that matter most for the next program.
- **Termination by integration**: there is no separate support team; the prototype team is **reabsorbed into production engineering** for the 100 000-unit run.
- **Celebrate** — away from the plant, with everyone who contributed.

13.8 Key Takeaways

- Termination provides **sponsor acceptance, complete records, as-built documentation, and retained lessons learned.**
- Types: **extinction** (stopped), **addition** (becomes permanent), **integration** (reabsorbed), **starvation** (slow deliberate neglect).
- Work the **seven elements**: verify scope, close contracts, close administration, lessons learned, history file, final report, celebrate.
- Closeout fails because sign-off feels like "done", the next project is louder, and lessons learned is treated as bookkeeping. Resist all three.
- **Cancel** when the **cost to complete exceeds the remaining benefit** — the money already spent is **sunk** and irrelevant to that decision.
- Famous failures share the same causes: **scope not understood, cost concealed, testing cut, interfaces unplanned, sunk-cost momentum.**

13.9 Case Study: Meridian R/C — Week 13

CASE STUDY FILE — Week 13: The Trailblazer Ships

Thirteen weeks after Daniel Cho signed a $800,000 charter for a single prototype, you close the file on the Meridian Trailblazer with a phone call from Renata Ford. The drop/durability test (activity G) and the toy-safety certification (activity H) — the two milestones every other decision this quarter was built to protect — both pass. Kestrel Retail Group approves the prototype and awards Meridian the **100,000-unit production contract** for next year's holiday season. The bet you made in Week 1, choosing aluminium over steel and carbon fibre for the body and frame, paid off exactly as planned: durable enough to pass, affordable enough to scale.

This is a **termination by integration**, not addition: there is no separate production-support team to hand the project to, because the project *becomes* production. Priya Nandakumar and Sofia Reyes are reassigned directly into production engineering, carrying the tooling and testing knowledge they built over thirteen weeks straight into the ramp-up. Marcus Webb stays on supplier contracts, now renegotiating MCS's $136,000 motor-and-controller price at 100,000-unit volume instead of prototype volume. You report to Daniel Cho a final Estimate at Completion of ~**$798,000** against his $800,000 ceiling — effectively on budget, a margin of about $2,100 that vindicates the bottom-up number Daniel questioned so hard back in Week 9.

Closing the history file, three lessons learned carry forward into the production program. First, the **$9,000 MCS negotiation saving** from Week 10: using the Paragon quote as leverage, without ever intending to

switch suppliers, protected both the proven vendor relationship and the schedule. Second, **Priya's Week 11 overallocation**, resolved by shifting frame tooling (activity D) inside its own ten days of float rather than adding headcount — proof that a resource conflict is not always a hiring problem. Third, the **Week 2 communication near-miss**, where a spec change missed Kestrel's Vancouver buyer and cost three days and $4,000 — the RACI matrix adopted afterward is the reason the Week 13 sign-off reached every stakeholder the first time. The schedule, the budget, the people, and the lessons all land in the same file: the Trailblazer prototype is finished, and the production program it built starts from everything this one got right.

13.10 Practice Problems

Problem 1: Sunk cost trap

A project has consumed $1.2M of a $1.5M budget. Finishing needs $500 000 (the last $300 000 of budget plus a $200 000 overrun). The delivered product is now worth $350 000. The sponsor says "we've spent $1.2M — we can't stop now." Evaluate.

Solution

```
cost_to_finish <- 500000
remaining_benefit <- 350000
cat(sprintf("Net from here: $%s - $%s = $%s\n",
            format(remaining_benefit, big.mark = " "),
            format(cost_to_finish, big.mark = " "),
            format(remaining_benefit - cost_to_finish, big.mark = " ")))
```

```
Net from here: $350 000 - $5e+05 = $-150 000
```

Finishing turns $500 000 of new spend into $350 000 of value — a $150 000 loss **from today**. The $1.2M is gone either way. Cancel (or de-scope until the remaining cost drops below $350 000). "We can't stop now" is the sunk-cost fallacy.

Problem 2: Which termination type?

1. An automaker keeps its vehicle-launch team on as the permanent production-support team.
2. A cancelled R&D project's engineers return to the design department.
3. A failed pilot is shut down, its budget audited, its test rig sold.
4. A VP's favourite project keeps a $1/year budget so it stays "active" on paper.

Answers

1. **Addition.**
2. **Integration.**
3. **Extinction.**
4. **Starvation.**

Problem 3 (discussion): the team wants to skip lessons learned

The RC-vehicle project delivered late and over budget but the product passed. The exhausted team wants to skip the lessons-learned review and start on the production ramp. As PM, what do you do and why?

Discussion points

This is exactly the moment the review matters most — a troubled project is where the carryover value is highest, and the next program will hit the *same* new-line learning curve and *same* supplier risk. Keep it short and structured, focus on **what went wrong and why** (not blame), capture 2–3 concrete, assigned actions, and put it in the history file before the team disperses and the memory fades. Skipping it guarantees the next project repeats the overrun.

13.11 Further Resources

What Is Project Management? (Association for Project Management): https://www.youtube.com/watch?v=Jk-JwtScIlw

Bibliography